

NovaBASIC User Guide

A Modern BASIC for a Classic CPU

Welcome to NovaBASIC

Every great journey starts with a single command. – *Unknown programmer, circa 1982*

What NovaBASIC Is

NovaBASIC v1.0 is a modernized 6502 BASIC interpreter for the e6502 virtual computer. It is derived from Lee Davison's *Enhanced BASIC 2.22p5* (EhBASIC), one of the cleanest and most complete open-source 6502 BASIC implementations ever written. NovaBASIC keeps the entire EhBASIC core – every numeric function, every string operation, the full DO/LOOP and FOR/NEXT machinery – and adds a hardware command layer that lets you drive the e6502's graphics, sound, file system, and expansion memory directly from BASIC.

The programming model is classic line-numbered BASIC. You type a line with a number at the front, press Return, and it is stored in the program. You type RUN and it executes from the lowest line number. Simple, immediate, and still deeply satisfying.

When NovaBASIC boots, the screen clears to a black background with light-grey text and a dark-grey border. You see:

```
NovaBASIC v1.0
Derived from EhBASIC 2.22p5
xxxxx BASIC bytes free
```

The number shown is the amount of free BASIC program memory. The cursor waits for your first command.

Note

Boot sets the screen background to color 0 (black), the foreground text to color 15 (light grey), and the border to color 11 (dark grey). You can change these at any time with `COLOR fg[,bg[,border]]`.

Runtime Clock Model

Nova VM runs from a single CPU-cycle clock source. By default the target rate is 12 MHz (12,000,000 cycles/second), and video frame timing is derived from that clock at 60 Hz. This keeps VSYNC, raster IRQ behavior, timers, and music sequencing synchronized to one timing model.

You can override runtime behavior with environment variables:

- NOVA_CPU_HZ: target CPU rate in cycles/second (default 12,000,000).
- NOVA_TURBO: if set to 1, true, yes, or on, run unthrottled.
- NOVA_TIMING_LOG: if set, emit periodic timing telemetry for diagnostics.

What You Can Do

Here is what NovaBASIC puts at your fingertips:

- Write and run classic line-numbered BASIC programs in the tradition of the Commodore 64, Apple II, and BBC Micro.
- Draw directly to a **320x200 pixel bitmap** with **16 colors** using PLOT, LINE, CIRCLE, RECT, FILL, and PAINT.
- Animate up to **16 hardware sprites**, each 16x16 pixels and multicolor, with per-sprite shape, position, and flip control.
- Play **6-voice synthesized sound** (two SID chips) with ADSR envelopes and four waveforms – triangle, sawtooth, pulse, and noise.
- Save and load programs and data files with SAVE, LOAD, DIR, and DEL.
- Access **512KB of banked expansion memory** for large data sets, sprite sheets, or music sequences.
- Drop into **6502 assembly language** via SYS and POKE/PEEK for performance-critical inner loops.

How to Read This Guide

This manual is organized so each chapter builds on the previous one. You do not need to read it straight through – jump to whichever section you need – but if you are new to NovaBASIC the order makes sense:

- **Chapters 1–3** Getting started, the edit-run workflow, and the core language: variables, arrays, control flow, operators, and built-in functions.
- **Chapters 4–5** Graphics and sprites; sound and music. These two chapters cover everything you need to build a real game or demo.
- **Chapters 6–7** Expansion memory and low-level access. Read these when your projects outgrow the built-in 64KB address space or when you want to call hand-written assembly routines.
- **Appendices** Complete command reference, memory map, hardware register summary, error codes, and system limits.

Cross-references appear as “see Section X.Y” or “see Appendix A.” Every command name is typeset in **this style** throughout the text.

Your First Program

Let us get something on screen right now. Type each line exactly as shown, pressing Return after each one, then type RUN and press Return.

Try It

```
10 PRINT "HELLO ,_NOVABASIC!"
20 FOR I=1 TO 5
30     PRINT "COUNT:_";I
40 NEXT I
RUN
```

Expected output:

HELLO, NOVABASIC! COUNT: 1 COUNT: 2 COUNT: 3 COUNT: 4 COUNT: 5

Six lines of output, then the cursor returns to the Ready prompt. Congratulations – you have just run your first NovaBASIC program.

Your First Session

Tell me and I forget. Teach me and I remember. Involve me and I learn. – *Benjamin Franklin*

The fastest way to get comfortable with NovaBASIC is to use it. This chapter walks you through everything you need for a productive first session: entering programs, editing them, saving and loading files, and building the habits that will serve you on every project after this one.

The Edit-Run Cycle

NovaBASIC stores your program as an ordered list of numbered lines. When you type a line beginning with a number and press Return, that line is added to the program in the correct position. When you type RUN, execution begins at the lowest line number and proceeds in order.

This is the fundamental loop:

TYPE A LINE -> LIST ->
RUN -> FIX -> RUN

Here is a small program to enter right now:

```
10 PRINT "NOVABASIC_IS_READY"
20 X = 7
30 PRINT "SEVEN_SQUARED_IS_"; X*X
```

Type LIST to see the program back. Type RUN to execute it. Now change line 20 by retyping it with a new value:

```
20 X = 12
```

The old line 20 is replaced. Type RUN again and the result changes. To delete a line entirely, type its number alone and press Return:

```
30
```

Line 30 is gone. LIST confirms it.

Essential Editing Commands

Command	What It Does
LIST	Display the entire program currently in memory.
LIST 20-40	Display only lines 20 through 40 (inclusive).
NEW	Clear the program from memory. Cannot be undone.
RUN	Execute the program starting from the lowest line number.
CONT	Continue execution after a STOP statement.

Warning

NEW erases everything in memory immediately. Save your work with **SAVE** before typing **NEW** if you want to keep it.

Saving and Loading

Programs are saved to and loaded from the virtual file system using four commands:

- **SAVE "name"** Write the current program to disk under the given name. The **.bas** extension is added automatically.
- **LOAD "name"** Load a saved program from disk into memory, replacing anything currently there.
- **DIR** List all saved BASIC programs.
- **DEL "name"** Delete a saved program. This cannot be undone.

Note

Filenames may contain letters (**A--Z**, **a--z**), digits (**0--9**), underscores, hyphens, and dots. Maximum length is 63 characters. Names are case-sensitive on disk.

Here is a complete save-and-reload sequence:

```
SAVE "MYPROG"  
DIR  
NEW  
LOAD "MYPROG"  
RUN
```

DIR shows **MYPROG.bas** in the listing. After **LOAD** and **RUN**, the program executes as if you had just typed it in.

Working Efficiently

Good habits established early make everything easier later.

Tip

- **Number lines by tens.** Use line numbers 10, 20, 30... rather than 1, 2, 3. This leaves room to insert new lines between existing ones without renumbering.
- **Use REM freely while learning.** A comment line costs a little memory but saves a lot of confusion. You can remove them once the code is stable.
- **Build and run in small steps.** Add a few lines, RUN, verify the output, add more. Chasing bugs through a hundred lines you typed without testing is no fun.
- **Group lines by function.** A common convention: 100s for initialisation, 200s for input, 300s for game logic, 800s for output routines, 900s for cleanup and quit. Any scheme that makes sense to you is fine; the important thing is to pick one and use it.

The Round-Trip Exercise

The following exercise takes you through writing, saving, clearing, reloading, and running a program. Every step matters – do not skip any of them.

Try It

```
10 PRINT "ROUND TRIP COMPLETE"
```

Then, at the prompt (no line number):

```
SAVE "ROUNDTrip"  
NEW  
DIR  
LOAD "ROUNDTrip"  
RUN
```

Expected:

- After SAVE: the prompt returns immediately with no error.
- After NEW: LIST shows nothing.
- After DIR: ROUNDTrip.bas appears in the listing.
- After LOAD and RUN: the screen prints ROUND TRIP COMPLETE.

If you see ROUND TRIP COMPLETE at the end, you have mastered the complete NovaBASIC workflow. Everything else is built on top of exactly this.

Language Fundamentals

Simplicity is the ultimate sophistication. – *Leonardo da Vinci*

NovaBASIC inherits a rich language from EhBASIC 2.22p5 and extends it with hardware-specific commands. This chapter covers the core language features you will use in almost every program: variables, arrays, control flow, operators, built-in functions, and direct memory access.

Variables

NovaBASIC has two kinds of variable: **numeric** and **string**.

- **Numeric variables** hold floating-point numbers. Names begin with a letter and may contain letters and digits (e.g. A, SC, X1).
- **String variables** hold text. Names end with a dollar sign (e.g. N\$, NM\$, MS\$).

Variable names are **case-insensitive**, and the first two characters are significant. SC, SCORE, and score all refer to the same numeric variable. For clear type-in programs, use one- or two-character names such as SC for score, LV for lives, and PX/PY for player position. String variables have their own type, so SC and SC\$ are separate variables. Numeric variables default to 0; string variables default to the empty string. You do not need to declare a variable before using it.

When writing shared BASIC framework code, reserve a prefix so games do not collide with it. The Nova Fun & Games common framework uses Z* variables, so game listings should avoid names beginning with Z.

```
10 SC = 0
20 NM$ = "PLAYER_ONE"
30 LV = 3
40 PRINT NM$; "HAS "; LV; "LIVES"
50 SC = SC + 100
60 PRINT "SCORE: "; SC
```

Arrays

Use DIM to declare an array before using it. The default lower bound is 0, so DIM A(10) creates 11 elements: A(0) through A(10).

```
10 DIM A(10)
20 FOR I=0 TO 10
30   A(I) = I * I
40 NEXT I
50 FOR I=0 TO 10
60   PRINT "A("; I; ")="; A(I)
70 NEXT I
```

Two-dimensional arrays work the same way:

```
10 DIM GR(7,7)
20 FOR R=0 TO 7
30   FOR C=0 TO 7
40     GR(R,C) = R*8 + C
50   NEXT C
60 NEXT R
```

Note

String arrays are also supported: DIM LB\$(9) creates ten string slots.

Control Flow

FOR / NEXT Loops

The workhorse of NovaBASIC iteration. The optional STEP clause sets the increment; if omitted, it defaults to 1.

```

10 FOR I = 1 TO 10
20   PRINT I; " ";
30 NEXT I
40 PRINT
50 REM count down with STEP
60 FOR N = 10 TO 1 STEP -1
70   PRINT N; " ";
80 NEXT N

```

IF / THEN / ELSE

```

10 INPUT "ENTER A NUMBER: "; N
20 IF N > 100 THEN PRINT "BIG" ELSE PRINT "SMALL"
30 IF N = 42 THEN PRINT "THE ANSWER"

```

THEN may be followed by a statement or a line number (as a GOTO shorthand). ELSE is optional.

DO / LOOP

DO/LOOP supports four variants for flexible looping:

```

10 X = 1
20 DO
30   PRINT X
40   X = X + 1
50 LOOP WHILE X <= 5

100 Y = 10
110 DO UNTIL Y = 0
120   PRINT Y
130   Y = Y - 1
140 LOOP

```

GOSUB / RETURN

Use subroutines to avoid repeating code. GOSUB branches to a line number and RETURN comes back:

```

10 GOSUB 1000
20 GOSUB 1000
30 END
1000 REM draw border subroutine
1010 PRINT "-----"
1020 RETURN

```

GOTO

GOTO unconditionally jumps to a line number. Use it sparingly; GOSUB/RETURN and structured loops are usually cleaner.

Shorthand Commands

DEC and INC

INC and DEC increment or decrement a variable by 1 without a full assignment statement. They are faster and more readable than `X = X + 1`.

```
10 SC = 0
20 INC SC : INC SC : INC SC
30 PRINT "SCORE:␣"; SC
40 DEC SC
50 PRINT "AFTER␣DEC:␣"; SC
```

WAIT

`WAIT addr, mask [, xor]` busy-waits (polls in a tight loop) until a memory location satisfies a bit condition:

`(PEEK(addr) XOR xor) AND mask <> 0`

The *xor* parameter defaults to 0 if omitted. `WAIT` is useful for polling hardware status registers:

```
10 REM Wait for timer IRQ pending (bit 0 of $BA41)
20 WAIT $BA41, 1
```

```
10 REM Wait for a key press (non-zero at $A00F)
20 WAIT $A00F, $FF
```

Warning

`WAIT` blocks the BASIC interpreter completely until the condition is met. If the condition is never satisfied, the program hangs. Use `STOP` (Ctrl+C) to break out.

RESET

`RESET` performs a full hardware reset: it restarts the CPU, returns all custom chips to their initial state, clears display/video state, stops audio, and restarts the BASIC interpreter from cold start. All variables and the current program are lost.

```
10 PRINT "RESETTING␣IN␣3..."
20 FOR I=3 TO 1 STEP -1 : PRINT I : NEXT I
30 RESET
```

Operators

Operator	Type	Description
+	Arithmetic	Addition
-	Arithmetic	Subtraction or unary negation
*	Arithmetic	Multiplication
/	Arithmetic	Division

Operator	Type	Description
$\wedge$	Arithmetic	Exponentiation ($2^8 = 256$)
$=$	Comparison	Equal to
$<$	Comparison	Less than
$>$	Comparison	Greater than
$\leq$	Comparison	Less than or equal to
$\geq$	Comparison	Greater than or equal to
$<>$	Comparison	Not equal to
AND	Logical	Logical (bitwise) AND
OR	Logical	Logical (bitwise) OR
NOT	Logical	Logical (bitwise) NOT
EOR	Logical	Exclusive OR
$\ll$	Bit	Left shift (LSHIFT)
$\gg$	Bit	Right shift (RSHIFT)
BITSET	Bit	Set bits by mask at an address
BITCLR	Bit	Clear bits by mask at an address
BITTGL	Bit	Toggle bits by mask at an address
BITTST(a,m)	Bit	Test bits by mask at an address

Built-in Functions

Numeric Functions

Function	Description
INT(x)	Truncate to integer (towards zero)
ABS(x)	Absolute value
SGN(x)	Sign: -1, 0, or 1
SQR(x)	Square root
RND(1)	Pseudo-random number in $[0, 1)$
LOG(x)	Natural logarithm
EXP(x)	e^x
SIN(x)	Sine (radians)
COS(x)	Cosine (radians)
TAN(x)	Tangent (radians)
ATN(x)	Arctangent (radians)
PI	The constant pi 3.14159
TWOPI	The constant 2pi 6.28318
MAX(a,b)	Larger of two values
MIN(a,b)	Smaller of two values
PEEK(addr)	Read a byte from memory address <i>addr</i>
DEEK(addr)	Read a 16-bit word from address <i>addr</i>
FRE(0)	Free BASIC program memory in bytes
POS(0)	Current cursor column position

String Functions

Function	Description
LEN(<i>s</i>)	Number of characters in the string
CHR(<i>n</i>)	Character whose ASCII code is <i>n</i>
ASC(<i>s</i>)	ASCII code of the first character
STR(<i>n</i>)	Convert number to string
VAL(<i>s</i>)	Convert string to number
LEFT(<i>s</i> , <i>n</i>)	Leftmost <i>n</i> characters
RIGHT(<i>s</i> , <i>n</i>)	Rightmost <i>n</i> characters
MID(<i>s</i> , <i>p</i> , <i>n</i>)	Characters starting at position <i>p</i> * <i>n</i>
UCASE(<i>s</i>)	Convert to uppercase
LCASE(<i>s</i>)	Convert to lowercase
HEX(<i>n</i>)	Hexadecimal string representation of <i>n</i>
BIN\$(<i>n</i>)	Binary string representation of <i>n</i>

Memory Access

NovaBASIC gives you direct read and write access to the 6502 address space. This is useful for reading hardware registers, patching values, and interfacing with assembly routines.

- **PEEK(addr)** Read one byte (0–255) from address *addr*.
- **POKE addr, val** Write one byte to address *addr*.
- **DEEK(addr)** Read a 16-bit little-endian word from *addr* and *addr*+1.
- **DOKE addr, val** Write a 16-bit little-endian word.
- **VARPTR(v)** Return the memory address of variable *v*. Useful when passing variable addresses to assembly routines.

```

10 REM read and print two bytes at $0300
20 PRINT PEEK(768); PEEK(769)
30 REM write a counter value
40 POKE 768, 42
50 PRINT PEEK(768)
```

Warning

Writing to the wrong address can crash the virtual machine or corrupt the BASIC interpreter. Know what you are writing to before you POKE into system areas. See the memory map in Appendix memmap for safe zones.

Cursor Control

The text cursor position is set with LOCATE *x*, *y* where *x* is the column (0–79) and *y* is the row (0–49).

Cursor **visibility** is controlled via bit 0 of the VGC register at \$A00A:

```

10 POKE $A00A, 0 : REM hide cursor
20 LOCATE 10, 5 : PRINT "SCORE: 0"
30 POKE $A00A, 1 : REM show cursor
```

Bit 0 set enables the blinking cursor; bit 0 clear hides it. The hardware reset default is hidden, so runtimes should explicitly enable the cursor only while waiting for text input.

NovaBASIC manages this automatically for the normal command prompt and INPUT: the cursor is visible while BASIC is waiting for line input, and hidden while a program is executing. Programs

that need manual control can still write \$A00A directly.

Style Tips

Tip

- **One idea per line.** Statements can be chained with colons (:), but a single clear statement per line is easier to read and debug.
- **Group by function.** Put constants and configuration near the top (lines 10–90), input routines in the 100s, game logic in the 200s–500s, display routines in the 600s, cleanup in the 900s.
- **Name things meaningfully within two characters.** LV is a good short name for lives, SC for score, PX/PY for player position, and DX/DY for direction or velocity. Avoid long-looking names because only the first two characters identify the variable.
- **Define constants up front.** 10 ML=5 : SS=2 at the top of the program means you tune the game by changing one line, not hunting through hundreds.

Putting It Together

Try It

Enter and run this program. It builds an array of random values and prints them in indexed form.

```
10 DIM A(5)
20 FOR I=1 TO 5
30   A(I) = INT(RND(1) * 100)
40 NEXT I
50 FOR I=1 TO 5
60   PRINT "A(";I;") = ";A(I)
70 NEXT I
```

Expected: Five lines printed, each showing an index and a random integer between 0 and 99. The values differ every time you RUN.

Try modifying the program: change 100 to 10 to get single-digit values, or change 5 to 20 to see a longer array. The structure stays the same; only the constants change.

Graphics and Sprites

A blank screen is a canvas waiting to be claimed.

– e6502 Virtual Computer Design Notes

NovaBASIC gives you direct access to a 320x200 pixel bitmap and a hardware sprite layer. Drawing commands operate on 16 colors; sprites add independently positioned, independently animated 16x16 objects on top of or behind the bitmap and text layers. This chapter covers the full graphics pipeline from mode selection to collision detection.

Display Modes

The virtual display has two independent layers: a text layer and a graphics bitmap layer. `MODE` selects how they are composited.

Mode	Description
0	Text only. The graphics bitmap is not rendered.
1	Graphics over text. Bitmap is drawn on top of text characters.
2	Text over graphics. Text characters are drawn on top of the bitmap.
3	Graphics and sprites only. No text layer is rendered.

The typical starting sequence for any graphics program is:

```
10 MODE 1
20 GCLS
30 GCOLOR 7
```

`MODE 1` activates pixel rendering. `GCLS` clears the bitmap to transparent (color 0). `GCOLOR` sets the active drawing color for all subsequent drawing commands.

Note

Color 0 is transparent in the graphics layer. Setting a pixel to color 0 with `PLOT 0` or `FILL` erases it, letting the text layer or background show through. This is equivalent to `UNPLOT`.

To return to plain text output, switch back to `MODE 0`. You do not need to clear the bitmap when switching modes; the pixel data is preserved and will reappear if you switch back to `MODE 1` or `MODE 2`.

Drawing Commands

All drawing commands use the color set by `GCOLOR`. Coordinates must fall within the screen boundaries of $X = 0\text{--}319$ and $Y = 0\text{--}199$; pixels outside that range are silently clipped and no error is raised.

Command reference

Command	Description
<code>GCOLOR c</code>	Set the active drawing color. c is 0–15. If $c = 0$, NovaBASIC uses the current text foreground color instead of transparent.
<code>GCLS</code>	Clear the entire graphics bitmap to transparent (color 0). Does not affect the text layer.
<code>PLOT x,y</code>	Set the pixel at (x,y) to the current drawing color.

Command	Description
UNPLOT x,y	Set the pixel at (x,y) to transparent (color 0), effectively erasing it.
LINE x_0,y_0,x_1,y_1	Draw a straight line from (x_0,y_0) to (x_1,y_1) in the current drawing color.
RECT x_0,y_0,x_1,y_1	Draw a rectangle outline. (x_0,y_0) is the top-left corner; (x_1,y_1) is the bottom-right corner.
FILL x_0,y_0,x_1,y_1	Draw a solid filled rectangle using the same corner convention as RECT.
CIRCLE cx,cy,r	Draw a circle outline centered at (cx,cy) with radius r pixels.
PAINT x,y	Flood-fill from seed point (x,y) , replacing all connected pixels of the same color with the current drawing color.

A drawing example

The following program draws a diagonal cross, a circle, and then fills the circle interior:

```

10 MODE 1 : GCLS
20 GCOLOR 9
30 LINE 0,0,319,199
40 LINE 319,0,0,199
50 GCOLOR 14
60 CIRCLE 160,100,60
70 GCOLOR 10
80 PAINT 160,100
90 VSYNC

```

Line 30–40 draws a white cross from corner to corner. Lines 60–80 add a yellow circle outline and then flood-fill the interior with green. VSYNC on line 90 holds the image for one frame before the program ends; without it the display may update before you see the result.

Tip

PAINT stops at pixel boundaries of a different color. Make sure the circle or region you want to fill has no gaps, otherwise the fill will leak out into the surrounding area. If in doubt, draw the boundary in one step and fill immediately after.

Animation with VSYNC

The virtual display runs at 60 Hz. VSYNC suspends program execution until the start of the next video frame. One VSYNC call therefore consumes exactly one frame period (16.7 ms). This is the correct tool for controlling animation speed.

A minimal animation loop that moves a point across the screen:

```

10 MODE 1 : GCLS : GCOLOR 11
20 X = 0 : Y = 100
30 VSYNC

```

```

40 UNPLOT X, Y
50 X = X + 2
60 IF X > 319 THEN X = 0
70 PLOT X, Y
80 GOTO 30

```

The pattern is always: wait for VSYNC, erase the old position, update coordinates, draw the new position. Erasing before moving eliminates the ghost trail that builds up if you draw without erasing.

Tip

For smooth movement, do all erase operations for a frame, update all positions, and do all draw operations – all within a single VSYNC period. Never call VSYNC between the erase and redraw steps for the same object; that produces a one-frame flicker every cycle.

Sprites

Sprites are hardware-accelerated 16x16 pixel objects that move independently of the bitmap. NovaBASIC supports 16 sprites (indices 0–15), each with its own shape, position, priority, and flip state. Sprites do not modify the bitmap; they are composited at render time.

Enabling and positioning sprites

A sprite must be enabled before it becomes visible:

```

10 SPRITE 0, ON
20 SPRITE 0, 160, 100

```

SPRITE *n*,ON activates sprite *n*. SPRITE *n*,*x*,*y* sets its screen position. Sprite positions use Nova canvas coordinates, matching the 320x200 graphics commands: X = 0–319 and Y = 0–199 are the visible canvas pixels. A 16x16 sprite is fully visible when its top-left position is X = 0–304 and Y = 0–184. Positions outside that range are accepted and clipped; sprites do not draw into the hardware border.

To hide a sprite, use SPRITE *n*,OFF. This makes the sprite invisible without erasing its shape data. You can re-enable it later with SPRITE *n*,ON and it will reappear at its last recorded position.

Sprite priority

Priority controls which layer a sprite is drawn on:

Priority	Layer position
0	Behind all layers (below text and graphics)
1	Between the text and graphics layers
2	In front of all layers (above text and graphics)

Priority is set via the MCP sprite tools when building shapes interactively; it can also be arranged by designing your program so that background sprites are enabled first and foreground sprites last.

Defining sprite pixels with SPRITEDATA

Each sprite is 16 pixels wide by 16 pixels tall. Pixel data is loaded one row at a time using SPRITEDATA:

SPRITEDATA *n*, *row*, *b1*, *b2*, *b3*, *b4*, *b5*, *b6*, *b7*, *b8*

- *n* is the sprite index (0–15).
- *row* is the row to define (0–15, top to bottom).
- *b1*–*b8* are eight byte values (0–255).

Each byte encodes *two* pixels. The high nibble (upper four bits) is the left pixel; the low nibble (lower four bits) is the right pixel. Color 0 is transparent; colors 1–15 are the standard 16-color palette. With eight bytes per row and two pixels per byte, each row is exactly 16 pixels wide.

How sprite byte encoding works

The diagram below shows one row of sprite data – 8 bytes producing 16 pixels. Each byte is split into two 4-bit nibbles, each selecting a color index:

In the example above, \$BB (decimal 187) fills both nibbles with color index B (11 = cyan), producing two solid cyan pixels. \$0B has a transparent left pixel and a cyan right pixel. \$00 leaves both pixels transparent.

The full 16x16 sprite is 16 such rows stacked vertically, each defined by a separate SPRITEDATA call. The complete shape occupies 128 bytes (8 bytes x 16 rows):

Each pair of columns shares one byte. The vertical lines mark byte boundaries. Colored cells are non-transparent pixels; gray cells are transparent (color 0).

The following example defines a simple 16x16 diamond shape in color 11 (cyan) and displays it:

```
10 REM DEFINE A DIAMOND SPRITE
20 SPRITEDATA 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
30 SPRITEDATA 0, 1, 0, 0, 0, 11, 0, 0, 0, 0, 0, 0
40 SPRITEDATA 0, 2, 0, 0, 177,177, 0, 0, 0, 0, 0, 0
50 SPRITEDATA 0, 3, 0, 187, 187,187,187, 0, 0, 0, 0, 0
60 SPRITEDATA 0, 4, 0, 187, 187,187,187, 0, 0, 0, 0, 0
70 SPRITEDATA 0, 5, 0, 0, 177,177, 0, 0, 0, 0, 0, 0
80 SPRITEDATA 0, 6, 0, 0, 0, 11, 0, 0, 0, 0, 0, 0
90 SPRITEDATA 0, 7, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
100 REM ROWS 8-15 REMAIN TRANSPARENT (NO SPRITEDATA = NO CHANGE)
110 SPRITE 0, ON
120 SPRITE 0, 152, 92
130 VSYNC
```

Note

Any row not explicitly defined by SPRITEDATA retains its previous pixel data. If you are reusing a sprite slot for a new shape, define all 16 rows (or clear the slot first). Rows you intentionally leave all-zero produce a fully transparent row.

The bytes in lines 30–80 use decimal notation. Working with hex notation is often more readable: 0xBB = decimal 187, which encodes color 11 in both nibbles (solid cyan on both pixels of that byte).

In NovaBASIC you can write hex literals directly in expressions using `&HBB` notation.

Sprite Collision Detection

NovaBASIC provides two collision functions that report when sprites overlap each other or touch non-transparent pixels on the background bitmap.

Function	Returns
SPRCOLL	16-bit mask of sprites that collided with another sprite.
SPRBG	16-bit mask of sprites that touched a non-transparent graphics pixel.

Both functions return an integer bitmask. Bit `k` being set means sprite `k` is involved in that collision type. Use `AND` with the appropriate bit to test for a specific sprite.

Warning

`SPRCOLL` and `SPRBG` clear their latched masks after reading them. Read each function once per frame and store the result in a variable. If you call the same function a second time in the same frame you will get zero, missing collisions that occurred between reads.

A practical collision loop pattern:

```
100 VSYNC
110 C = SPRCOLL
120 B = SPRBG
130 IF C <> 0 THEN GOSUB 500
140 IF B <> 0 THEN GOSUB 600
150 REM UPDATE POSITIONS HERE
160 GOTO 100
```

Lines 110–120 read both registers once and store them. Lines 130–140 branch to handler routines only if a collision has occurred. All position updates happen after the collision check so that the same frame's register values are used consistently.

To test whether sprite 2 participated in any sprite-sprite collision, check bit 2 of the `SPRCOLL` result:

```
200 C = SPRCOLL
210 IF (C AND 4) <> 0 THEN PRINT "SPRITE_2_COLLIDED"
```

Color Palette

NovaBASIC starts with a 16-color palette inspired by the Commodore 64. All fixed-index graphics, text, sprite, background, and border colors use the same indices. `RegPaletteMode` at `$AOE9` selects the fixed palette: write 0 for the default C64/Nova palette or 1 for IBM EGA colors.

Index	Color	Index	Color
0	Black	8	Orange
1	White	9	Brown
2	Red	10	Light Red
3	Cyan	11	Dark Grey
4	Purple	12	Medium Grey
5	Green	13	Light Green
6	Blue	14	Light Blue
7	Yellow	15	Light Grey

The background color defaults to 0 (black), the text foreground to 15 (light grey), and the border to 11 (dark grey). Use `COLOR fg[,bg[,border]]` for text, background, and border colors. Text cells store both foreground and background in Color RAM; **REVERSE** swaps them for subsequent output, and **FLASH** marks subsequent text cells as blinking.

The EGA mode keeps the same 0–15 color indices but changes the RGB values to the IBM EGA 16-color palette. This is useful for DOS EGA asset ports:

```
POKE $A0E9,1 : REM IBM EGA palette
POKE $A0E9,0 : REM C64/Nova palette
```

`RegPaletteMode` controls the fixed VGC palette used by text, bitmap graphics, sprites, background, and border.

The Copper (Raster Effects)

The copper is a per-pixel register-write system inspired by the Amiga’s Copper coprocessor. It lets you change display registers at precise screen positions, enabling effects like color gradient backgrounds, split-screen modes, and parallax scrolling – all without any CPU involvement.

How it works

The VGC stores up to 128 independent copper *lists*. Each list is a program of up to 256 events, each specifying:

- A screen position (X = 0–319, Y = 0–199).
- A target register.
- A value to write to that register.

When copper is enabled, the renderer checks for copper events at every pixel position. When it reaches a pixel that has a scheduled event, the register write fires immediately and affects all subsequent pixels on that frame.

Writable registers

The copper can write to four VGC core registers and the full sprite register block:

Register	Address	Effect
RegMode	\$A000	Change display mode mid-screen
RegBgCol	\$A001	Change background color mid-screen
RegScrollX	\$A005	Shift horizontal scroll offset
RegScrollY	\$A006	Shift vertical scroll offset
RegScrollCtl	\$A0EA	Bit 0 = RegScrollX bit 8, bit 1 = apply scroll to graphics, bit 2 = apply scroll to text
A040–A0BF	(sprite regs)	All sprite register fields (see below)

COPPER BASIC keyword

The COPPER keyword provides direct access to the copper system:

Syntax	Purpose
COPPER ADD x, y, BGCOL, value	Add event: set background color at position.
COPPER ADD x, y, MODE, value	Add event: set display mode at position.
COPPER ADD x, y, SCROLLX, value	Add event: set horizontal scroll at position.
COPPER ADD x, y, SCROLLY, value	Add event: set vertical scroll at position.
COPPER ADD x, y, \$A0EA, value	Add event: set scroll control at position.
COPPER ADD x, y, SPRX(n), value	Set sprite <i>n</i> X low byte at position.
COPPER ADD x, y, SPRXH(n), value	Set sprite <i>n</i> X high byte at position.
COPPER ADD x, y, SPRY(n), value	Set sprite <i>n</i> Y position at position.
COPPER ADD x, y, SPRSHAPE(n), value	Set sprite <i>n</i> shape slot at position.
COPPER ADD x, y, SPRFLAGS(n), value	Set sprite <i>n</i> flags at position.
COPPER ADD x, y, SPRPRI(n), value	Set sprite <i>n</i> priority at position.
COPPER CLEAR	Remove all events from the current target list.
COPPER ON	Enable copper execution each frame.
COPPER OFF	Disable copper execution.
COPPER LIST <i>n</i>	Set target list to <i>n</i> (0–127). Subsequent ADD/CLEAR edit this list.
COPPER LIST END	Reset target list back to the active list.
COPPER USE <i>n</i>	Switch the active list to <i>n</i> at the next vblank.

For sprite register names, *n* is the sprite index (0–15). Each name maps to the absolute address \$A040 + *n**8 + *field_offset*. All eight sprite register fields, including **TransColor** (+7), are copper-writable.

If an event at the same position for the same register already exists, the value is replaced. Each list holds up to 256 events.

Multiple lists. The VGC stores 128 copper lists (0–127). By default everything operates on list 0. Use **COPPER LIST *n*** to direct subsequent **ADD/CLEAR** commands to list *n*. Use **COPPER USE *n*** to tell the renderer to switch to list *n* at the next vertical blank – this avoids tearing because the swap

happens atomically between frames. Editing a list and swapping the active pointer are independent, so you can build one list while the renderer displays another (double buffering).

Sprite multiplexing with the copper

The VGC has 16 hardware sprites, but the copper can make them appear as many more by rewriting sprite registers between rows. The technique is called **sprite multiplexing**: after a sprite's row of pixels has been drawn, the copper repositions it further down the screen with a different shape slot, effectively reusing the same hardware sprite for a second (or third) object.

The basic recipe:

1. Set initial sprite positions and shapes for the top of the screen.
2. Use `COPPER ADD` to rewrite `SPRY(n)` and `SPRSHAPE(n)` at scanlines between the rows of sprites, moving the sprite to its next position and swapping its shape.
3. At scanline 0 of the next frame, restore the original values so the top-of-screen sprites reappear.

For example, to display sprite 0 at both Y=20 and Y=120 with different shapes:

```
10 REM Set initial position (top instance)
20 SPRITE 0, 100, 20
30 SPRITESHape 0, 0
40 SPRITE 0, ON
50 REM Copper: at scanline 70, reposition to Y=120, shape 1
60 COPPER ADD 0, 70, SPRY(0), 120
70 COPPER ADD 0, 70, SPRSHAPE(0), 1
80 REM Restore at scanline 0 for next frame
90 COPPER ADD 0, 0, SPRY(0), 20
100 COPPER ADD 0, 0, SPRSHAPE(0), 0
110 COPPER ON
```

The demo program `docs/programs/sprite_multiplex.bas` shows a complete working example with multiple multiplexed sprites.

Low-level copper control via POKE

You can also control the copper by writing directly to the VGC command registers (A010–A016) and then writing the command byte to \$A010.

Code	Command
\$1B	Copper Add – add an event to the target list. P0/P1 = X (16-bit), P2 = Y, P3/P4 = register (0–15 or A000–A00F), P5 = value.
\$1C	Copper Clear – remove all events from the target list.
\$1D	Copper Enable – start executing the active copper list each frame.
\$1E	Copper Disable – stop executing the copper program.
\$20	Copper List – set target list. P0 = list index (0–127).

Code	Command
\$21	Copper Use – set pending active list. P0 = list index (0–127). Takes effect at next vblank.
\$22	Copper List End – reset target list back to the active list.

Example: background color gradient

The following program creates a vertical color gradient by scheduling a background color change at the start of each group of rows:

```
10 COPPER CLEAR
20 FOR I = 0 TO 12
30     COPPER ADD 0, I * 15, BGCOLOR, I + 2
40 NEXT I
50 COPPER ON
```

The loop on lines 20–40 adds 13 events, each changing the background color at Y positions 0, 15, 30, ..., 180 to successive color indices. Line 50 enables the copper. Each frame the background will display as a gradient of bands.

Tip

Copper events fire at per-pixel granularity. Setting $X = 0$ for all events produces clean horizontal bands. Setting different X values within the same row creates vertical split effects.

Note

Each copper list holds up to 256 events. Events are sorted by position automatically. Multiple events at the same position are applied in register-index order. The VGC stores 128 lists; all data is host-side and consumes no 6502 address space.

Sprite Shape Slots

The VGC provides 256 shape slots (indices 0–255), each storing a 16x16 pixel sprite image (128 bytes). By default, sprite n uses shape slot n . The **SPRITESHAPE** command lets you reassign which slot a sprite renders:

SPRITESHAPE n , $shape$

- n is the sprite index (0–15).
- $shape$ is the shape slot (0–255) to assign to that sprite.

Multiple sprites can share the same shape slot, so you can display several identical objects without duplicating pixel data. Shape slots beyond 15 must be populated via **DMACOPY** into sprite shape memory (space 4) or register-level commands.

SPRITESHAPE writes to the memory-mapped sprite register at $\$A044 + n*8$ (the shape field in the sprite register block).

Sprite Registers

Each of the 16 sprites has an 8-byte register block starting at \$A040. The register layout per sprite is:

Offset	Field	Description
+0	XLo	X position, low byte
+1	XHi	X position, high byte
+2	YLo	Y position (unsigned byte)
+3	YHi	Reserved; reads as 0
+4	Shape	Shape slot index (0–255)
+5	Flags	Bit 0: horizontal flip, Bit 1: vertical flip, Bit 7: enable
+6	Priority	0 = behind all, 1 = between layers, 2 = in front
+7	TransparentColor	Per-sprite transparent color index (0–15). Default 0 (black is transparent). Values >15 disable transparency (all 16 colors opaque).

Sprite 0 occupies A040–A047, sprite 1 occupies A048–A04F, and so on up to sprite 15 at A0B8–A0BF. These registers can be written directly with POKE or targeted by the copper for per-scanline updates.

SPRITESET – Per-Register Sprite Control

SPRITESET provides direct write access to any field in a sprite’s 8-byte register block. The write is synchronised to the vertical blank interval, so no mid-frame visual glitching occurs.

SPRITESET *sprite*, *field*, *value*

- **sprite** is the sprite index (0–15).
- **field** is the register offset within the sprite block (0–7).
- **value** is the byte to write (0–255).

The eight field offsets map directly to the sprite register layout described in Section [sprite-registers](#):

Field	Register
0	X position, low byte
1	X position, high byte
2	Y position (unsigned byte)
3	Reserved; reads as 0
4	Shape slot index (0–255)
5	Flags (bit 0: H-flip, bit 1: V-flip, bit 7: enable)
6	Priority (0=behind all, 1=between layers, 2=in front)
7	Transparent color index (0–15; values >15 disable transparency)

For example, to set sprite 3’s transparent color to color index 5:

```
10 SPRITESET 3, 7, 5
```

Note

Sprite attribute writes are hardware-buffered and published at the next frame boundary, so BASIC code can write them immediately without manually waiting for vblank. This ensures the change takes effect at a frame boundary and avoids partial-frame artifacts when changing position or shape fields.

Reading Sprite Position

`SPRITEX(n)` and `SPRITEY(n)` return the current screen position of sprite `n` by reading directly from the sprite register block.

Function	Returns
<code>SPRITEX(n)</code>	Current X position of sprite <code>n</code> as an unsigned 16-bit Nova canvas coordinate.
<code>SPRITEY(n)</code>	Current Y position of sprite <code>n</code> as an unsigned 8-bit Nova canvas coordinate.

`SPRITEX` combines the X low/high registers. `SPRITEY` returns the Y low byte; the Y high register is reserved.

```
10 SPRITE 0, 160, 80
20 PRINT SPRITEX(0), SPRITEY(0)
```

This prints 160 and 80, confirming the position set on line 10.

Try It Now

Try It

Type and run the following program to see `MODE`, `GCLS`, `GCOLOR`, `RECT`, `CIRCLE`, and `PAINT` working together:

```
10 MODE 1 : GCLS : GCOLOR 10
20 RECT 10, 10, 309, 189
30 GCOLOR 14 : CIRCLE 160, 100, 50
40 GCOLOR 12 : PAINT 160, 100
```

Expected result: a green rectangle border frames the screen; inside it a yellow circle outline encloses a solid red filled region.

Try modifying `GCOLOR` values (1–15) and the `CIRCLE` radius to explore the coordinate system. Then add a second `CIRCLE` call on a new line and re-run to see both circles on the same canvas.

NovaBASIC supports up to 16 hardware sprites, each 16x16 pixels with 4-bit color (16 colors per sprite). Sprites are rendered by the VGC independently of the text and graphics layers, making them ideal for game characters, projectiles, and other moving objects.

Enabling and Positioning

Each sprite has an index from 0 to 15. To use a sprite, enable it and set its position:

```
SPRITE 0,ON
SPRITE 0,100,80
```

The first command enables sprite 0. The second sets its position to x=100, y=80. Sprite coordinates use the same 320x200 Nova canvas as the graphics commands. X=0 and Y=0 are the top-left visible canvas pixel. A 16x16 sprite is fully on screen when its top-left position is X=0-304 and Y=0-184. Positions outside that range are allowed, but the sprite is clipped by the visible canvas and never draws into the hardware border.

Defining Shapes

Sprites use shape slots (0-255) stored in dedicated shape RAM. Define pixel data with **SPRITEDATA**:

Try It

```
10 REM SIMPLE ARROW SPRITE
20 SPRITEDATA 0,0,"0000000100000000"
30 SPRITEDATA 0,1,"0000001110000000"
40 SPRITEDATA 0,2,"0000011111000000"
50 SPRITEDATA 0,3,"0000000100000000"
60 SPRITEDATA 0,4,"0000000100000000"
70 SPRITE 0,ON
80 SPRITESHape 0,0
90 SPRITE 0,160,100
```

Each **SPRITEDATA** call defines one row (0-15) of a shape slot. The hex string encodes 16 pixels, each being a color index 0-F. Color 0 is transparent by default.

Priority Layers

Sprites render at one of three priority levels:

Priority	Constant	Rendering Order
0	Behind	Behind text and graphics
1	Between	Between background and text
2	Front	In front of everything

```
SPRITE 0,PRIORITY,2
```

Collision Detection

The VGC detects two types of collisions each frame:

- Sprite-sprite: Two enabled sprites overlap on non-transparent pixels
- Sprite-background: A sprite overlaps a non-zero graphics pixel

Read full 16-sprite collision masks with the `SPRCOLL` and `SPRBG` functions. Bit 0 is sprite 0, bit 8 is sprite 8, and bit 15 is sprite 15:

```
10 VSYNC
20 C = SPRCOLL
30 B = SPRBG
40 IF (C AND 1) <> 0 THEN PRINT "SPRITE_0_HIT_ANOTHER_SPRITE"
50 IF (C AND 256) <> 0 THEN PRINT "SPRITE_8_HIT_ANOTHER_SPRITE"
60 IF (B AND 32768) <> 0 THEN PRINT "SPRITE_15_HIT_BACKGROUND"
70 GOTO 10
```

For interrupt-driven sprite-sprite collision handling from BASIC, set a handler with `ON SPRITE COLLISION GOSUB` line, read `SPRCOLL` inside the handler, and finish with `RETIRQ`:

```
10 ON SPRITE COLLISION GOSUB 1000
20 REM GAME LOOP
30 GOTO 20
1000 C = SPRCOLL
1010 IF (C AND 1) <> 0 THEN PRINT "SPRITE_0_COLLIDED"
1020 IF (C AND 512) <> 0 THEN PRINT "SPRITE_9_COLLIDED"
1030 RETIRQ
```

Note

`SPRCOLL` and `SPRBG` clear their latched masks after reading them. Store the returned value if you need to check multiple bits.

Flipping

Sprites can be flipped horizontally or vertically without modifying shape data:

```
SPRITE 0,FLIPX,ON
SPRITE 0,FLIPY,ON
```

Warning

Flipping applies to the rendered sprite only. The underlying shape data in shape RAM is unchanged.

Sound and Music

Music is the arithmetic of sounds as optics is the geometry of light.

– Claude Debussy

NovaBASIC includes two SID chip emulators – software recreations of the MOS 6581 Sound Interface Device made famous by the Commodore 64. Six independent voices with four waveforms, ADSR envelopes, and a programmable filter deliver authentic chiptune sound. Simple `SOUND`, `VOLUME`, and `INSTRUMENT` commands write directly to those SID chips. `MUSIC` and `MML` are separate: they are handled by the ESP-hosted music path and wavetable synth support.

This chapter covers every sound command from simple one-shot notes to full multi-voice compositions.

Quick-Start Overview

The sound system has three layers, each building on the one below:

1. **SOUND** – play a single note on the SID chip. Specify a MIDI note number, duration in frames, and an optional instrument preset.
2. **INSTRUMENT** – define a reusable preset that sets the SID waveform and ADSR envelope. Up to 16 presets (slots 0–15).
3. **MUSIC** – load MML (Music Macro Language) sequences into the hosted music engine and play them back with tempo, looping, and per-frame effects.

A minimal program that plays a note:

```
10 VOLUME 12
20 SOUND 60, 30
```

Line 10 sets master volume. Line 20 plays MIDI note 60 (middle C) for 30 frames (half a second at 60 Hz).

The SOUND Command

SOUND note, duration [, instrument]

- **note** – MIDI note number (0–127). Middle C is 60; A4 (concert pitch 440 Hz) is 69. See the MIDI note table below.
- **duration** – length in 1/60-second frames. A value of 60 plays for one second; 30 plays for half a second.
- **instrument** – optional instrument preset (0–15). If omitted, instrument 0 is used.

If **duration** is zero, no new sound is started.

Note

SOUND is fire-and-forget. It starts a note on one of the six SID voices and returns immediately. NovaBASIC advances **SOUND** durations from the frame service used by **VSYNC**, so game loops should call **VSYNC** once per frame for reliable note lengths.

Common MIDI note numbers

Note	MIDI	Approx. Frequency
C3	48	131 Hz
C4 (Middle C)	60	262 Hz
D4	62	294 Hz
E4	64	330 Hz
F4	65	349 Hz
G4	67	392 Hz
A4	69	440 Hz
B4	71	494 Hz
C5	72	523 Hz
C6	84	1047 Hz

The formula is: $\text{frequency} = 440 \times 2^{((\text{midi} - 69) / 12)}$.

A simple melody

```
10 VOLUME 12
20 DATA 60, 62, 64, 65, 67, 69, 71, 72
30 FOR N = 1 TO 8
40   READ M
50   SOUND M, 15
60   FOR I = 1 TO 15 : VSYNC : NEXT I
70 NEXT N
```

Each note plays for 15 frames (250 ms). The VSYNC loop holds the program for the same duration before the next note fires.

Note

SOUND does not block program execution. Use a VSYNC loop after each SOUND call to create the gap between notes and to let the sound scheduler turn notes off at the requested duration.

The INSTRUMENT Command

INSTRUMENT id, waveform, attack, decay, sustain, release

Defines a reusable sound preset in one of 16 instrument slots.

- **id** – slot number (0–15).
- **waveform** – SID waveform byte: 10 = triangle, 20 = sawtooth, 40 = pulse, 80 = noise.
- **attack** – attack rate (0–15). 0 is instantaneous; 15 is slowest.
- **decay** – decay rate (0–15). How quickly the volume drops from peak to the sustain level.
- **sustain** – sustain level (0–15). The steady-state volume held while the note plays. 15 = full volume; 0 = silent (percussive).
- **release** – release rate (0–15). How quickly the volume fades to silence after the note ends.

```
10 REM BRIGHT PULSE LEAD
20 INSTRUMENT 0, $40, 0, 9, 0, 6
30 REM WARM SAWTOOTH PAD
40 INSTRUMENT 1, $20, 4, 6, 12, 8
50 REM NOISE DRUM HIT
60 INSTRUMENT 2, $80, 0, 3, 0, 2
```

Note

Instrument 0 is pre-initialized at boot with: pulse waveform (\$40), attack 0, decay 9, sustain 0, release 6, pulse width 2048. All other slots (1–15) start as copies of slot 0.

SID waveform reference

Value	Name	Character
\$10	Triangle	Soft and mellow; flute-like. Good for gentle melodies and background pads.
\$20	Sawtooth	Buzzy and harmonically rich. Good for brass-like leads and bass lines.
\$40	Pulse	Bold, hollow, classic chiptune sound. Pulse width can be modulated via MML for evolving timbres.
\$80	Noise	Unpitched random output. Use for drums, hi-hats, explosions, and ambient textures.

ADSR envelope overview

The four parameters shape how a note’s volume changes over time:

1. **Attack** ramps from silence to full amplitude.
2. **Decay** drops from full amplitude to the sustain level.
3. **Sustain** holds at a constant level while the note plays.
4. **Release** fades from the sustain level to silence after the note ends.

Sustain is a *level* (0–15); the other three are *rate* values where 0 is fastest and 15 is slowest. This matches the original SID chip behavior.

Instrument recipes

Sound	Wave	A	D	S	R	Notes
Chiptune lead	\$40	0	9	0	6	Sharp attack, no sustain
Warm pad	\$20	8	6	12	10	Slow fade-in, high sustain
Bass	\$20	0	5	8	4	Instant attack, medium body
Snare drum	\$80	0	3	0	2	Short noise burst
Hi-hat	\$80	0	1	0	1	Very short noise tick
Organ	\$10	0	0	15	4	Triangle at full sustain
Pluck	\$40	0	12	0	8	Fast decay, no sustain

VOLUME

`VOLUME level [, voice]`

Sets the SID volume. When `voice` is omitted, sets the master volume (0–15). When `voice` is given (1–6), sets the per-voice volume (0–15) for that voice. Both use the same 4-bit range. See the Per-Voice Volume section below for details. The SID master volume powers up at 0, so BASIC programs should set `VOLUME` before playing sound. The default per-voice volume is 15 (full).

Per-Voice Volume

Note

NovaBASIC extension. The original MOS 6581 SID chip only has a single master volume control shared by all three voices. NovaBASIC adds per-voice volume that allows individual voice levels to be set independently.

VOLUME *level*, *voice*

- *level* – volume level (0–15). 0 is silent; 15 is full volume.
- *voice* – voice number (1–6). If omitted, sets the master volume instead.

Both master and per-voice volume use the same 0–15 range, matching the 4-bit volume resolution of the original MOS 6581 SID chip.

```
10 VOLUME 15
20 VOLUME 15, 1 : REM VOICE 1 FULL
30 VOLUME 5, 2 : REM VOICE 2 QUIET
40 VOLUME 10, 3 : REM VOICE 3 MEDIUM
```

Per-voice volume is applied after the ADSR envelope but before the filter and master volume. This means it acts as a scaling factor on top of the envelope – it does not replace it. The default for all voices is 15 (full), so programs that never set per-voice volume behave identically to a standard SID.

Use cases include MIDI velocity mapping, balancing a loud bass against a delicate melody, and per-voice fade-in/fade-out effects.

Under the hood, per-voice volume is stored in SID extension registers \$1D–\$1F (relative to each chip’s base address). These registers do not exist on the real MOS 6581/8580.

The MUSIC Engine

The music engine is the hosted MML path. You write melodies and rhythms as text strings using Music Macro Language, load them into voices, and let the engine handle timing, instrument switching, and per-frame effects automatically. This path is separate from the direct SID SOUND command.

Loading and Playing Sequences

MUSIC *voice*, "*mml-string*"

- *voice* – voice number 1–6.
- "*mml-string*" – an MML sequence (see Section mml).

Additional subcommands control playback:

Command	Description
MUSIC PLAY	Start playback of all loaded voices.
MUSIC STOP	Stop playback and silence all music voices.
MUSIC TEMPO bpm	Set tempo in beats per minute. Default is 120.
MUSIC LOOP ON	Enable looping; voices restart when all finish.

Command	Description
MUSIC LOOP OFF	Disable looping (default).
MUSIC PRIORITY v1[,...v6]	Set hosted MML voice priority on hosts that support priority routing. Simple SID SOUND uses its own round-robin scheduler.

A Complete Music Example

```

10 VOLUME 12
20 REM DEFINE INSTRUMENTS
30 INSTRUMENT 0, $40, 0, 9, 0, 6
40 INSTRUMENT 1, $20, 0, 5, 8, 4
50 REM LOAD VOICES
60 MUSIC 1, "T140_I0_L8_04_CDEFGAB>C"
70 MUSIC 2, "T140_I1_L4_03_C_G_C_G"
80 REM START PLAYBACK
90 MUSIC LOOP ON
100 MUSIC PLAY

```

Line 60 loads a melody into voice 1: tempo 140, instrument 0, eighth notes, octave 4, ascending C major scale. Line 70 loads a bass line into voice 2: quarter notes, octave 3, alternating C and G. Line 100 starts playback; MUSIC LOOP ON on line 90 means the music repeats indefinitely.

Querying Hosted Music Status

Two functions let you check what the hosted music engine is doing:

Function	Returns
PLAYING	1 if music is currently playing, 0 if stopped.
MNOTE(voice)	The MIDI note number currently sounding on voice (1–6), or 0 if that voice is silent.

```

200 IF PLAYING THEN GOTO 200
210 PRINT "MUSIC_FINISHED"

```

MML Reference

Music Macro Language (MML) is a compact text notation for music. Each voice receives its own MML string. The parser is case-insensitive; all input is converted to uppercase before processing.

Notes and Rests

Syntax	Description
C D E F G A B	Play a note. Pitch is determined by the current octave.
C# or C+	Sharp (raise one semitone).

Syntax	Description
C-	Flat (lower one semitone).
R	Rest (silence for the note duration).

Duration

A number following a note or rest sets its length as a note-value denominator:

Denominator	Ticks	Name
1	384	Whole note
2	192	Half note
4	96	Quarter note
8	48	Eighth note
16	24	Sixteenth note
32	12	Thirty-second note

Internally, one quarter note equals 96 ticks.

A dot (.) after the duration extends it by 50%: `C4.` plays for $96 \times 1.5 = 144$ ticks (dotted quarter).

If no duration is given, the default length set by `L` is used (initially a quarter note).

Ties

The ampersand (&) ties two durations together into a single sustained note:

`C4&8` -> quarter + eighth = 144 ticks

Multiple ties can be chained: `C4&4&4` plays for $96 + 96 + 96 = 288$ ticks. A single `NoteOn` event is emitted with the combined duration.

Octave

Cmd	Description
O4	Set absolute octave (range 1–7; default 4).
>	Octave up (clamped to 7).
<	Octave down (clamped to 1).

MIDI note calculation: $\text{midi} = (\text{octave} + 1) * 12 + \text{semitone}$, where C=0, D=2, E=4, F=5, G=7, A=9, B=11.

Default Length

`L8` sets the default note/rest duration to eighth notes. All subsequent notes and rests that omit an explicit duration will use this value.

Tempo

T120 sets the tempo to 120 beats per minute. The default is 120. Tempo can appear anywhere in the MML string and takes effect immediately. At 120 BPM, one quarter note lasts exactly 0.5 seconds.

Note

Tempo is global. If multiple voices contain T commands, the last one processed wins. It is best practice to set tempo in voice 1 only.

Instrument Selection

I3 switches the current voice to instrument slot 3 (defined earlier with the INSTRUMENT BASIC command). Instrument changes take effect on the next note.

Loops

Square brackets repeat a section:

[CDEF]3 -> plays C D E F three times

The repeat count follows the closing bracket. If omitted, the default is 1 (no repetition). Loops do not nest.

Arpeggios

Curly braces define an arpeggio – a rapid cycling through multiple notes:

\{CEG\}4 -> cycle C, E, G at 60 Hz for one quarter note

Each frame advances to the next note in the list. Accidentals are supported inside the braces (\{C#EG#\}). The arpeggio duration follows the closing brace using standard duration syntax.

Per-Frame Effects

The music engine processes the following effects on every frame (60 Hz). These effects are set within MML and remain active until changed or a new note resets them.

Vibrato 6 sets vibrato depth to 6. Higher values produce wider pitch oscillation. 0 turns vibrato off.

The vibrato oscillates at approximately 2.9 Hz (sine wave). The pitch offset is proportional to both the depth value and the current note frequency.

Portamento (Pitch Slide) / before a note causes the voice to slide from the current pitch to the target note rather than jumping instantly.

C4 /E4 -> play C, then glide smoothly to E

The slide rate is approximately 1/8 of the frequency distance per frame.

Pulse Width @P2048 sets the SID pulse width to 2048 (range 0–4095). This only affects the pulse waveform (\$40). A value of 2048 gives a 50% duty cycle (square wave); lower or higher values create thinner, more nasal timbres.

Pulse Width Modulation (PWM) @PS+ starts sweeping the pulse width upward; @PS– sweeps downward; @PS0 stops the sweep. The sweep rate is +/-32 per frame, clamped to the 0–4095 range. PWM gives the pulse waveform a rich, evolving character.

```
60 MUSIC 1, "@P1024L2C4E4G>C"
```

Filter Cutoff and Resonance @F1024,8 sets the SID filter cutoff to 1024 (range 0–2047) and resonance to 8 (range 0–15). The resonance parameter is optional; if omitted, it defaults to 0.

Filter Mode

Cmd	Mode
@FL	Low-pass (cuts highs, warm sound)
@FB	Band-pass (emphasizes a frequency band)
@FH	High-pass (cuts lows, thin sound)
@FO	Filter off

Filter Sweep @FS+ sweeps the filter cutoff upward; @FS– sweeps downward; @FS0 stops the sweep. The sweep rate is +/-8 per frame, clamped to 0–2047.

```
60 MUSIC 1, "@FLF200,12L4O3[CDEFGAB>C<]2"
```

This creates a classic filter sweep effect: low-pass filter starting at cutoff 200 with high resonance, sweeping upward through the melody.

MML Command Summary

Command	Parameters	Description
A–G	[#/+/-][dur][.]	Play note
R	[dur][.]	Rest
O	1–7	Set octave
>	–	Octave up
<	–	Octave down
L	denominator	Default note length
T	bpm	Tempo (default 120)
I	0–15	Select instrument slot
V	0–15	Per-voice volume (default 15)
&	[note]dur	Tie (extend note duration)
[...]n	repeat count	Loop section n times
{notes}	[dur][.]	Arpeggio
	depth (0=off)	Vibrato
/	–	Portamento (next note slides)

Command	Parameters	Description
@P	0–4095	Set pulse width
@PS	+, -, 0	PWM sweep direction
@F	cutoff[,res]	Filter cutoff (0–2047) and resonance (0–15)
@FL	–	Low-pass filter
@FB	–	Band-pass filter
@FH	–	High-pass filter
@FO	–	Filter off
@FS	+, -, 0	Filter sweep direction

Whitespace, tabs, newlines, and pipe characters (|) are ignored and can be used freely to format MML strings for readability.

File Playback

File-backed playback is available as convenience BASIC commands:

```
10 SIDPLAY "commando",1
20 MIDPLAY "theme"
30 SFLOAD "piano.sf2"
```

SIDPLAY "name"[,song] plays a SID file, defaulting to song 1. SIDSTOP stops SID playback. MIDPLAY "name" plays a MIDI file through the hosted music engine, and MIDSTOP stops it. SFLOAD "name" loads a new soundfont for MIDI playback.

Hosted Playback Status

Assembly and NDK programs can poll AUDIO_STATUS/MUSIC_STATUS at \$BA50. The status byte is updated during hosted SID/MIDI playback and during long asset loads such as soundfont loading:

Symbol	Bit	Description
AUDIO_STATUS_SFX	\$01	Hosted sound effect active.
AUDIO_STATUS_MUSIC	\$02	Hosted music playback active.
AUDIO_STATUS_SID	\$04	Current hosted music source uses SID voices.
AUDIO_STATUS_WTS	\$08	Current hosted music source uses WTS/MIDI voices.
AUDIO_STATUS_LOADING	\$10	Hosted music or soundfont assets are loading.

SIDPLAY, MIDPLAY, and SFLOAD in NovaBASIC are blocking commands, so the loading bit is primarily for assembly programs that issue FIO commands directly or otherwise keep a UI loop alive while the host finishes a load.

Hosted Playback Metadata

When `SIDPLAY` or `MIDPLAY` starts playback, NovaHost publishes a single metadata block for the active song. `SID` and `MIDI` use the same ABI so players, visualizers, and other NDK programs do not need separate code paths. `SID` metadata comes from the `SID` header title, author, and copyright fields. `MIDI/WTS` metadata comes from the compiled `.nms` header when the stream was created by a metadata-aware Nova CLI; older streams fall back to the filename.

Symbol	Address	Description
<code>AUDIO_META_TYPE</code>	<code>\$BAB0</code>	1 for <code>SID</code> , 3 for <code>MIDI/WTS</code> .
<code>AUDIO_META_TITLE</code>	<code>\$BAB3</code>	32-byte null-padded title or filename fallback.
<code>AUDIO_META_AUTHOR</code>	<code>\$BAD3</code>	32-byte null-padded author/composer when known.
<code>AUDIO_META_COPYRIGHT</code>	<code>\$BAF3</code>	32-byte null-padded copyright/comment when known.
<code>AUDIO_META_SONGS</code>	<code>\$BB19</code>	<code>SID</code> subtune count or <code>MIDI</code> track count when known.
<code>AUDIO_META_DURATION_L/H</code>	<code>\$BB1A</code>	Duration in seconds when known.
<code>AUDIO_META_FLAGS</code>	<code>\$BB1C</code>	Music-specific flags.
<code>AUDIO_META_SOUNDFont</code>	<code>\$BB8E</code>	64-byte null-padded active <code>MIDI/WTS</code> soundfont name. Empty for <code>SID</code> playback.

`SID` flags currently include `AUDIO_META_FLAG_SID_6581`, `AUDIO_META_FLAG_SID_8580`, `AUDIO_META_FLAG_STEREO`, and `AUDIO_META_FLAG_NTSC`.

Graphics File I/O

NovaBASIC can save and load VGC memory spaces to disk:

Command	Description
<code>GSAVE "name", space, offset, len</code>	Save <code>len</code> bytes from VGC memory space starting at <code>offset</code> to a <code>.gfx</code> file.
<code>GLOAD "name", space, offset[, len]</code>	Load a <code>.gfx</code> file into VGC memory space at <code>offset</code> . If <code>len</code> is omitted, the entire file is loaded.

Assembly applications can load NovaDraw `.nvg/NVG1` images with the `NVG.LOAD` runtime helper. NovaHost clears the graphics bitmap, decodes the spans, and streams the pixels directly into graphics bitmap memory.

VGC memory spaces:

Space	Contents
1	Character RAM (4000 bytes)
2	Color RAM (4000 bytes)
3	Graphics bitmap (64000 bytes)
4	Sprite shape RAM (32768 bytes; 256 slots)
6	Tile data RAM

```

10 MODE 1 : GCLS
20 GCOLOR 10 : CIRCLE 160, 100, 80
30 GSAVE "mycircle", 3, 0, 64000
40 REM LATER...
50 GLOAD "mycircle", 3, 0

```

Music Engine Architecture

For advanced users, understanding the engine internals helps write better music and avoid common pitfalls.

Voice allocation

Simple **SOUND** effects use a small round-robin scheduler over the six SID voices. The command starts the note and returns; **VSYNC** advances the countdowns and gates notes off when their duration expires. MML playback uses the hosted music path instead of this SID scheduler.

The Second SID Chip

NovaBASIC includes two SID chips for a total of six voices. The first SID (D400) handles voices 1–3; the second SID (D420) handles voices 4–6. A legacy mirror at \$D500 routes transparently to the second SID.

From BASIC, the second SID is fully transparent – simply use voice numbers 4–6 with the **MUSIC** command:

```

10 INSTRUMENT 0,64,2,8,12,6
20 MUSIC 1,"T140_I0_04_C4_E4_G4_C5_2"
30 MUSIC 4,"T140_I0_03_C2_G2"
40 MUSIC PLAY

```

Voice 4 plays on the second SID chip. All instrument definitions, tempo, and loop settings apply equally to both chips.

Timing

The engine ticks at 60 Hz. Tempo is converted to ticks per frame:

ticks per frame = 96 x BPM/3600

At 120 BPM this is 3.2 ticks per frame. A quarter note (96 ticks) takes exactly 30 frames = 0.5 seconds.

Effect processing order

Each frame, active effects are processed in this order:

1. Arpeggio (cycle to next note)
2. PWM sweep (+/-32 per frame, clamped 0–4095)
3. Vibrato (sine wave at 2.9 Hz)
4. Portamento (slide 1/8 of remaining distance per frame)
5. Filter sweep (+/-8 per frame, clamped 0–2047)

Composition Tips

Tip

- Define all instruments before loading music sequences.
- Use **I** in MML to switch instruments mid-voice for timbral variety.
- Keep melody, harmony, and bass on separate voices. Each voice has its own instrument, octave, and effect state.
- Use **|** characters in MML strings as bar-line separators for readability: "L8 CDEF | GABC".
- At 120 BPM: quarter = 30 frames, eighth = 15, sixteenth = 7.5. Use **T** to control tempo rather than adjusting note lengths.
- Use **@PS+** and **@PS-** on pulse waveforms for rich, evolving textures.
- Combine **@FL** with **@FS+** for classic acid-bass filter sweeps.
- The noise waveform (**\$80**) with short ADSR makes convincing drums. Load a noise instrument and trigger it with **SOUND** while the music plays.

Deprecated Commands

Warning

The following commands from earlier versions have been superseded:

- **WAVE** – raises a syntax error. Use **INSTRUMENT** instead.
- **ENVELOPE** – replaced by **INSTRUMENT**. Programs should be updated to use the new six-parameter syntax.

Try It Now

Try It

Type and run the following program to hear a six-voice arrangement with instrument presets, MML sequences, filter effects, and looping:

```
10 VOLUME 12
20 INSTRUMENT 0, $40, 0, 9, 0, 6
30 INSTRUMENT 1, $20, 0, 5, 8, 4
40 INSTRUMENT 2, $80, 0, 3, 0, 2
50 MUSIC 1, "T120 I0 L8 04 CDEFGAB>C2"
60 MUSIC 2, "T120 I1 L4 03 C G C G"
70 MUSIC 3, "T120 I2 L8 04 R C R C R C R C"
80 MUSIC LOOP ON
90 MUSIC PLAY
```

Expected result: a three-voice loop with a pulse-wave melody, sawtooth bass, and noise percussion on SID 1. The music repeats until you type `MUSIC STOP` in direct mode. Try assigning additional voices (4–6) to use the second SID chip.

Experiments: - Change T120 to T180 for a faster tempo. - Add vibrato to voice 1: change the MML to start with "T120 I0 4 L8 04 ...". - Add a filter sweep to voice 2: "T120 I1 @FL @F200,10 @FS+ L4 03 C G C G". - Balance the mix with per-voice volume: add `VOLUME 15, 1 : VOLUME 8, 2 : VOLUME 5, 3` before `MUSIC PLAY` to push the melody forward and pull back the bass and drums.

Expansion Memory

"The problem with a 64-kilobyte address space is not that it is small. It is that everything you want to do next is just barely larger than it."

The 6502 CPU can directly address 64 KB. That is enough for code, stack, screen RAM, and a modest program, but it leaves little room for data-heavy work: large sprite sheets, level maps, sample tables, or pre-rendered buffers quickly overflow the available RAM. The NovaBASIC expansion memory system solves this by providing an additional 512 KB of RAM that lives outside the CPU address space, accessible through a set of commands and a window-mapping mechanism.

Why Expansion Memory?

The CPU address bus is 16 bits wide. After subtracting ROM, the VGC, the sound controller, the file I/O coprocessor, the XMC registers, and screen RAM, your usable BASIC RAM runs from 0280 to 9FFF – approximately 39 KB. That is enough for programs and variables, but becomes tight the moment you try to cache a full screen buffer, store multiple music tracks, or keep several sprite-sheet layers in memory simultaneously.

Expansion memory (**XRAM**) gives you up to 512 KB of additional storage with no impact on the CPU address space. You cannot execute code from XRAM directly, but you can:

- Store and retrieve arbitrary data blocks by raw offset.
- Name blocks and recall them without tracking raw addresses.

- Map 256-byte XRAM pages directly into CPU address windows for transparent byte-level access via PEEK and POKE.
- Use documented fixed workspaces for runtimes that need predictable scratch storage.

The XRAM hardware is controlled by the Expansion Memory Controller (XMC), mapped at \$BA00. You normally never touch those registers directly – the BASIC commands handle all of it for you.

Banks and Status

XRAM is divided into **banks**, each 64 KB in size. With the default 512 KB configuration there are **8 banks** (numbered 0 through 7). One bank is always *active*; raw-offset commands operate within that bank.

BASIC deliberately keeps this banked model: you select **XBANK n**, then pass a 16-bit offset to XRAM commands. Assembly programs should use the shared XRAM runtime instead, where the same location is expressed as a flat 24-bit address:

```
flat address = bank * 65536 + offset
```

For example, **XBANK 3: XPOKE \$2000,42** writes the same byte an assembly program would address as \$032000.

Checking memory status: XMEM

Type **XMEM** at the prompt to print a status summary:

```
XMEM
```

```
8 BANKS, 512 KB XRAM, BANK 0, USED 5, FREE 2043 PAGES, NAMED 0
```

The output shows total bank count, total XRAM capacity, the currently active bank, and the number of 256-byte pages that are in use versus free across the entire 512 KB space. Two thousand and forty-eight free pages means 512 KB is available.

Selecting a bank: XBANK n

```
XBANK 3
```

Selects bank 3 as the active bank. Subsequent **XPOKE**, **XPEEK**, and raw **STASH/FETCH** operations use offsets within that bank. Valid range is 0 to 7 for the default 512 KB configuration.

Note

Named blocks (**STASH "name" / FETCH "name"**) are allocated automatically across the full XRAM space and are not limited to the active bank. The active bank setting only affects raw-offset operations.

Single-Byte Access

For simple tasks – writing a flag, reading a configuration byte, probing a value – the single-byte commands are the most direct interface.

XPOKE *offset,value*

Writes *value* (0–255) to *offset* within the currently active bank. Offsets run from 0 to 65535.

```
XBANK 0
XPOKE 0,42
XPOKE 1,255
```

XPEEK(*offset*)

Reads a single byte from *offset* in the active bank and returns it as a numeric value. You can use it anywhere a number is valid:

```
PRINT XPEEK(0)
V = XPEEK(1) + XPEEK(2)
```

Tip

XPOKE and XPEEK are convenient for a handful of bytes or for inspecting specific locations. For moving blocks of data use STASH and FETCH, which issue a single hardware transfer rather than looping byte by byte.

Bulk Transfers

Moving data in and out of XRAM is the core workflow for anything non-trivial. The raw bulk-transfer commands operate on explicit offsets within the active bank.

Raw STASH – CPU RAM to XRAM

STASH *ramaddr,xramoffset,length*

Copies *length* bytes starting at CPU address *ramaddr* into XRAM starting at *xramoffset* within the active bank.

Raw FETCH – XRAM to CPU RAM

FETCH *ramaddr,xramoffset,length*

Copies *length* bytes from XRAM at *xramoffset* back to CPU RAM starting at *ramaddr*. The destination must be within writable RAM (0000–BFFF).

Direct file streaming: XLOAD and XSAVE

```
XLOAD "STORY.Z3",0
XSAVE "SNAPSHOT",0,4096
```

XLOAD streams a file directly into XRAM without staging it through CPU RAM. XSAVE streams a range of XRAM directly back to disk. Both commands use the active XBANK plus a 16-bit *offset* as the starting address. XLOAD accepts an optional *length*; if omitted or zero, the entire file is loaded, which is useful for assets larger than a single 64 KB bank.

Filenames without an extension use **.xram**. Explicit extensions are honored, so Z-machine story files such as ZORK1.Z3 can be loaded directly into XRAM.

Because XLOAD and XSAVE stream directly between files and XRAM, they are the right path for assets that are too large or too inconvenient to stage through normal BASIC RAM.

BASIC XRAM command summary

Command	Purpose
XMEM	Print XRAM capacity, active bank, used pages, and free pages.
XBANK n	Select the active 64 KB bank for raw BASIC XRAM commands.
XPOKE offset,value	Write one byte in the active bank.
XPEEK(offset)	Read one byte in the active bank.
STASH ramaddr,offset,length	DMA copy CPU RAM to XRAM in the active bank.
FETCH ramaddr,offset,length	DMA copy XRAM in the active bank to CPU RAM.
XLOAD "name",offset[,length]	Stream a file directly into XRAM.
XSAVE "name",offset,length	Stream XRAM directly to a file.
STASH "name",ramaddr,length	Store a named block.
FETCH "name",ramaddr	Fetch a named block.
XDIR	List named blocks.
XDEL "name"	Delete a named block.
XMAP window,offset	Map a 256-byte XRAM page into \$BC00-\$BFFF.
XUNMAP window	Unmap a mapped XRAM window.

Example: saving and restoring a character screen buffer

Text character memory is a VGC memory space, not a direct CPU address range. Use DMA or the VDC-style VRAM port to snapshot it. This example copies the 4000-byte 80x50 character plane to XRAM bank 0 and restores it later:

```
10 REM Save character screen to XRAM
20 XBANK 0
30 DMACOPY 1,0,5,0,4000
40 PRINT "SCREEN STASHED."
50 REM ... do other things ...
60 REM Restore character screen from XRAM
70 DMACOPY 5,0,1,0,4000
80 PRINT "SCREEN RESTORED."
```

Note

Color RAM is VGC memory space 2 and is also 4000 bytes. Copy it to a second XRAM offset if you need a full text + color snapshot.

Named Blocks

Raw offsets require you to track where each block lives. The named-block interface lets you store data under a string name and retrieve it without knowing its physical XRAM address.

Storing a named block: STASH "name",ramaddr,length

```
STASH "MYDATA",2048,16
```

Allocates a region in XRAM, copies 16 bytes from CPU address 2048 into it, and registers the block under the name MYDATA. Names are 1 to 28 characters and are case-insensitive. If a block with that name already exists and is large enough for the new data, it is overwritten in place. If the existing block is too small, it is freed and a new one is allocated automatically.

Loading a named block: FETCH "name",ramaddr

```
FETCH "MYDATA",2048
```

Finds the named block and copies its full contents back to CPU RAM at *ramaddr*. No length argument is required; the XMC remembers the exact size.

Listing named blocks: XDIR

```
XDIR
```

Prints all named blocks sorted alphabetically (case-insensitive), showing each name and its stored size in bytes.

Deleting a named block: XDEL "name"

```
XDEL "MYDATA"
```

Removes the named block and releases its XRAM pages back to the free pool.

Complete example: named-block round-trip

```
10 FOR I=0 TO 15:POKE 2048+I,I+10:NEXT I
20 STASH "MYDATA",2048,16
30 FOR I=0 TO 15:POKE 2048+I,0:NEXT I
40 FETCH "MYDATA",2048
50 FOR I=0 TO 15:PRINT PEEK(2048+I);:NEXT I
60 XDEL "MYDATA"
```

Line 10 writes values 10 through 25 into RAM at address 2048. Line 20 stashes them to XRAM under the name MYDATA. Line 30 zeroes the same RAM region to confirm the data is no longer in CPU RAM. Line 40 fetches the block back. Line 50 prints the restored values – you should see 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25. Line 60 cleans up.

Low-Level Allocation

For programs that need to manage XRAM regions dynamically without names, two lower-level commands are available.

XALLOC *length*

Allocates *length* bytes from the low-*XRAM* software heap. This is mostly useful for diagnostics and tooling that inspect *XMC* registers directly; ordinary BASIC programs should usually use named *STASH/FETCH* blocks or raw offsets.

XALLOC 4096

XFREE *offset,length*

Releases the *XRAM* range starting at *offset* with the given *length*. Any named or unnamed blocks that overlap that range are removed from the allocation table.

Assembly applications should avoid heap-style *XRAM* allocation. Use the flat **xram.s** routines with a documented fixed workspace and clear that workspace at program launch. **xram.inc** defines the shared layout bands used by NovaZ and the NVG loader.

Warning

XRESET clears all allocation tracking in one step – named blocks, unnamed allocations, and page usage records are all discarded and cannot be recovered. The raw bytes in *XRAM* are not erased, but every name and handle is gone. Use **XRESET** only when you want a clean slate, such as at program startup.

Memory Windows

For the highest-performance *XRAM* access – or for assembly code that needs to use ordinary load and store instructions against *XRAM* – you can map an *XRAM* page directly into the CPU address space using a **window**.

There are four windows, each exactly 256 bytes wide, at fixed CPU addresses:

Window	CPU Address Range	Size
0	BC00–BCFF	256 bytes
1	BD00–BDFF	256 bytes
2	BE00–BEFF	256 bytes
3	BF00–BFFF	256 bytes

When a window is mapped, any **PEEK** or **POKE** to its CPU address range reads or writes directly into *XRAM*. No transfer command is needed.

XMAP *window,offset*

Maps the 256-byte *XRAM* page that contains *offset* into the specified window (0–3). The offset is rounded down to a 256-byte page boundary automatically.

```
10 REM Map XRAM page 0 into window 0 at $BC00
20 XMAP 0,0
30 FOR I=0 TO 255
40 POKE 48128+I,I
```

```
50 NEXT I
60 PRINT PEEK(48128);" ";PEEK(48255)
```

\$BC00 = 48128 decimal. After XMAP 0,0, writes to BC00–BCFF go directly into XRAM. The PRINT on line 60 should show 0 255.

XUNMAP window

Unmaps the specified window. Reads and writes to that CPU address range no longer reach XRAM.

```
XUNMAP 0
```

Warning

Mapped windows are shared address space visible to both BASIC and any running assembly code. If two windows are mapped to the same XRAM page, or if BASIC and assembly both access the same window, coordinate ownership carefully to avoid corruption. Unmap windows you are finished with.

Error Codes

When an XRAM command fails, NovaBASIC reports an error derived from the XMC error register. The complete set:

Code	Meaning
0	No error
1	Address out of range
2	Bad arguments
3	Named block not found
4	No space available
5	Invalid name
6	End of directory

Try It Now

```
Named block round-trip
10 FOR I=0 TO 15:POKE 2048+I,I+10:NEXT I
20 STASH "MYDATA",2048,16
30 FOR I=0 TO 15:POKE 2048+I,0:NEXT I
40 FETCH "MYDATA",2048
50 FOR I=0 TO 15:PRINT PEEK(2048+I);:NEXT I
60 XDEL "MYDATA"
```

Expected result: line 50 prints 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25, confirming the data survived the round-trip through XRAM and back.

Assembly and Special Chips

"To understand what the machine is actually doing, you have to speak

its language -- and that language is always closer to the metal than any other abstraction on top of it."

NovaBASIC is built on a 6502 core and runs on top of a set of memory-mapped coprocessors. Most programs never need to touch hardware registers directly, but knowing where everything lives gives you the full picture: why certain address ranges are reserved, what BASIC commands actually do at the hardware level, and how to write assembly routines that cooperate cleanly with the BASIC runtime.

The Memory Map

The full 64 KB address space is partitioned as follows:

Range	Size	Purpose	Access
0000–00FF	256 B	Zero Page	R/W
0100–01FF	256 B	Stack	R/W
0200–027F	128 B	System Vectors	R/W
0280–9FFF	39 KB	BASIC RAM	R/W
A000–A01F	32 B	VGC Registers	R/W
A040–A0BF	128 B	Sprite Registers (16x8)	R/W
A0C0–A0DF	32 B	Tile Engine Registers	R/W
A0E0–A0E4	5 B	VDC-style VRAM Port	R/W
A100–A13F	64 B	Network Interface Controller (NIC)	R/W
B9A0–B9EF	80 B	File I/O Controller (FIO)	R/W
BA00–BA3F	64 B	Expansion Memory Controller (XMC)	R/W
BA40–BA4F	16 B	Timer Controller	R/W
BA50–BA62	19 B	Hosted Music Status (14 voices)	RO
BA63–BA75	19 B	DMA Controller	R/W
BA83–BA9B	25 B	Blitter Controller	R/W
BA9C–BAA1	6 B	Board Input registers	R/W1C
BC00–BFFF	1024 B	XRAM Windows (when mapped)	R/W
C000–FFFF	16 KB	ROM (NovaBASIC)	R only
D400–D41F	32 B	SID chip 1 (write-intercepted)	W only
D420–D43F	32 B	SID chip 2 (write-intercepted)	W only

Everything from A000 upward through BFFF is hardware I/O or managed window space. Writing to ROM (\$C000+) has no effect.

System Vectors at \$0200

At boot, NovaBASIC's ROM initializes page \$02 with EhBASIC internal vectors and the BASIC IRQ/NMI handler pointers. The layout is inherited from EhBASIC 2.22p5:

Address	Purpose
0200–0202	Ctrl-C flag, byte, and timeout
0203–0204	VEC_CC – Ctrl-C check vector
0205–0206	VEC_IN – input vector

Address	Purpose
0207–0208	VEC_OUT – output vector
0209–020A	VEC_LD – load vector
020B–020C	VEC_SV – save vector
020D–0216	IRQ handler code area (set by IRQ)
0217–0220	NMI handler code area (set by NMI)

Hardware controller base addresses are fixed constants; use the values from Appendix memmap directly in your assembly code.

Talking to Hardware from BASIC

Because the hardware controllers are memory-mapped, you can read and write their registers with ordinary PEEK and POKE calls from BASIC. This is the simplest way to experiment with hardware state or build lightweight diagnostic tools.

Reading the frame counter

The VGC increments a frame counter register at \$A008 on every display frame. Reading it gives you a running frame tick useful for timing and animation:

```
PRINT PEEK(40968)
```

\$A008 = 40968 decimal. The counter wraps from 255 to 0 (it is an 8-bit register).

Checking the active sprite count

```
PRINT "SPRITES: ";PEEK(40969)
```

\$A009 = 40969 decimal. This read-only register holds the number of currently enabled sprites.

Reading keyboard input

```
K = PEEK(40975)
```

\$A00F = 40975 decimal. This is the VGC character input register. Reading it returns the last character received, or 0 if none. BASIC's own GET command uses the same register.

Character output

Writing a character code to \$A00E (40974 decimal) emits that character to the current cursor position, exactly as BASIC's PRINT does internally:

```
POKE 40974,65
```

```
REM prints the letter A
```

The SYS and USR Interface

NovaBASIC provides two ways to execute machine code from within a BASIC program.

SYS addr[,a[,x[,y]]]

Jumps to *addr* after pushing a BASIC return address on the stack. Execution resumes in BASIC after the machine-code routine executes an RTS.

The optional byte arguments initialize the 6502 A, X, and Y registers before the jump:

```
10 SYS 36864
REM jumps to machine code at $9000
```

```
20 SYS 36864,60,10,1
REM jumps with A=60, X=10, Y=1
```

When the routine returns, NovaBASIC stores the returned registers in the SYS mailbox bytes `SYS_REGA`, `SYS_REGX`, and `SYS_REGY` from `runtime/asm/nova.inc`. These are intended as the stable bridge for small BASIC-to-assembly calls.

Use `ADDR("LABEL")` when BASIC code needs one of the generated runtime labels instead of a hard-coded address:

```
10 SYS ADDR("AUDIO.SIDPLAY")
20 PRINT ADDR("SYS.REGA")
```

The lookup table lives in the extension ROM and is generated from the same `@label` comments as the assembly reference. `ADDR()` only exposes labels with resolved ROM, RAM, or MMIO addresses; extension-only and link-time library labels remain documented for assembly callers but are not valid SYS targets.

Runtime labels are documented from `@label` comments in the assembly sources. See `docs/assembly/runtime-labels.md` for the generated routine and pseudo-register ABI reference.

Reusable ca65 support code lives under `runtime/asm`. For the current library map, include/link rules, and pointers to sprite, animation, XRAM, audio, network, and UI helpers, see `docs/assembly/library-reference.md`.

USR(x)

Calls a user-defined machine-code routine, passing the numeric value *x* through the 6502 floating-point accumulator (FAC). The routine can read, modify, and return a value through the same register. `USR(x)` is a numeric function and its result can be used in an expression:

```
10 V = USR(42)
20 PRINT "RESULT: ";V
```

The address of the USR routine is set by storing a 16-bit pointer in the appropriate zero-page location before calling. Consult the EhBASIC 2.22 documentation for the exact zero-page addresses used by the USR vector.

Interrupts

The 6502 supports two interrupt lines: the maskable IRQ and the non-maskable NMI. NovaBASIC lets you handle both from within a BASIC program, which is useful for writing interrupt-driven input handlers, timing routines, and co-operative multitasking sketches.

Setting up a handler: IRQ linenumber and NMI linenumber

```
10 IRQ 1000
20 NMI 2000
```

When an IRQ fires, execution branches to line 1000. When an NMI fires, execution branches to line 2000. The handlers are ordinary BASIC subroutines.

Returning from a handler: RETIRQ and RETNMI

The last statement in an IRQ handler must be `RETIRQ`; the last statement in an NMI handler must be `RETNMI`. These are not interchangeable with `RETURN` – they restore the correct CPU state and re-enable the interrupt flag.

```
1000 REM IRQ handler
1010 PRINT "IRQ FIRED"
1020 RETIRQ
2000 REM NMI handler
2010 PRINT "NMI FIRED"
2020 RETNMI
```

Warning

Interrupt handlers run in the context of the BASIC interpreter. Keep them short. Avoid file I/O, heavy computation, or anything that re-enters the interpreter in an unexpected state. Long handlers can cause instability.

Board input change IRQs from BASIC

ULX3S board buttons and DIP switches can raise IRQ when their debounced state changes. `$BA9C` and `$BA9D` remain the live polling registers; `$BA9E`–`$BAA1` are the opt-in IRQ/change latch registers.

```
10 IRQ 1000
20 POKE $BA9F,3:POKE $BAA0,127:POKE $BAA1,15
30 POKE $BA9E,3
40 GOTO 40
1000 S=PEEK($BA9F):B=PEEK($BA9C):W=PEEK($BA9D)
1010 BM=PEEK($BAA0):WM=PEEK($BAA1)
1020 POKE $BAA0,BM:POKE $BAA1,WM:POKE $BA9F,S
1030 RETIRQ
```

Bit 0 of `$BA9E`/`$BA9F` is button changes; bit 1 is DIP-switch changes. `$BAA0` and `$BAA1` identify which button or switch bits changed and are cleared by writing 1 bits back.

Sprite collision IRQs from BASIC

BASIC has a dedicated command for sprite-sprite collision interrupts. The VGC sets one bit for each sprite that participated in a collision. `SPRCOLL` returns the full 16-bit mask, clears the latched mask, and acknowledges the VGC sprite collision IRQ source.

```
10 ON SPRITE COLLISION GOSUB 1000
20 REM MAIN GAME LOOP
```

```

30 GOTO 20
1000 C = SPRCOLL
1010 IF (C AND 1) <> 0 THEN PRINT "SPRITE_0_COLLIDED"
1020 IF (C AND 256) <> 0 THEN PRINT "SPRITE_8_COLLIDED"
1030 RETIRQ

```

For sprite-background collisions, poll SPRBG once per frame unless your program installs its own assembly IRQ handler.

Sprite collision IRQs from assembly

Assembly programs use two layers:

- The CPU IRQ vector at \$FFFE/\$FFFF decides which routine runs.
- VGC_IRQ_ENABLE selects which VGC IRQ sources can raise the CPU IRQ line.

The shared runtime helpers in `runtime/asm/vgc.s` and `runtime/asm/sprite.s` handle the common setup. Link those files into the program, or include them once from the program's build.

```

.include "nova.inc"
.include "vgc.inc"
.include "sprite.inc"

```

```

COLL_LO = NVR0L
COLL_HI = NVR0H

```

```

init_sprite_irq:
    LDA    #>sprite_irq_handler
    LDY    #<sprite_irq_handler
    JSR    vgc_irq_install

    JSR    sprite_collision_irq_enable
    RTS

```

```

sprite_irq_handler:
    PHA
    PHX
    PHY

    LDA    VGC_IRQ_STATUS
    AND    #VGC_IRQ_SPRCOLL
    BEQ    @done

    JSR    sprite_collision_read_clear
    STY    COLL_LO
    STA    COLL_HI

    ; Bit 0 in COLL_LO = sprite 0, bit 0 in COLL_HI = sprite 8.
    LDA    COLL_HI
    AND    #$01

```



```

BEQ    @done
; Handle sprite 8 collision here.

@done:
    PLY
    PLX
    PLA
    RTI

```

If one handler services multiple VGC IRQ sources, test `VGC_IRQ_STATUS` for each enabled source and acknowledge every source you handle. The sprite helpers `sprite_collision_read_clear` and `sprite_background_read_clear` both return the 16-bit mask in `A:Y`, clear the matching collision latch, and acknowledge the matching VGC IRQ source.

XRAM Assembly Runtime

NovaVM provides a reusable ca65 XRAM runtime in `runtime/asm/xram.inc` and `runtime/asm/xram.s`. BASIC keeps its `XBANK` plus 16-bit offset model, but assembly code should use the runtime's flat 24-bit address registers: `XRAM_ADDRL`, `XRAM_ADDRM`, and `XRAM_ADDRH`.

For standalone assembly programs, link `runtime/asm/xram.s` into the program or include it once after your own code. Do not copy the XMC, DMA, or FIO register sequences into each project; the runtime is the reusable boundary for byte access, DMA-backed bulk copies, fills, and direct `XLOAD/XSAVE` streaming. If a runtime needs BASIC's named-block allocator and XMC register command contract, link `runtime/asm/xmc.s` as well.

Standalone runtimes should use fixed XRAM workspaces rather than dynamic hardware allocation. `xram.inc` defines the shared high-XRAM bands used by NovaZ and the NVG loader, while the low 256 KB stays available to BASIC/XMC named blocks. A runtime that needs private scratch should claim a documented range and clear it at launch.

Those symbols are aliases over the Nova pseudo-register ABI:

Symbol	Pseudo-register
<code>XRAM_ADDRL/M/H</code>	<code>NVR0L</code> , <code>NVR0H</code> , <code>NVR1L</code>
<code>XRAM_DATA</code>	<code>NVR1H</code>
<code>XRAM_RAML/H</code>	<code>NVR2L/H</code>
<code>XRAM_LENL/H</code>	<code>NVR3L/H</code>

All routines return `A=0` on success and `A=1` on error. They also update `XMC_STATUS` and `XMC_ERRCODE`. BASIC commands use wrapper entry points that copy their XMC MMIO arguments into the same pseudo-register layout.

Label	Purpose
<code>xram_read8</code>	Read the byte at <code>XRAM_ADDRL/M/H</code> into <code>XRAM_DATA</code> .
<code>xram_write8</code>	Write <code>XRAM_DATA</code> to <code>XRAM_ADDRL/M/H</code> .

Label	Purpose
<code>xram_copy_from_ram</code>	DMA copy from CPU RAM <code>XRAM_RAML/H</code> to XRAM <code>XRAM_ADDRL/M/H</code> .
<code>xram_copy_to_ram</code>	DMA copy from XRAM <code>XRAM_ADDRL/M/H</code> to CPU RAM <code>XRAM_RAML/H</code> .
<code>xram_fill</code>	DMA fill <code>XRAM_LEN/H</code> bytes at <code>XRAM_ADDRL/M/H</code> with <code>XRAM_DATA</code> .
<code>xram_xload</code>	Stream a file directly into XRAM.
<code>xram_xsave</code>	Stream XRAM directly to a file.
<code>xram_wait_dma</code>	Wait for a DMA transfer and translate DMA errors into XMC status.
<code>xmc_process</code>	Optional <code>runtime/asm/xmc.s</code> entry point for the XMC register command contract.

VTEXT Assembly Runtime

`runtime/asm/vtext.inc` and `runtime/asm/vtext.s` provide reusable VGC text regions for assembly programs and language runtimes. A caller defines a rectangle, local cursor, packed color, text attributes, and wrap/scroll flags; VTEXT handles clear, line clear, character output, and blitter-backed region scrolls across the character, color, and text-attribute planes.

This is the right layer for fixed status rows, editor panes, REPL regions, and NovaZ story windows. The library does not know about Z-machine windows; NovaZ maps story-file windows onto VTEXT regions.

Example: reading one byte from XRAM in assembly

```
; Set 24-bit address 0x010000 (bank 1, offset 0)
STZ XRAM_ADDRL
STZ XRAM_ADDRM
LDA #$01
STA XRAM_ADDRH
JSR xram_read8
BNE error
LDA XRAM_DATA
```

Note

Bulk copies use the DMA controller with XRAM as space 5. Use `XRAM_LEN/H` for the length and `XRAM_RAML/H` for the CPU RAM endpoint.

Direct XRAM file streaming from assembly

Assembly programs should call the shared `xram_xload` and `xram_xsave` routines instead of writing the raw FIO registers directly:

Symbol	Meaning for <code>xram_xload/xram_xsave</code>
<code>XRAM_ADDRL/M/H</code>	Flat 24-bit XRAM source/destination
<code>XRAM_LENL/H</code>	Transfer length; zero means full file for <code>xram_xload</code>
<code>XRAM_NAMELEN</code>	Filename length, 1–63 bytes
<code>XRAM_NAMEPTR_L/H</code>	Pointer to filename bytes in CPU RAM

On success, `A=0` and `XRAM_LENL/H` contains the actual transferred byte count. On error, `A=1` and both `FIO` and `XMC` status/error registers are updated.

```

story_name:
    .byte "ZORK1.Z3"
story_name_end:

    lda #<story_name
    sta XRAM_NAMEPTR_L
    lda #>story_name
    sta XRAM_NAMEPTR_H
    lda #(story_name_end - story_name)
    sta XRAM_NAMELEN

    stz XRAM_ADDRL
    stz XRAM_ADDRM
    stz XRAM_ADDRH
    stz XRAM_LENL
    stz XRAM_LENH
    jsr xram_xload
    bne load_error

```

Filenames without an extension use `.xram`; explicit extensions are honored. See `docs/assembly/xram.md` for the full assembly ABI, build pattern, and fixture-backed integration tests.

Fixed-Address Overlays

`runtime/asm/overlay.inc` and `runtime/asm/overlay.s` provide a reusable loader for fixed-address executable overlays. The resident application owns a CPU RAM slot, links each overlay for that exact slot address, and calls `overlay_load_fixed` to validate a `NOVO` header and stream the payload into RAM through the shared pager/FIO path.

The first overlay runtime deliberately avoids relocation and dependency graphs: one active overlay slot, fixed load address, absolute entry vectors, and `A=0/A=1` status returns. This is the intended layer for optional app modules such as network lobby code, game-specific network adapters, local AI engines, and diagnostics that should not live permanently in the app kernel.

See `docs/assembly/overlay.md` for the header layout, loader ABI, and entry-call rules.

VGC Register-Level Programming

The VGC command pipeline works by writing parameters to registers A011–A01E, then writing the command byte to \$A010. The VGC executes the command synchronously. This is exactly what every graphics BASIC command does under the hood.

From assembly you can issue any VGC command directly:

```
; Issue GCLS command (clear graphics layer)
LDA \#$07
STA $A010
```

The full VGC command code reference, including copper commands (1B–1E) and memory I/O commands (19–1A), is documented in Appendix memmap.

SID Chip Access

The SID chip registers at D400–D41F are write-intercepted within the ROM address range. Assembly code can write to them directly:

```
; Set voice 0 to sawtooth waveform, gate on
LDA \#$21
STA $D404
```

The SID register layout matches the MOS 6581. Per-voice registers occupy 7 bytes each (voice 0 at \$D400, voice 1 at \$D407, voice 2 at \$D40E). Filter and volume registers are at D415–D418. See Appendix memmap for the full register map.

Warning

The BASIC INSTRUMENT, SOUND, and VOLUME commands manage SID registers directly. Direct SID register writes from assembly can conflict with active BASIC sound effects unless you coordinate voice use yourself.

Copper Programming from Assembly

From BASIC, the CUPPER keyword provides high-level access (see Chapter graphics). From assembly, write parameters to A011–A016 and the command code to \$A010:

```
; Add copper event: at Y=50, X=0, set BgCol to color 5
LDA \#00 : STAA011
LDA \#00 : STAA012
LDA \#50 : STA $A013
LDA \#01 : STAA014
LDA \#00 : STAA015
LDA \#05 : STAA016
LDA \#1B : STAA010
; Enable copper
LDA \#1D : STAA010
```

The copper also supports sprite registers at A040–A0BF. Pass the absolute address as P3/P4 (16-bit little-endian). For example, to change sprite 0's Y position at scanline 80:

```

; Copper event: at Y=80, X=0, set sprite 0 YLo to 150
LDA \#00 : STAA011
LDA \#00 : STAA012
LDA \#80 : STA $A013
LDA \#42 : STAA014
LDA \#A0 : STAA015
LDA \#150 : STA $A016
LDA \#1B : STAA010

```

Here P3 = \$A042 (sprite 0's Y register). P4 is reserved and should be 0. See Chapter graphics for the COPPER keyword syntax and sprite multiplexing patterns.

Sprite Registers from Assembly

The sprite register block at A040–A0BF provides direct memory-mapped access to all 16 sprites. Each sprite occupies 8 bytes with a stride of 8:

Sprite positions are Nova canvas coordinates. $X = 0\text{--}319$ and $Y = 0\text{--}199$ are the visible 320x200 graphics canvas; a 16x16 sprite is fully visible at top-left positions $X = 0\text{--}304$ and $Y = 0\text{--}184$. Values outside the visible range are useful for off-screen entry and exit and are clipped by the VGC.

Offset	Field	Size
+0	X position, low byte	1 byte
+1	X position, high byte	1 byte
+2	Y position (unsigned byte)	1 byte
+3	Reserved; reads as 0	1 byte
+4	Shape slot index (0–255)	1 byte
+5	Flags (bit 0=hflip, bit 1=vflip, bit 7=enable)	1 byte
+6	Priority (0=behind, 1=between, 2=front)	1 byte
+7	TransColor (per-sprite transparent color index, 0–15; >15 = all opaque)	1 byte

Sprite n starts at $\$A040 + n*8$. For example, sprite 3 starts at \$A058. To position sprite 3 at $X=200$, $Y=100$ and enable it:

```

; Sprite 3 base = $A058
LDA \#200 : STA $A058
LDA \#00 : STA $A059
LDA \#100 : STA $A05A
LDA \#00 : STA $A05B
; Enable (bit 7 of flags)
LDA \#$80 : STA $A05D

```

Writing directly to sprite registers is faster than going through the VGC command interface and is the preferred approach from assembly code.

DMA Controller from Assembly

The DMA controller at \$BA63 transfers data between six unified memory spaces. The pattern is: load parameters into registers, then write \$01 to the command register to start.

```
10 REM -- DMA copy 4000 bytes from CPU $6000 to char RAM --
20 POKE $BA63, 0 : REM source space = CPU RAM
30 POKE $BA64, 1 : REM dest space = Char RAM
40 POKE $BA65, 0 : POKE $BA66, $60 : POKE $BA67, 0
50 REM source addr = $6000 (low, mid, high)
60 POKE $BA68, 0 : POKE $BA69, 0 : POKE $BA6A, 0
70 REM dest addr = 0 (start of char RAM)
80 POKE $BA6B, $A0 : POKE $BA6C, $0F : POKE $BA6D, 0
90 REM length = 4000 ($0FA0)
100 POKE $BA6E, 0 : REM mode = copy (not fill)
110 POKE $BA63, 1 : REM start!
120 IF PEEK($BA61) = 1 THEN 120 : REM poll until not busy
130 IF PEEK($BA61) <> 2 THEN PRINT "Error:"; PEEK($BA62)
```

For fill mode, set bit 0 of DmaMode (BA6E) and load the fill byte into 'DmaFillValue' (BA6F).

Blitter from Assembly

The blitter at \$BA80 performs 2D rectangular copies and fills with row stride. Set up source and destination addresses, width, height, and stride, then write \$01 to BltCmd.

```
10 REM -- Scroll color RAM up by 1 row using blitter --
20 REM Source: row 1 (offset 80), Dest: row 0 (offset 0)
30 REM Width: 80, Height: 24, Stride: 80
40 POKE $BA83, 2 : POKE $BA84, 2 : REM src/dst = color RAM
50 POKE $BA85, 80 : POKE $BA86, 0 : POKE $BA87, 0
60 REM source offset = 80 (row 1)
70 POKE $BA88, 0 : POKE $BA89, 0 : POKE $BA8A, 0
80 REM dest offset = 0 (row 0)
90 POKE $BA8B, 80 : POKE $BA8C, 0 : REM width = 80
100 POKE $BA8D, 24 : POKE $BA8E, 0 : REM height = 24
110 POKE $BA8F, 80 : POKE $BA90, 0 : REM src stride = 80
120 POKE $BA91, 80 : POKE $BA92, 0 : REM dst stride = 80
130 POKE $BA93, 0 : REM mode = copy
140 POKE $BA80, 1 : REM start!
150 IF PEEK($BA81) = 1 THEN 150
```

Color-key mode: set bit 1 of BltMode (BA93) and load the transparent color into 'BltColorKey' (BA95). Source pixels matching the color key are skipped.

Network Controller from Assembly

The NIC at \$A100 provides TCP networking. To connect as a client from register-level code:

```
10 REM -- Connect to 127.0.0.1 port 8080 --
20 POKE $A102, 0 : REM slot 0
30 REM Write hostname to name buffer ($A120+)
40 H$ = "127.0.0.1"
50 FOR I = 1 TO LEN(H$)
```

```

60 POKE $A11F + I, ASC(MID$(H$, I, 1))
70 NEXT I
80 POKE $A11F + LEN(H$) + 1, 0 : REM null terminate
90 POKE $A108, $90 : POKE $A109, $1F : REM port 8080
100 POKE $A100, 1 : REM connect command
110 REM Poll slot status for connected bit
120 IF (PEEK($A118) AND 1) = 0 THEN 120

```

To send data, copy the message into CPU RAM, then set `NicDmaAddrL/H` and `NicDmaLen`, and write `$03` to `NicCmd`. To receive, write `$04` and read `NicMsgLen` for the actual length.

The Timer Controller

The timer controller at `$BA40` generates periodic IRQ interrupts, firing once every N video frames (at 60 Hz). It has no dedicated BASIC keyword – you configure it with `POKE` and handle its interrupts with `IRQ`.

Registers

Address	Name	Purpose
<code>\$BA40</code>	TimerCtrl	Bit 0 = enable
<code>\$BA41</code>	TimerStatus	Bit 0 = IRQ pending (reading clears)
<code>\$BA42</code>	TimerDivL	Divisor low byte
<code>\$BA43</code>	TimerDivH	Divisor high byte

The timer fires an IRQ every *divisor* video frames. A divisor of 60 fires once per second; a divisor of 1 fires every frame. The pending flag is cleared automatically when you read `TimerStatus`.

Example: one-second heartbeat

```

10 T = 0
20 IRQ 1000
30 POKE $BA42, 60 : POKE $BA43, 0
40 REM divisor = 60 frames = 1 second
50 POKE $BA40, 1 : REM enable
60 DO : LOOP UNTIL T >= 5
70 POKE $BA40, 0 : REM disable
80 PRINT "DONE"
90 END
1000 T = T + 1
1010 PRINT "TICK_"; T
1020 RETIRQ

```

Note

Disable the timer (`POKE $BA40, 0`) before changing the divisor.

Timer from Assembly

The timer at \$BA40 fires an IRQ every N video frames. To set up a 1-second timer (60 frames):

```
10 IRQ 1000
20 POKE $BA42, 60 : POKE $BA43, 0 : REM divisor = 60
30 POKE $BA40, 1 : REM enable timer
40 GOTO 40
1000 PRINT "TICK_";
1010 RETIRQ
```

Reading `TimerStatus` (\$BA41) clears the pending IRQ flag. The timer must be disabled with `POKE $BA40, 0` before changing the divisor.

Try It Now

Read the frame counter via PEEK

```
10 REM Read frame counter via PEEK
20 FOR I=1 TO 60:VSYNC:NEXT I
30 PRINT "FRAMES: ";PEEK(40968)
```

\$A008 = 40968 decimal. After 60 VSYNC waits (approximately one second at 60 Hz), the frame counter register reflects the elapsed frame ticks. The value will be somewhere in the range 0–255 because the counter wraps after 256 frames. Run the program several times and observe how the value changes.

DMA and Blitter

Speed is not a luxury – it is the difference between a program that feels alive and one that feels like it is apologising.

– e6502 Virtual Computer Design Notes

Every POKE loop that copies a screen buffer byte-by-byte is burning CPU cycles that your program could be spending on game logic, animation, or music. NovaBASIC includes two hardware accelerators that move data without occupying the CPU: the **DMA controller** for flat bulk transfers and the **blitter** for two-dimensional rectangular copies and fills. Both work across six unified memory spaces – CPU RAM, character RAM, color RAM, the graphics bitmap, sprite shape memory, and expansion RAM – so data can be moved between any combination of those regions in a single command. This chapter covers both controllers, their BASIC keywords, and the memory space model they share.

The DMA Controller

Direct Memory Access (DMA) is a hardware technique that transfers a block of bytes between two memory addresses without executing a load/store loop on the CPU. The CPU hands a source address, destination address, and length to the controller and the transfer happens autonomously. From the perspective of a BASIC program the operation appears instantaneous.

The DMA controller understands six **unified memory spaces**. Each space is identified by a small integer that you pass as a parameter:

ID	Space	Size	Notes
0	CPU RAM	64 KB	Full 6502 address space
1	Character RAM	4,000 bytes	80x50 text cells
2	Color RAM	4,000 bytes	80x50 color attributes
3	Graphics Bitmap	64,000 bytes	320x200 pixels, 4-bit color
4	Sprite Shapes	32,768 bytes	256 shape slots x 128 bytes each
5	Expansion RAM	up to 512 KB	Uses current XBANK

For spaces 1 through 4, the address parameter is a zero-based byte offset into that hardware memory region – so address 0 in space 1 is the first character cell, address 80 is the start of the second row, and so on.

DMACOPY srcSpace, srcAddr, dstSpace, dstAddr, length

Copies *length* bytes from offset *srcAddr* in *srcSpace* to offset *dstAddr* in *dstSpace*.

```
10 REM Copy 4000 bytes from CPU RAM at $6000 to character RAM
20 DMACOPY 0, 24576, 1, 0, 4000
```

Here space 0 is CPU RAM, 24576 is \$6000 decimal, space 1 is character RAM, and the destination offset is 0 (top-left cell). After line 20 executes, the 4000-byte buffer you prepared in CPU RAM has been written directly into the text screen.

```
10 REM Load graphics data from XRAM bank 0 into the bitmap
20 XBANK 0
30 DMACOPY 5, 0, 3, 0, 64000
```

Space 5 is expansion RAM. The current XBANK (set on line 20) determines which 512 KB bank is addressed. Line 30 copies all 64,000 bytes of the graphics bitmap from the start of XRAM bank 0.

DMAFILL dstSpace, dstAddr, length, value

Fills *length* bytes starting at offset *dstAddr* in *dstSpace* with the constant byte *value*.

```
10 REM Clear character screen with space characters (ASCII 32)
20 DMAFILL 1, 0, 4000, 32
```

```
10 REM Fill color RAM with color 7 (white-on-black attribute)
20 DMAFILL 2, 0, 4000, 7
```

Status functions

Function	Returns
DMASTATUS	0 = idle, 1 = busy, 2 = done, 3 = error
DMAERR	0 = no error; non-zero = hardware error code
DMACOUNT	Number of bytes transferred by the last operation

Note

DMACOPY and DMAFILL are asynchronous: they start the transfer and return immediately. Poll DMASTATUS until it changes from busy (1) to done (2) or error (3) when you need to wait for completion.

The Blitter

The DMA controller treats memory as a flat one-dimensional sequence of bytes. The **blitter** adds a second dimension: it understands that memory is arranged as rows of a fixed width and can copy or fill rectangular regions without touching the bytes between rows.

The key concept is **stride**: the number of bytes from the beginning of one row to the beginning of the next. For the 80-column text screen, stride is 80. For the 320-pixel-wide graphics bitmap, stride is 160 (320 pixels at 4 bits each, packed into 2 nibbles per byte). If source and destination have different strides – for example when copying a narrow block into a wider canvas – you specify each independently.

BLITCOPY srcSpace, srcAddr, srcStride, dstSpace, dstAddr, dstStride, width, height

Copies a rectangle *width* bytes wide by *height* rows tall from the source region to the destination region.

```
10 REM Scroll the character screen up by one row.
20 REM Copy rows 1-24 (offset 80) to rows 0-23 (offset 0),
30 REM then blank the last row.
40 BLITCOPY 1, 80, 80, 1, 0, 80, 80, 24
50 DMAFILL 1, 1920, 80, 32
```

Line 40 copies 24 rows of 80 characters, shifting the entire screen up. Line 50 fills the vacated bottom row (offset 1920 = 24x80) with spaces.

```
10 REM Copy a 10x8 block from CPU RAM at $3000 into color RAM
20 REM at column 5, row 3 (offset = 3*80+5 = 245).
30 BLITCOPY 0, 12288, 10, 2, 245, 80, 10, 8
```

The source stride is 10 because the block is stored compactly (row-by-row, 10 bytes per row) in CPU RAM. The destination stride is 80 because color RAM is 80 columns wide.

BLITFILL dstSpace, dstAddr, dstStride, width, height, value

Fills a rectangle *width* bytes wide by *height* rows tall in *dstSpace* with the constant byte *value*, respecting row stride.

```
10 REM Draw a 20x5 colored rectangle starting at column 10, row 2.
20 REM offset = 2*80+10 = 170. Stride = 80.
30 BLITFILL 2, 170, 80, 20, 5, 9
```

Without the blitter this would require a nested FOR/NEXT loop: one outer loop over five rows and an inner loop over twenty columns, each issuing a POKE. BLITFILL replaces all of that with a single hardware call.

Color-key transparency

At the register level, bit 1 of the blitter mode register (**\$BA93**) enables **color-key** mode: source bytes that equal the color key value (stored in **\$BA95**) are skipped, leaving the destination byte unchanged. This is sprite-style transparency for arbitrary memory regions. The **BLITCOPY** keyword does not expose this flag directly; use **POKE \$BA93, 2** before issuing a raw blitter start (**POKE \$BA80, 1**) to enable it.

Status functions

Function	Returns
BLITSTATUS	0 = idle, 1 = busy, 2 = done, 3 = error
BLITERR	0 = no error; non-zero = hardware error code
BLITCOUNT	Number of bytes written by the last operation

Tip

Use **DMACOPY** and **DMAFILL** for flat, one-dimensional transfers – copying a saved screen buffer, filling a region of expansion RAM, or moving a block of arbitrary bytes. Use **BLITCOPY** and **BLITFILL** when your data is inherently two-dimensional and you need to respect row boundaries, such as scrolling text, drawing rectangles, or working with sprite graphics. Choosing the right tool avoids writing stride arithmetic in BASIC loops.

Note

BLITCOPY and **BLITFILL** are also asynchronous. Use **BLITSTATUS**/**BLITERR**/**BLITCOUNT** to monitor progress and final result.

Memory Spaces Reference

The table below summarises the space IDs accepted by both the DMA controller and the blitter. Address offsets within each space are always zero-based.

ID	Space	Size	Notes
0	CPU RAM	64 KB	Full 6502 address space. Offsets are absolute CPU addresses. Writes to ROM (C000–FFFF) are silently ignored.
1	Character RAM	4,000 bytes	80x50 text cells, row-major. Offset 0 is top-left; offset 79 is top-right; offset 80 is the start of row 1.

ID	Space	Size	Notes
2	Color RAM	4,000 bytes	One color attribute byte per text cell, same layout as character RAM. Low nibble = foreground, high nibble = background.
3	Graphics Bitmap	64,000 bytes	320x200 pixels, two 4-bit pixels packed per byte (low nibble = left pixel). Row stride is 160 bytes.
4	Sprite Shapes	32,768 bytes	256 shape slots, 128 bytes each. Each slot holds a 16x16 pixel image, two pixels per byte. Sprites reference slots via shape index registers.
5	Expansion RAM	up to 512 KB	The current XBANK setting is automatically applied as the high address byte. Set XBANK before any DMA or blitter operation that uses space 5.
6	Tile Data RAM	32,768 bytes	Tile definition data.
7	Text Attribute RAM	4,000 bytes	One style byte per text cell, same layout as Character RAM. Bit 0 = flash.

Note

When space 5 (XRAM) is the source or destination, the DMA and blitter controllers read the current XBANK register at the moment the transfer starts. Changing XBANK mid-transfer is not safe; always set the desired bank before issuing the command.

DMA and Sprite Shape Memory

DMACOPY can transfer shape data directly into sprite shape memory (space 4). This is the fastest way to swap sprite animations at runtime: pre-store multiple animation frames in expansion RAM and copy them on demand.

```
10 REM Copy one 128-byte shape from XRAM offset 0 to shape slot 0
```

```
20 XBANK 0
30 DMACOPY 5, 0, 4, 0, 128
```

Shape slot addresses are computed as `slot x 128`. For example, shape slot 5 starts at byte offset 640 in sprite shape space:

```
10 REM Copy shape data from XRAM into slot 5
20 DMACOPY 5, 0, 4, 640, 128
```

Tip

DMA writes to sprite shape memory are **glitch-free** – the renderer snapshots shape data at the start of each frame, so transfers can happen at any time without waiting for vblank or disabling sprites. This makes DMA-driven sprite animation safe and straightforward.

Try It Now

Try It

The program below combines all four commands to produce a continuously scrolling rainbow of color bars using only hardware accelerators – no inner loops, no per-byte POKEs.

```
10 REM -- DMA & Blitter Demo -- 20 REM Clear screen 30 DMAFILL 1,
0, 4000, 32 40 DMAFILL 2, 0, 4000, 0 50 REM Draw 50 horizontal
color bars, one per row 60 FOR I = 0 TO 49 70 BLITFILL 2, I
*80, 80, 80, 1, I MOD 16 80 NEXT I 90 REM Scroll colors up
continuously 100 BLITCOPY 2, 80, 80, 2, 0, 80, 80, 49 110 BLITFILL
2, 49*80, 80, 80, 1, RND(1)*16 120 VSYNC 130 GOTO 100
```

Lines 30–40 clear both character and color RAM in two hardware calls. Lines 60–80 paint 50 horizontal bars of different colors by calling `BLITFILL` once per row. Lines 100–130 form an animation loop: `BLITCOPY` shifts the entire color layer up by one row, `BLITFILL` inserts a new random-color bar at the bottom, and `VSYNC` holds the frame rate at 60 Hz.

Experiments to try: - Change the fill value on line 30 to something other than 32 (ASCII space) to see different characters fill the screen – try 42 (asterisk) or 64 (at-sign). - Replace space 2 on lines 70, 100, and 110 with space 1 to scroll character codes instead of colors. - Try `DMACOPY 1, 0, 2, 0, 4000` to mirror the character RAM layout into color RAM and see the effect on the display. - Use space 5 (XRAM) as a destination: stash a full color-screen snapshot with `DMACOPY 2, 0, 5, 0, 4000` and restore it later.

Networking

The value of a network grows with the number of machines connected to it.

– Robert Metcalfe

NovaBASIC includes a built-in Network Interface Controller (NIC) that gives your programs TCP client networking from ordinary BASIC lines. You can open outbound connections to remote servers, send and receive text messages, and close the connection – all without touching a single hardware register.

The NIC provides four independent connection **slots** numbered 0 through 3. Each slot can connect to a remote host independently. Messages are automatically framed so that each **NSEND** call corresponds to exactly one **NRECV\$** call on the other end.

The Network Controller

The NIC implements message-oriented TCP networking. Rather than giving you a raw byte stream, it packages each message with a one-byte length prefix on the wire. This framing is invisible to BASIC: you call **NSEND** with a string and the receiver calls **NRECV\$** to get that same string back intact. You never manually prefix or parse message lengths.

Key characteristics of the NIC:

- **Four independent slots** (0–3), each with its own connection state.
- **Message queuing** – up to 16 incoming messages may be queued per slot before the sender is blocked at the hardware level.
- **Maximum message size** of 256 bytes per send/receive.
- **DMA-backed transfers** between NIC and 6502 RAM; BASIC hides this from you entirely.

The NIC registers occupy A100–A13F. You will not normally write to those addresses directly – the BASIC networking commands issue all necessary register writes on your behalf.

Note

The NIC handles all TCP details: handshake, buffering, framing, and teardown. From BASIC's point of view you simply send strings and receive strings. The length-prefix framing on the wire is automatic and transparent.

Connecting as a Client

A client connection opens a TCP socket to a remote host. All four NIC commands used for client operation – **NOPEN**, **NSEND**, **NRECV\$**, and **NCLOSE** – work with the same slot number from open to close.

Opening a connection: **NOPEN slot, "hostname", port**

```
NOPEN 0, "127.0.0.1", 8080
```

Establishes a TCP connection on *slot* (0–3) to *hostname* at *port* (1–65535). The hostname must be a quoted string; dotted-decimal IPv4 addresses are the most reliable form. The command blocks briefly while the TCP handshake completes; once it returns, the slot is ready to send and receive.

Sending data: **NSEND slot, "message"**

```
NSEND 0, "Hello, server!"
```

Transmits *message* through *slot*. The NIC automatically adds the one-byte length prefix before sending; the application on the far end sees only the payload. Maximum payload length is 256 bytes.

Receiving data: NRECV\$(slot)

```
A$ = NRECV$(0)
```

Dequeues and returns the next waiting message on *slot* as a string. If no message is queued, NRECV\$ returns an empty string immediately – it does not wait. Always poll with NREADY first to confirm a message is available before calling NRECV\$.

Checking for messages: NREADY(slot)

NREADY(slot) returns -1 (true) if at least one message is waiting in the slot's receive queue, or 0 (false) if the queue is empty. Use it in a tight polling loop:

```
40 IF NOT NREADY(0) THEN 40
```

Closing a connection: NCLOSE slot

```
NCLOSE 0
```

Closes the TCP connection on *slot* and releases all associated resources. Always call NCLOSE when you are finished with a slot, even if the remote end has already disconnected. This is required to return the slot to a clean state so it can be reused.

Complete client example

```
10 REM -- Simple TCP Client --
20 NOPEN 0, "127.0.0.1", 8080
30 NSEND 0, "Hello, server!"
40 IF NOT NREADY(0) THEN 40
50 A$ = NRECV$(0)
60 PRINT "Reply: "; A$
70 NCLOSE 0
```

Line 20 connects to a server running on the same machine at port 8080. Line 30 sends a greeting. Line 40 polls until a reply arrives. Line 50 reads the reply into A\$. Line 60 prints it. Line 70 closes the slot.

Tip

NREADY and the polling loop on line 40 are the standard pattern for waiting on a response. Because NovaBASIC is single-threaded, the loop will spin tightly until data arrives. For interactive programs you can add a VSYNC inside the loop to yield to the display system and keep the screen responsive while waiting.

Status and Error Handling

NSTATUS(slot) bitmask

NSTATUS(slot) returns a byte whose bits describe the current state of *slot*:

Bit	Name	Meaning
0	Connected	An active TCP connection exists on this slot.
1	DataReady	At least one message is waiting in the receive queue.
2	SendReady	The slot is ready to accept an outbound message (normally always set when Connected).
3	Error	A connection or I/O error has occurred on this slot.
4	RemoteClosed	The remote peer has closed its end of the connection.

Testing individual bits

Use the AND operator to isolate a specific bit:

```
REM Test bit 0 (Connected)
IF (NSTATUS(0) AND 1) = 0 THEN PRINT "Disconnected"

REM Test bit 4 (RemoteClosed)
IF (NSTATUS(0) AND 16) <> 0 THEN GOSUB 900
```

Detecting disconnection

There are two ways the remote end can disappear:

- **Clean close** – the remote side called a close equivalent. Bit 4 (*RemoteClosed*) is set and bit 0 (*Connected*) clears.
- **Abrupt drop** – the network connection is lost without a graceful close. Bit 3 (*Error*) may be set; bit 0 clears.

In either case, when bit 0 drops to zero the connection is gone. Check `(NSTATUS(slot) AND 1) = 0` as your primary disconnection test.

Cleaning up after disconnection

Regardless of which party initiated the close, always call `NCLOSE slot` once you detect a disconnection. This returns the slot to the idle state and allows `NOPEN` to be called on it again.

Warning

Always check `NSTATUS` before calling `NSEND`. If bit 0 (*Connected*) is clear, the connection has dropped and `NSEND` will fail silently – your message will be discarded without any error being reported to BASIC. Test the connection state at the top of every send/receive loop.

Try It Now

Try It

Start a TCP echo server on another machine, then enter this client and type RUN:

```
10 REM -- Test Client --
20 NOPEN 0, "127.0.0.1", 8080
30 NSEND 0, "Hello from client!"
40 IF NOT NREADY(0) THEN 40
50 A$ = NRCV$(0)
60 PRINT "Echo: "; A$
70 NCLOSE 0
```

The server window should print the message and its byte length. The client window should print ECHO: HELLO FROM CLIENT!.

Experiments to try:

- Open slot 1 in the client program with a second NOPEN call and send a different message on each slot simultaneously to observe independent operation.
- Extend the client into a loop that sends ten numbered messages and prints each echo, then closes the connection.

Command Reference

BITCLR

Clear bits: `mem[addr] = mem[addr] AND NOT mask`.

BITSET

Set bits: `mem[addr] = mem[addr] OR mask`.

BITTGL

Toggle bits: `mem[addr] = mem[addr] EOR mask`.

BITTST

Return -1 if *all* masked bits are set at *addr*, else 0.

BLITCOPY

Copy rectangle: *w* bytes wide, *h* rows, with source stride *sstr* and destination stride *dstr*.

See the Dma And Blitter guide for more details.

BLITFILL

Fill rectangle with byte *val*.

See the Dma And Blitter guide for more details.

CALL

NovaBASIC uses **SYS addr** for machine-code entry. **CALL** is kept here only as a compatibility note for older drafts of the documentation.

Use **SYS** for current programs.

CIRCLE

Draw a circle outline centred at (cx, cy) .

See the Graphics And Display guide for more details.

CLEAR

Clear all variables and arrays; program is retained.

CLS

Clear the text screen and home the cursor.

See the Language Fundamentals guide for more details.

COLOR

Set text foreground, optional background, and optional border color (0–15).

See the Language Fundamentals guide for more details.

CONT

Continue execution after **STOP** or Ctrl-C.

COPPER__ADD

Add copper event: set *reg* to *val* at position (x, y) . Registers: BGCOL, MODE, SCROLLX, SCROLLY, \$A0EA (scroll control), SPRX(*n*), SPRXH(*n*), SPRY(*n*), SPRSHAPE(*n*), SPRFLAGS(*n*), SPRPRI(*n*). Sprite Y high is reserved.

Advanced use: numeric register \$FE does not write a display register. It raises VGC IRQ pending bits from *val*, so the copper can request an interrupt at an arbitrary beam position.

See the Graphics And Display guide for more details.

COPPER__CLEAR

Remove all events from the current target list.

See the Graphics And Display guide for more details.

COPPER_LIST

Set target list to n (0–127). Subsequent ADD/CLEAR edit this list. Reset target list back to the active list.

See the Graphics And Display guide for more details.

COPPER_OFF

Disable copper execution.

See the Graphics And Display guide for more details.

COPPER_ON

Enable copper execution each frame.

See the Graphics And Display guide for more details.

COPPER_USE

Switch active copper list to n at next vblank (double-buffering).

See the Graphics And Display guide for more details.

DATA

Embed constant values for READ.

DEC

Decrement a numeric variable by 1.

Variable names are case-insensitive, and only the first two characters are significant. **SC**, **SCORE**, and **score** refer to the same variable.

DEEK

Read a 16-bit word (little-endian) from a 6502 address (function).

See the Expansion Memory guide for more details.

DEF

Define a single-line user function.

Function and parameter names follow the normal variable-name rule: names may be written longer, but only the first two characters are significant.

DEL

Delete a saved program from disk.

See the First Session guide for more details.

DIM

Declare a single-dimension array.

Array names follow the normal variable-name rule: names may be written longer, but only the first two characters are significant. `GR`, `GRID`, and `grid` refer to the same array.

DIR

List all saved `.bas` programs.

See the First Session guide for more details.

DMACOPY

Copy *len* bytes from space *ss* address *sa* to space *ds* address *da*.

See the Dma And Blitter guide for more details.

DMAFILL

Fill *len* bytes in space *ds* at address *da* with byte *val*.

See the Dma And Blitter guide for more details.

DO

Begin an indefinite loop body.

DOKE

Write a 16-bit word (little-endian) to a 6502 address.

See the Expansion Memory guide for more details.

END

Terminate program and return to direct mode.

FETCH

Copy bytes from XRAM back into CPU RAM.

Raw form:

```
FETCH ram,offset,length
```

The raw form reads from *offset* in the active `XBANK`.

Named form:

```
FETCH "name",ram
```

The named form loads the full named block and uses the stored block length.

See the Expansion Memory guide for more details.

FILL

Draw a filled solid rectangle.

See the Graphics And Display guide for more details.

FLASH

Turn on flashing text for subsequent printed characters.

Flashing is stored per text cell, so text printed while **FLASH** is active keeps flashing after **FLASHOFF**.

FLASHOFF

Turn off flashing for subsequent printed characters.

FONT

Switches the active character font for the entire screen.

Syntax

FONT **n**

n – Font slot number (0-7):

Slot	Font	Description
0	CP437	DOS/IBM character set (default)
1	PETSCII Upper	C64 uppercase + graphic characters
2	PETSCII Lower	C64 lowercase + uppercase

Examples

```
10 FONT 1
20 PRINT "HELLO_IN_PETSCII"
30 FONT 0
```

Keyboard Graphics

When a PETSCII font is active (slots 1 or 2), special key combinations produce graphic characters:

- **Shift + A-Z** – Shifted graphic characters (line-drawing, shapes)
- **Ctrl + A-Z** – Commodore-key graphic characters (additional shapes)

See the PETSCII Characters guide for the full character map.

Notes

- Font affects display only – stored program data is unchanged
- Works with POKE too: POKE 40967,1 (40967 = \$A007)
- The BASIC prompt and all text output use the active font

FOR

Begin a counted loop; step defaults to 1.

Variable names are case-insensitive, and only the first two characters are significant. `IX`, `INDEX`, and `index` refer to the same loop variable.

GCLS

Clear the graphics layer to transparent (all pixels 0).

See the Graphics And Display guide for more details.

GCOLOR

Set the graphics draw color (0–15; only the low nibble is used).

See the Graphics And Display guide for more details.

GET

Read a single keypress into variable (non-blocking).

Variable names are case-insensitive, and only the first two characters are significant. Prefer one- or two-character names such as `K` or `KY`.

GLOAD

Load a `.gfx` file into VGC memory. If *len* is omitted, loads the entire file. *space* uses the VDC-style VRAM/DMA plane IDs: 1=screen, 2=color, 3=gfx bitmap, 4=sprite shapes, 7=text attributes.

See the First Session guide for more details.

GOSUB

Call subroutine at line; return address is stacked.

GOTO

Jump unconditionally to a line number.

GSAVE

Save VGC memory to a `.gfx` file. *space* uses the VDC-style VRAM/DMA plane IDs: 1=screen, 2=color, 3=gfx bitmap, 4=sprite shapes, 7=text attributes.

See the First Session guide for more details.

IF

Conditional execution; `ELSE` is optional.

INC

Increment a numeric variable by 1.

Variable names are case-insensitive, and only the first two characters are significant. `SC`, `SCORE`, and `score` refer to the same variable.

INPUT

Display optional prompt and read from keyboard.

Variable names are case-insensitive, and only the first two characters are significant. Prefer one- or two-character names such as `NM$` or `IN$`.

INSTRUMENT

Define instrument preset *id* (0–15). *wave*: 10=tri,20=saw, 40=pulse,80=noise. *a,d,s,r*: ADSR values 0–15.

See the Sound And Music guide for more details.

IRQ

Redirect the IRQ vector to a BASIC line number handler.

For sprite collision events, use `ON SPRITE COLLISION GOSUB line`. For board button/DIP-switch change events, enable `$BA9E`, handle status at `$BA9F`, and acknowledge the W1C latches before `RETIRQ`.

See the Assembly guide for more details.

LET

Assign value to variable (`LET` is optional).

Variable names are case-insensitive, and only the first two characters are significant. `SC`, `SCORE`, and `score` refer to the same variable.

LINE

Draw a straight line between two points.

See the Graphics And Display guide for more details.

LIST

Display program lines; omit range to list all.

LOAD

Load a previously saved program from disk.

See the First Session guide for more details.

LOCATE

Move the text cursor to column *x* (0–79), row *y* (0–24).

See the Language Fundamentals guide for more details.

LOOP

End loop; re-enter while *expr* is true. End loop; re-enter until *expr* becomes true.

MIDPLAY

Load and play a standard MIDI file through the hosted music engine.

```
10 MIDPLAY "entertainer"  
20 IF PLAYING THEN GOTO 20  
30 PRINT "DONE"
```

The engine auto-selects the busiest MIDI channels, maps GM instruments to the available Nova audio voices, and converts MIDI velocity to per-voice volume.

See the Sound And Music guide for more details.

MIDSTOP

Stop MIDI file playback.

See the Sound And Music guide for more details.

MODE

Display mode: 0=text only, 1=graphics over text, 2=text over graphics, 3=graphics+sprites only (no text).

See the Graphics And Display guide for more details.

MUSIC

Load MML sequence into *voice* (1–6).

See the Sound And Music guide for more details.

MUSIC_LOOP

Enable looping (restart when all voices finish). Disable looping (default).

See the Sound And Music guide for more details.

MUSIC_PLAY

Start music playback.

See the Sound And Music guide for more details.

MUSIC_PRIORITY

Set hosted MML voice priority on hosts that support priority routing. Simple SID **SOUND** uses its own round-robin scheduler.

See the Sound And Music guide for more details.

MUSIC_STOP

Stop music and silence all music voices.

See the Sound And Music guide for more details.

MUSIC_TEMPO

Set playback tempo (default 120).

See the Sound And Music guide for more details.

NCLOSE

Close a connection.

See the Networking guide for more details.

NEW

Clear program and all variables from memory.

NEXT

Advance loop variable and branch back if not done.

Variable names are case-insensitive, and only the first two characters are significant. **IX**, **INDEX**, and **index** refer to the same loop variable.

NMI

Redirect the NMI vector to a BASIC line number handler.

See the Assembly guide for more details.

NOPEN

Connect to a remote TCP server.

See the Networking guide for more details.

NSEND

Send a message string.

See the Networking guide for more details.

NULL

Set the number of null (zero) bytes sent after each carriage return.

ON

Branch to the n th line in the list. Call the n th subroutine in the list.

PAINT

Flood-fill from seed point (x,y) .

See the Graphics And Display guide for more details.

PEEK

Read an 8-bit byte from a 6502 address (function).

See the Expansion Memory guide for more details.

PLOT

Set pixel at (x,y) to the current draw color.

See the Graphics And Display guide for more details.

POKE

Write an 8-bit byte to a 6502 address.

See the Expansion Memory guide for more details.

PRINT

Output values to screen; ; suppresses newline.

READ

Read successive values from DATA statements.

Variable names are case-insensitive, and only the first two characters are significant. Prefer one- or two-character names in type-in listings.

RECT

Draw a rectangle outline.

See the Graphics And Display guide for more details.

REM

Program comment; rest of line is ignored by interpreter.

RESET

Perform a full hardware and CPU reset. This returns the CPU and custom chips to their initial state, including display/video memory, sprites, DMA, blitter, audio chips, and host-facing devices.

RESTORE

Reset the DATA pointer, optionally to a line.

RETIRQ

Return from an IRQ handler (re-enables interrupts).

See the Assembly guide for more details.

RETNMI

Return from an NMI handler.

See the Assembly guide for more details.

RETURN

Return from the most recent GOSUB.

REVERSE

Turn on reverse text for subsequent printed characters.

Without arguments, NovaBASIC swaps the current text foreground and background. With `fg,bg`, reverse text uses those explicit colors instead.

REVERSEOFF

Turn off reverse text for subsequent printed characters.

RUN

Execute program from the lowest line number.

SAVE

Save the current BASIC program to disk as `name.bas`.

See the First Session guide for more details.

SFLOAD

Load a soundfont for MIDI playback.

```
10 SFLOAD "piano.sf2"  
20 MIDPLAY "song.mid"
```

See the Sound And Music guide for more details.

SIDPLAY

Play a `.sid` file. The optional *song* argument defaults to 1.

See the Sound And Music guide for more details.

SIDSTOP

Stop SID file playback.

See the Sound And Music guide for more details.

SOUND

Start a fire-and-forget SID note. *note* is a MIDI note byte, *dur* is the duration in video frames, and optional *inst* selects instrument preset 0–15 (default 0). Call `VSYNC` in loops so NovaBASIC can advance durations and gate notes off on time.

See the Sound And Music guide for more details.

SPRITE

Enable (show) sprite *n*, disable it, or set its screen position. Sprite positions use Nova canvas coordinates: X=0–319 and Y=0–199 are visible. A 16x16 sprite is fully visible at top-left positions X=0–304 and Y=0–184. Out-of-range positions are clipped at the visible canvas; sprites do not draw into the hardware border.

`SPRITE COLLISION ON` enables the VGC sprite-collision IRQ source. `SPRITE COLLISION OFF` disables it, and `SPRITE COLLISION CLEAR` clears the latched 16-bit sprite collision mask and acknowledges the pending IRQ bit.

See the Graphics And Display guide for more details.

SPRITEDATA

Define one row of sprite shape data (8 bytes, 16 pixels). Bytes b3–b8 are optional; omitted bytes leave those pixels unchanged.

See the Graphics And Display guide for more details.

SPRITESET

Write *value* to register field *field* (0–7) of sprite *n* (0–15). Field 0 accepts an unsigned 16-bit Nova canvas X value and writes X low/high. Other fields are bytes: 2=Nova canvas Y, 3=reserved, 4=shape, 5=flags, 6=priority, 7=transparent color. Sprite attribute writes are hardware-buffered and published at the frame boundary.

See the Graphics And Display guide for more details.

SPRITESHAPE

Assign shape slot *shape* (0–255) to sprite *n* (0–15).

See the Graphics And Display guide for more details.

STASH

Copy *length* bytes from CPU RAM into XRAM.

Raw form:

```
STASH ram,offset,length
```

The raw form writes to *offset* in the active XBANK.

Named form:

```
STASH "name",ram,length
```

The named form allocates XRAM automatically and records the block under *name*, so you can later fetch it without tracking its raw address.

See the Expansion Memory guide for more details.

STOP

Break execution; CONT resumes at next statement.

SWAP

Exchange the values of two variables.

Variable names are case-insensitive, and only the first two characters are significant. SC, SCORE, and score refer to the same variable.

SYS

Execute a machine-code routine at 6502 address *addr*. Optional byte arguments load the 6502 A, X, and Y registers before the call. The routine must return with RTS to resume BASIC.

Example:

```
10 SYS 36864
20 SYS 36864,60,10,1
```

See the Assembly guide for more details.

UNPLOT

Clear pixel at (*x,y*) to 0 (transparent).

See the Graphics And Display guide for more details.

VOLUME

Set SID volume.

- VOLUME level – set master volume (0–15).
- VOLUME level, voice – set per-voice volume (0–15) on voice 1–6.

Both master and per-voice volume share the same 0–15 range, matching the 4-bit resolution of the original SID. Per-voice volume is a NovaBASIC extension not present on the real MOS 6581. Default per-voice volume is 15 (full). The SID master volume powers on at 0, so set **VOLUME** before playing sound. Setting a per-voice volume to 0 silences that voice without affecting others.

```
10 VOLUME 15          : REM MASTER VOLUME MAX
20 VOLUME 12, 1       : REM VOICE 1 SLIGHTLY REDUCED
30 VOLUME 5, 2        : REM VOICE 2 QUIET
```

See the Sound And Music guide for more details.

VPEEK

Read one byte from VGC video memory.

plane selects the video memory plane and uses the same IDs as DMA, blitter, and **GSAVE/GLOAD**: 1=character RAM, 2=color RAM, 3=graphics bitmap, 4=sprite shape RAM, and 7=text attribute RAM.

Text character, color, and attribute memory use **address = row * 80 + column**.

```
10 A=VPEEK(1,0)       : REM character at upper-left
20 C=VPEEK(2,0)       : REM color attribute at upper-left
```

VPOKE

Write one byte to VGC video memory.

plane selects the video memory plane and uses the same IDs as DMA, blitter, and **GSAVE/GLOAD**: 1=character RAM, 2=color RAM, 3=graphics bitmap, 4=sprite shape RAM, and 7=text attribute RAM.

Text character, color, and attribute memory use **address = row * 80 + column**.

```
10 VPOKE 1,0,65       : REM "A" at upper-left
20 VPOKE 2,0,$1E      : REM yellow foreground on blue background
```

VSYNC

Wait for the next video frame boundary (60 Hz).

See the Graphics And Display guide for more details.

WAIT

Busy-wait until **(PEEK(addr) XOR xor) AND mask** is non-zero.

WIDTH

Set the output line width in characters.

XALLOC

Allocate an unnamed XRAM block using the low-level allocator. Prefer named **STASH/FETCH** blocks or raw offsets unless you are writing diagnostics or tooling that inspects the XMC registers directly.

See the Expansion Memory guide for more details.

XBANK

Select the active 64 KB XRAM bank; n must be less than the total bank count. Raw BASIC XRAM commands combine the active bank with their 16-bit offset. The equivalent assembly flat address is $n * 65536 + \text{offset}$.

See the Expansion Memory guide for more details.

XDEL

Delete a named XRAM block.

See the Expansion Memory guide for more details.

XDIR

List all named XRAM blocks and their sizes.

See the Expansion Memory guide for more details.

XFREE

Release a raw XRAM range in the active **XBANK** from usage tracking. Any named or unnamed allocation metadata overlapping that range is removed.

See the Expansion Memory guide for more details.

XLOAD

Load a file directly into XRAM at *offset* in the active **XBANK**. If *length* is omitted or zero, the entire file is loaded. Filenames without an extension use **.xram**; explicit extensions such as **.z3** are honored.

XLOAD streams through the shared XRAM runtime and does not consume a CPU RAM staging buffer. BASIC addresses XRAM as **XBANK** plus a 16-bit offset, while the underlying runtime converts that to the same flat 24-bit address used by assembly callers.

Examples:

```
10 REM LOAD WHOLE FILE INTO XRAM BANK 0 OFFSET 0
20 XBANK 0
30 XLOAD "STORY.Z3",0

10 REM LOAD ONLY THE FIRST 4096 BYTES
20 XBANK 2
30 XLOAD "LEVEL1",8192,4096
```

XMAP

Map a 256-byte XRAM page in the active **XBANK** to CPU window 0–3 (**\$BC00–\$BFFF**). The low byte of *offset* is ignored because windows are page-aligned.

See the Expansion Memory guide for more details.

XMEM

Print XRAM bank count and page usage statistics.

See the Expansion Memory guide for more details.

XPEEK

Read one byte from *offset* in the active **XBANK**. **XPEEK** is a numeric function and can be used inside expressions.

See the Expansion Memory guide for more details.

XPOKE

Write one byte to *offset* in the active **XBANK**. Use **STASH** for block copies; **XPOKE** is intended for small byte-level updates or diagnostics.

See the Expansion Memory guide for more details.

XRESET

Clear all XRAM allocation and named-block state. **XRESET** does not need to erase every raw byte in XRAM, but all names, handles, and page-usage records are discarded.

See the Expansion Memory guide for more details.

XSAVE

Save *length* bytes directly from XRAM at *offset* in the active **XBANK** to a file. Filenames without an extension use **.xram**; explicit extensions are honored.

XSAVE streams through the shared XRAM runtime and does not require a CPU RAM staging buffer. BASIC addresses XRAM as **XBANK** plus a 16-bit offset, while the underlying runtime converts that to the same flat 24-bit address used by assembly callers.

Example:

```
10 REM SAVE 4096 BYTES FROM BANK 1 OFFSET 0
20 XBANK 1
30 XSAVE "SNAPSHOT",0,4096
```

XUNMAP

Unmap a CPU window (0–3).

See the Expansion Memory guide for more details.

Function Reference

ABS

Absolute value.

ADDR

Returns the 16-bit address for a generated runtime label that has a resolved ROM, RAM, or MMIO address.

Labels are generated from `@label` comments in the assembly runtime sources and are documented in `docs/assembly/runtime-labels.md`. Extension-only or link-time library labels may appear in the generated reference without being available to `ADDR()`. Unknown or unresolved labels raise `Function Call Error`.

ASC

ASCII code of the first character.

ATN

Arctangent of n , result in radians.

BIN

Binary string representation of integer n .

BIN_STR

Binary string representation of integer n .

BLITCOUNT

Number of bytes written in last blit operation.

See the Dma And Blitter guide for more details.

BLITERR

Blitter error code from last operation.

See the Dma And Blitter guide for more details.

BLITSTATUS

Blitter status: 0=idle, 1=busy, 2=ok, 3=error.

See the Dma And Blitter guide for more details.

BUMPED

Legacy sprite-to-background collision helper. For all 16 sprites, prefer **SPRBG**, which returns the full 16-bit latched background collision mask and clears it.

See the Graphics And Display guide for more details.

CHR

Single-character string for ASCII code n .

CHR_STR

Single-character string for ASCII code n .

COLLISION

Legacy sprite-to-sprite collision helper. For all 16 sprites, prefer **SPRCOLL**, which returns the full 16-bit latched collision mask and clears it.

See the Graphics And Display guide for more details.

COS

Cosine of n radians.

DMACOUNT

Number of bytes transferred in last DMA operation.

See the Dma And Blitter guide for more details.

DMAERR

DMA error code from last operation.

See the Dma And Blitter guide for more details.

DMASTATUS

DMA status: 0=idle, 1=busy, 2=ok, 3=error.

See the Dma And Blitter guide for more details.

EXP

e raised to the power n .

FRE

Free BASIC program memory in bytes (x is ignored).

HEX

Hexadecimal string representation of integer n .

HEX_STR

Hexadecimal string representation of integer n .

INT

Truncate toward zero to integer.

LCASE

String converted to lowercase.

LEFT

First n characters of string.

LEN

Length of string in characters.

LOG

Natural logarithm ($\ln n$).

MATAN2

Return an 8-bit angle for the signed vector **dx,dy**.

The result uses 0..255 for one full turn: 0 points right, 64 points up, 128 points left, and 192 points down.

MAX

The larger of two numeric values.

MCOS

Return cosine from the math coprocessor for an 8-bit angle, where 0..255 is one full turn.

The result is signed 1.7 fixed point in the range -128..127.

MCROSSH

Return the signed high word of the signed 16-bit 2D cross product.

The full operation is $\mathbf{ax*by} - \mathbf{ay*bx}$. Use **MCROSSL** for the unsigned low word.

MCROSSL

Return the unsigned low word of the signed 16-bit 2D cross product.

The full operation is $\mathbf{ax*by} - \mathbf{ay*bx}$. Use **MCROSSH** for the signed high word.

MDIST

Return the math coprocessor's approximate distance for signed 16-bit deltas.

The approximation is $\max(\text{abs}(\mathbf{dx}), \text{abs}(\mathbf{dy})) + 3/8 * \min(\text{abs}(\mathbf{dx}), \text{abs}(\mathbf{dy}))$.

MDIV

Divide signed 16-bit **n** by signed 16-bit **d** using the math coprocessor.

Returns the signed quotient. Division by zero returns -1.

MDOTFX

Return the signed Q8.8 dot product of two signed Q8.8 vectors.

The result is saturated to signed 16-bit Q8.8.

MDOTS16H

Return the signed high word of the signed 16-bit vector dot product.

The full operation is $\mathbf{ax*bx} + \mathbf{ay*by}$. Use **MDOTS16L** for the unsigned low word.

MDOTS16L

Return the unsigned low word of the signed 16-bit vector dot product.

The full operation is $\mathbf{ax*bx} + \mathbf{ay*by}$. Use **MDOTS16H** for the signed high word.

MID

Substring starting at *start* (1-based), optional length.

MIN

The smaller of two numeric values.

MLEN2

Return the low 16 bits of $\mathbf{x*x} + \mathbf{y*y}$ using the math coprocessor vector unit.

Use small vectors when calling from BASIC if the full squared length matters.

MMUL16H

Return the signed high word of a signed 16-bit by signed 16-bit multiply.

Use **MMUL16L(a,b)** for the unsigned low word of the same 32-bit product.

MMUL16L

Return the unsigned low word of a signed 16-bit by signed 16-bit multiply.

Use MMUL16H(a,b) for the signed high word of the same 32-bit product.

MMULFX

Multiply two signed Q8.8 fixed-point values using the math coprocessor.

The result is a saturated signed Q8.8 value.

MNOTE

Current hosted music MIDI note number on *voice* (1–6), or 0 if silent. Direct **SOUND** effects are not reported by this function.

See the Sound And Music guide for more details.

MREM

Divide signed 16-bit **n** by signed 16-bit **d** using the math coprocessor.

Returns the signed remainder. Division by zero returns -1.

MRND

Return one 0..255 random byte from the math coprocessor RNG.

MSCALX

Scale signed Q8.8 vector component **x** by signed Q8.8 scalar **s**.

Returns the scaled signed Q8.8 x component.

MSCALY

Scale signed Q8.8 vector component **y** by signed Q8.8 scalar **s**.

Returns the scaled signed Q8.8 y component.

MSIN

Return sine from the math coprocessor for an 8-bit angle, where 0..255 is one full turn.

The result is signed 1.7 fixed point in the range -128..127.

NREADY

-1 if a message is ready, 0 otherwise.

See the Networking guide for more details.

NRECV

Receive next queued message as a string (max 255 chars).

See the Networking guide for more details.

NRECV_STR

Receive next queued message as a string (max 255 chars).

See the Networking guide for more details.

NSTATUS

Slot status bitmask (see Appendix memmap).

See the Networking guide for more details.

PI

Constant pi 3.14159...

PLAYING

1 if hosted music or file-backed playback is currently playing, 0 if stopped. Direct **SOUND** effects are not reported by this function.

See the Sound And Music guide for more details.

POS

Current text cursor column position (x is ignored).

RIGHT

Last n characters of string.

RND

Pseudo-random number in $[0, 1)$; n seeds or advances the sequence.

SADD

Address of the string's data in the string pool.

SGN

Sign of n : -1, 0, or 1.

SIN

Sine of n radians.

SPRBG

Return the 16-bit sprite-to-background collision bitmask. Bit n is set when sprite n touched a non-transparent graphics pixel. Reading this function clears the latched mask and acknowledges the sprite-background IRQ source.

SPRCOLL

Return the 16-bit sprite-to-sprite collision bitmask. Bit n is set when sprite n participated in a sprite collision. Reading this function clears the latched mask and acknowledges the sprite collision IRQ source.

SPRITE_X

Current X position of sprite n as an unsigned 16-bit Nova canvas coordinate, read directly from the sprite registers.

See the Graphics And Display guide for more details.

SPRITE_Y

Current Y position of sprite n as an unsigned 8-bit Nova canvas coordinate, read directly from the sprite registers.

See the Graphics And Display guide for more details.

SQR

Square root.

STR

Numeric value converted to a string.

STR_STR

Numeric value converted to a string.

TAN

Tangent of n radians.

TWOPI

Constant 2π 6.28318...

UCASE

String converted to uppercase.

USR

Call user machine-code routine; pass x in FAC, return value in FAC.

VAL

Numeric value parsed from the leading digits of a string.

VARPTR

Address of a numeric or string variable in memory.

Variable names are case-insensitive, and only the first two characters are significant. **VARPTR(SC)** and **VARPTR(SCORE)** refer to the same numeric variable.

Memory Map

Address Space Overview

The e6502 virtual computer presents a flat 64 KB address space to the 6502 CPU. Coprocessor regions (VGC, NIC, FIO, XMC, DMA, Blitter) respond to reads and writes within their assigned windows; all remaining space is RAM except the upper 16 KB (C000–FFFF) which is write-protected ROM.

Address Range	Size	Region
0000–00FF	256 B	Zero Page
0100–01FF	256 B	CPU Stack
0200–027F	128 B	System Vectors (IRQ/NMI handlers, BASIC vectors)
0280–9FFF	39,680 B	BASIC Program RAM
A000–A01F	32 B	Virtual Graphics Controller (VGC) registers and command interface
A03F	1 B	ROM swap/control register
A040–A0BF	128 B	Sprite Registers (16 sprites x 8 bytes)
A0E0–A0E4	5 B	VDC-style VRAM port (char/color/gfx/sprite/text-attribute memory)
A0E5	1 B	VGC display dimmer (0=black, 15=full brightness)
A0E6–A0EC	7 B	VGC text, fixed-palette, scroll, and collision high-byte controls
A0F0–A0FF	16 B	VGC IRQ block
A100–A13F	64 B	Network Interface Controller (NIC) registers
A140–A1FF	192 B	Wavetable Synthesizer (WTS) registers and event scheduler
B9A0–B9EF	80 B	File I/O Controller (FIO) registers
B9F0–B9FF	16 B	Boot/runtime coordination flags

Address Range	Size	Region
BA00–BA3F	64 B	Expansion Memory Controller (XMC) registers
BA40–BA4F	16 B	Timer Controller registers
BA50–BA62	19 B	Hosted Music Status and Voice Note Readback (14 voices)
BA63–BA75	19 B	DMA Controller registers
BA83–BA9B	25 B	Blitter Controller registers
BA9C–BAA1	6 B	Board Input registers
BAB0–BB1F	112 B	File metadata buffer
BB20–BB4F	48 B	Math Coprocessor registers
BB50–BB8D	62 B	SID sound-effect runtime state
BB8E–BBCD	64 B	Active MIDI/WTS soundfont name metadata
BC00–BFFF	1,024 B	XMC Memory Windows (4 x 256-byte mapped pages)
C000–FFFF	16,384 B	ROM (NovaBASIC interpreter)
D400–D41F	32 B	SID chip registers (inside ROM range; writes intercepted)
D420–D43F	32 B	SID chip 2 registers (inside ROM range; writes intercepted)
D500–D51F	32 B	SID chip 2 mirror (transparently routes to \$D420)

Note

The address range A020–A03E is reserved for language/host services. Unlisted addresses in A0ED–A0EF and A200–A9FF are not claimed by any coprocessor and fall through to the underlying flat RAM. The ranges BA9E–BAAF and BBCE–BBFF are similarly unallocated RAM. BAB0–BB1F is the file metadata buffer, BB20–BB4F is the math coprocessor, BB50–BB8D is used by NovaBASIC’s SID sound-effect scheduler, and BB8E–BBCD stores the active MIDI/WTS soundfont name. SID registers at D400–D43F occupy space within the ROM address range but are intercepted on write by the SID chip emulators.

Boot Runtime Flags

The range B9F0–B9FF is reserved for runtime boot coordination between the host and language ROMs. Language implementations should preserve these bytes unless they explicitly implement the same boot ABI.

Address	Name	Access	Description
\$B9F0	AutobootSkip	R/W	Non-zero disables the language’s startup autoboot attempt.

Address	Name	Access	Description
\$B9F1	AutobootActive	R/W	Non-zero while a startup autoboot program is being entered; runtimes use this to suppress broken-autoboot errors and fall back to Ready .

Board Input Registers

The board input registers expose physical board controls as active-high, debounced logical state. These addresses are the NovaVM software ABI for board inputs; the current FPGA implementation wires them to the ULX3S pushbuttons and DIP switches. Other boards, such as Arty, may use different physical pins or controls internally, but ports should keep these addresses for compatible application code where practical. Reads on hosts without physical board inputs return the host-provided state, which defaults to zero.

Address	Name	Access	Description
\$BA9C	BoardInputButtons	RO	Bits 0–6: PWR, FIRE1, FIRE2, UP, DOWN, LEFT, RIGHT. Bit 7 is zero.
\$BA9D	BoardInputSwitches	RO	Bits 0–3: DIP switches SW1–SW4. Bits 4–7 are zero.
\$BA9E	BoardInputIrqEnable	R/W	IRQ enable mask: bit 0 = button changes, bit 1 = switch changes.
\$BA9F	BoardInputIrqStatus	R/W1C	Pending change sources: bit 0 = one or more button bits changed, bit 1 = one or more switch bits changed. Write 1 bits to acknowledge.
\$BAA0	BoardInputButtonChanges	R/W1C	Latched mask of changed button bits since the last clear. Write 1 bits to clear.
\$BAA1	BoardInputSwitchChanges	R/W1C	Latched mask of changed DIP-switch bits since the last clear. Write 1 bits to clear.

Note

On ULX3S, \$BA9C maps bit 0 to PWR and bits 1–6 to FIRE1, FIRE2, UP, DOWN, LEFT, and RIGHT. PWR is also the FPGA reset input, so application code cannot normally poll it while it is held. \$BA9D maps bits 0–3 to ULX3S DIP switches SW1–SW4. The six game-style buttons and four DIP switches are intended for application use.

Board-input IRQs are opt-in. The CPU IRQ line is asserted while (BoardInputIrqStatus & BoardInputIrqEnable) != 0. Status and change masks latch after the debounced state changes, so handlers can read \$BA9C/\$BA9D, inspect \$BAA0/\$BAA1, then write 1 bits back to \$BA9F–\$BAA1 before returning from the interrupt. Existing polling code can ignore \$BA9E–\$BAA1.

Basic IRQ handler sketch:

```
10 IRQ 1000
20 POKE $BA9F,3:POKE $BAA0,127:POKE $BAA1,15
30 POKE $BA9E,3
40 GOTO 40
1000 S=PEEK($BA9F):B=PEEK($BA9C):W=PEEK($BA9D)
1010 BM=PEEK($BAA0):WM=PEEK($BAA1)
1020 POKE $BAA0,BM:POKE $BAA1,WM:POKE $BA9F,S
1030 RETIRQ
```

The visual BASIC demo docs/programs/board_input_demo.bas reads these registers each frame and lights on-screen indicators for the live button and DIP-switch state. docs/programs/board_input_irq_demo.bas uses the IRQ registers so the visual state is refreshed only after a debounced input change.

Math Coprocessor Registers

The math coprocessor occupies BB20–BB4F. Multi-byte arguments and results are little-endian. Writing the high byte of a trigger operand starts the operation; the FPGA holds RDY low until the result is available.

Address	Name	Access	Description
BB20 – –BB23	MathMul16A/B	R/W	Signed 16 x 16 multiply. Write \$BB23 to trigger; result is signed 32-bit in BB38 – –BB3B.
BB24 – –BB27	MathMulFxA/B	R/W	Signed Q8.8 multiply. Write \$BB27 to trigger; saturated Q8.8 result is in BB38 – –BB39.

Address	Name	Access	Description
<i>BB28</i> – –BB2D	MathDivN/D	R/W	Signed 32 / signed 16 division. Write \$BB2D to trigger; quotient is in <i>BB38</i> – –BB39, remainder is in <i>BB3A</i> – –BB3B.
\$BB2E	MathSinCosAngle	W	Write unsigned 8-bit angle to trigger; sine is in \$BB38 and cosine is in \$BB39 as signed 1.7 values.
<i>BB2F</i> – –BB32	MathAtanDy/Dx	R/W	Signed 16-bit dy/dx. Write \$BB32 to trigger; unsigned angle is in \$BB38 and approximate hypot is in <i>BB39</i> – –BB3A.
<i>BB33</i> – –BB36	MathDistDx/Dy	R/W	Signed 16-bit dx/dy. Write \$BB36 to trigger; approximate unsigned distance is in <i>BB38</i> – –BB39.
\$BB37	MathRng	R/W	Read advances xorshift32 and returns the low byte; writes seed one byte of the 32-bit state round-robin.
<i>BB38</i> – –BB3B	MathRes0–3	R	Shared result bank. Read before triggering another operation.
\$BB3C	MathCaps0	R	Capability bits: bit 0=MUL16, bit 1=MULFX, bit 2=SINCOS, bit 3=DIST_APPROX, bit 4=RNG, bit 5=DIV, bit 6=ATAN2.

Address	Name	Access	Description
\$BB3D	MathCaps1	R	Capability bits: bit 0=VEC_DOT_S16, bit 1=VEC_DOT_FX, bit 2=VEC_CROSS_S16, bit 3=VEC_LEN2, bit 4=VEC_SCALE_FX.
\$BB3E	MathStatus	R	Last operation status: bit 0=divide by zero, bit 1=overflow/saturation, bit 7=unimplemented operation.
\$BB3F	MathVersion	R	Math coprocessor ABI version; v2 reads \$02.
<i>BB40</i> – – <i>BB49</i>	MathVecA/B/Scalar	R/W	Vector operand bank: AX, AY, BX, BY, and scalar, all signed 16-bit values.
\$BB4E	MathVecOp	W	Write operation code to trigger: 1=DOT_S16, 2=DOT_FX, 3=CROSS_S16, 4=LEN2, 5=SCALE_FX. Results use <i>BB38</i> – – <i>BB3B</i> .

VGC Register Map

The Virtual Graphics Controller occupies A000–A01F, with extension registers at A0E0–A0FF. Registers A000–A00F are the core status and display registers. Registers A010–A01F are the command register and its 15 parameter slots; writing to \$A010 both stores the command byte and triggers immediate execution. The VGC IRQ block lives at A0F0–A0FF.

Core Registers (A000–A00F)

Address	Name	Access	Description
\$A000	RegMode	R/W	Display mode: 0=text only, 1=graphics over text, 2=text over graphics, 3=graphics and sprites only (no text).
\$A001	RegBgCol	R/W	Background color index (0–15); reset value is 0 (black).
\$A002	RegFgCol	R/W	Default foreground color index (0–15); reset value is 15 (light grey).
\$A003	RegCursorX	R/W	Text cursor column (0–79).
\$A004	RegCursorY	R/W	Text cursor row (0–49).
\$A005	RegScrollX	R/W	Horizontal scroll offset low byte. Bit 8 lives in RegScrollCtl bit 0.
\$A006	RegScrollY	R/W	Vertical scroll offset (0–199 modulo).
\$A007	RegFont	R/W	Active font slot (0–7). 0=CP437, 1=PETSCII Upper, 2=PETSCII Lower.
\$A008	RegStatus	RO	Frame counter; incremented each video frame. Writes are ignored.
\$A009	RegSpriteCount	RO	Count of currently enabled sprites (0–16). Writes are ignored.
\$A00A	RegCursorEnable	R/W	Bit 0 enables the cursor blink; reset default is hidden.
\$A00B	RegColSt	R/WC	Sprite-to-sprite collision low byte for sprites 0–7; write any value to clear this byte.
\$A00C	RegColBg	R/WC	Sprite-to-background collision low byte for sprites 0–7; write any value to clear this byte.

Address	Name	Access	Description
\$A00D	RegBorder	R/W	Border color index (0–15); reset value is 11 (dark grey).
\$A00E	RegCharOut	R/W	Character output port; writing outputs a character to the text screen.
\$A00F	RegCharIn	R/W	Character input port; reading dequeues the next keypress byte.

Text, Palette, Scroll, and Collision Extension Registers (A0E6–A0EC)

Address	Name	Access	Description
\$A0E6	RegTextFlags	R/W	Text output flags: bit 0=reverse, bit 1=use explicit reverse attribute, bit 2=flash.
\$A0E7	RegTextReverseAttr	R/W	Packed reverse color attribute used when bit 1 is set: high nibble=background, low nibble=foreground.
\$A0E8	RegGfxTransparentColor	R/W	Graphics-plane transparent color index (0–15); reset value is 0.
\$A0E9	RegPaletteMode	R/W	Fixed VGC palette selector. Bit 0: 0=C64/Nova palette, 1=IBM EGA palette. Reset value is 0.
\$A0EA	RegScrollCtl	R/W	Scroll control. Bit 0=RegScrollX bit 8, bit 1=apply scroll to graphics, bit 2=apply scroll to text. Reset value is 6 (graphics+text enabled, X bit 8 clear).
\$A0EB	RegColStHi	R/WC	Sprite-to-sprite collision high byte for sprites 8–15; write any value to clear this byte.

Address	Name	Access	Description
\$A0EC	RegColBgHi	R/WC	Sprite-to-background collision high byte for sprites 8–15; write any value to clear this byte.

IRQ Control (A0F0–A0FF)

Address	Name	Access	Description
\$A0F0	RegIrqEnable	R/W	Enabled IRQ source mask. Bit 0=vblank, bits 1–4=copper/user events 0–3, bit 5=sprite collision, bit 6=sprite-background collision.
\$A0F1	RegIrqStatus	R/W1C	Pending IRQ source mask. Write 1 bits to acknowledge/clear pending sources.
\$A0F2	RegIrqForce	WO	Set enabled pending bits for diagnostics/tests.
\$A0F3	RegIrqValid	RO	Valid implemented source mask (\$7F).

The CPU IRQ line is asserted while `(RegIrqStatus & RegIrqEnable) != 0`. Use \$A0F1 to acknowledge handled sources before returning from an IRQ handler. The VGC does not expose a fixed C64-style raster-line IRQ register; use copper events for beam-position IRQs.

Command Register and Parameters (A010–A01F)

Address	Name	Access	Description
\$A010	RegCmd	R/W	Command byte; writing triggers immediate command execution.
\$A011	RegP0	R/W	Parameter 0.
\$A012	RegP1	R/W	Parameter 1.
\$A013	RegP2	R/W	Parameter 2.
\$A014	RegP3	R/W	Parameter 3.
\$A015	RegP4	R/W	Parameter 4.
\$A016	RegP5	R/W	Parameter 5.

Address	Name	Access	Description
\$A017	RegP6	R/W	Parameter 6.
\$A018	RegP7	R/W	Parameter 7.
\$A019	RegP8	R/W	Parameter 8.
\$A01A	RegP9	R/W	Parameter 9.
\$A01B	RegP10	R/W	Parameter 10.
\$A01C	RegP11	R/W	Parameter 11.
\$A01D	RegP12	R/W	Parameter 12.
\$A01E	RegP13	R/W	Parameter 13.
\$A01F	RegP14	R/W	Parameter 14.

Multi-byte parameters (coordinates, sprite positions) are packed little-endian across consecutive parameter registers. For example, a 16-bit x-coordinate uses P0 (low byte) and P1 (high byte).

VGC Command Codes

All commands are invoked by writing the command byte to **RegCmd** (\$A010). Parameters must be loaded into **RegP0–RegP14** before the write.

Graphics Commands (01–09)

Code	Name	Parameters and behavior
\$01	CmdPlot	P0/P1 = x (16-bit), P2/P3 = y (16-bit). Set pixel to current draw color.
\$02	CmdUnplot	P0/P1 = x (16-bit), P2/P3 = y (16-bit). Clear pixel to 0 (transparent).
\$03	CmdLine	P0/P1 = x0, P2/P3 = y0, P4/P5 = x1, P6/P7 = y1. Draw Bresenham line.
\$04	CmdCircle	P0/P1 = cx, P2/P3 = cy, P4/P5 = radius. Draw circle outline.
\$05	CmdRect	P0/P1 = x0, P2/P3 = y0, P4/P5 = x1, P6/P7 = y1. Draw rectangle outline.
\$06	CmdFill	P0/P1 = x0, P2/P3 = y0, P4/P5 = x1, P6/P7 = y1. Draw filled rectangle.
\$07	CmdGcls	No parameters. Clear entire graphics bitmap to 0.
\$08	CmdGcolor	P0 low nibble = color index (0–15). Set current draw color.

Code	Name	Parameters and behavior
\$09	CmdPaint	P0/P1 = x (16-bit), P2/P3 = y (16-bit). Flood-fill from seed point.

Sprite Commands (10–18)

Code	Name	Parameters and behavior
\$10	CmdSprDef	P0 = sprite (0–15), P1 = x pixel (0–15), P2 = y pixel (0–15), P3 = color nibble. Set one pixel in sprite shape.
\$11	CmdSprRow	P0 = sprite (0–15), P1 = row (0–15), P2–P9 = 8 data bytes (two 4-bit pixels per byte). Define one sprite row.
\$12	CmdSprClr	P0 = sprite (0–15). Clear all 128 bytes of sprite shape data to 0.
\$13	CmdSprCopy	P0 = source sprite (0–15), P1 = destination sprite (0–15). Copy shape data.
\$14	CmdSprPos	P0 = sprite (0–15), P1/P2 = x (unsigned 16-bit), P3 = y (unsigned 8-bit), P4 reserved. Set screen position.
\$15	CmdSprEna	P0 = sprite (0–15). Enable sprite; increments RegSpriteCount .
\$16	CmdSprDis	P0 = sprite (0–15). Disable sprite; decrements RegSpriteCount .
\$17	CmdSprFlip	P0 = sprite (0–15), P1 = flags (0=none, 1=horizontal, 2=vertical, 3=both).
\$18	CmdSprPri	P0 = sprite (0–15), P1 = priority (0=behind all, 1=between text/gfx, 2=in front).

Tip

Sprite shape data is accessed through the VDC-style VRAM port, DMA, blitter, or sprite commands rather than a direct CPU memory window.

Memory I/O Commands (19–1A)

Code	Name	Parameters and behavior
\$19	CmdMemRead	P0 = memory space, P1/P2 = address (16-bit), P4 bit 0 = auto-increment. Read byte from VGC memory; result in P3.
\$1A	CmdMemWrite	P0 = memory space, P1/P2 = address (16-bit), P3 = data byte, P4 bit 0 = auto-increment. Write byte to VGC memory.

Memory spaces match the VDC-style VRAM port and DMA/blitter IDs: 1=character RAM (4000 B), 2=color RAM (4000 B), 3=graphics bitmap (64000 B), 4=sprite shape RAM (32768 B; 256 slots), 7=text attribute RAM. Auto-increment advances the address after each read or write.

Copper Commands (1B–1E, 20–22)

The copper triggers register writes or IRQ events at specific raster positions each frame. FPGA builds store 8 independent copper lists (0–7), each holding up to 32 events; simulation builds may expose more lists for software compatibility. Active-list swaps take effect at vblank.

Code	Name	Parameters and behavior
1B <i>CmdCopperAdd</i> P0/P1 = <i>X</i> (16-bit), P2 = <i>Y</i> , P3/P4 = <i>register</i> (0 – 15, A000 – A00F, A0EA for scroll control, A040 – A0BF for sprite registers, or 'FE' for IRQ event), P5 = value. Adds to the target list. Replaces existing event at same position/register. Max 32 events per list on FPGA.		
\$1C	CmdCopperClear	No parameters. Remove all events from the target list.
\$1D	CmdCopperEnable	No parameters. Start executing the active copper list each frame.
\$1E	CmdCopperDisable	No parameters. Stop executing copper.
\$20	CmdCopperList	P0 = list index (0–127). Set target list for ADD/CLEAR.

Code	Name	Parameters and behavior
\$21	CmdCopperUse	P0 = list index (0–127). Set pending active list (swaps at next vblank).
\$22	CmdCopperListEnd	No parameters. Reset target list to the currently active list.

System Reset Command (\$1F)

Code	Name	Parameters and behavior
\$1F	CmdSysReset	No parameters. Performs a full machine reset: CPU reset plus all custom-chip state, including VGC registers/video memory, sprites, DMA, blitter, SID chips, and host-facing devices. This is the command invoked by the BASIC RESET statement.

Copper-writable registers: RegMode (A000), RegBgCol (A001), RegScrollX (A005), RegScrollY (A006), RegScrollCtl (A0EA), and sprite registers A040–A0BF (all eight fields, including TransColor at offset +7).

SID Chip Registers

The SID chip occupies D400–D41F within the ROM address range. Writes to these addresses are intercepted by the SID emulator; reads return the underlying ROM byte. The register layout matches the original MOS 6581.

Per-Voice Registers (3 voices, 7 bytes each)

Offset	Description
+0	Frequency low byte.
+1	Frequency high byte (16-bit SID frequency units).
+2	Pulse width low byte.
+3	Pulse width high byte (12-bit, bits 0–11 only).
+4	Control register: bit 0=gate, bit 4=triangle, bit 5=sawtooth, bit 6=pulse, bit 7=noise.
+5	Attack (bits 7–4) / Decay (bits 3–0).
+6	Sustain (bits 7–4) / Release (bits 3–0).

Voice 0: D400–D406. Voice 1: D407–D40D. Voice 2: D40E–D414.

Filter and Volume Registers

Address	Description
\$D415	Filter cutoff low (bits 0–2).
\$D416	Filter cutoff high (bits 0–7).
\$D417	Resonance (bits 7–4) / Filter route (bits 3–0, one bit per voice + external).
\$D418	Volume (bits 3–0) / Filter mode (bit 4=LP, bit 5=BP, bit 6=HP).

Second SID Chip (D420–D43F)

A second SID chip is mapped at D420–D43F with an identical register layout offset by 20 from the primary. A legacy mirror at D500 – –D51F routes transparently to \$D420.

Voice mapping: voices 1–3 use SID 1 (*D400*), *voices4 – –6useSID2(D420)*. Simple BASIC SOUND, VOLUME, and INSTRUMENT write these SID registers directly. MUSIC voice, "mml" uses the hosted MML path.

Note

The BASIC commands INSTRUMENT, SOUND, and VOLUME manage SID registers automatically. Direct writes to \$D400+ or \$D420+ are for advanced use only and may conflict with active SID sound effects.

Timer Controller and Hosted Music Status

Timer Controller (BA40–BA4F)

The timer controller provides periodic interrupt generation. It can be configured from assembly for custom timing.

Address	Name	Access	Description
\$BA40	TimerCtrl	R/W	Control register: bit 0 = enable.
\$BA41	TimerStatus	RO	Status: bit 0 = IRQ pending (reading clears).
\$BA42	TimerDivL	R/W	Divisor low byte.
\$BA43	TimerDivH	R/W	Divisor high byte.

The timer fires an IRQ every *divisor* video frames (at 60 Hz). A divisor of 1 fires every frame; a divisor of 60 fires once per second. Reading *TimerStatus* clears the pending IRQ flag.

Hosted Music Status (BA50–BA62)

Address	Access	Description
\$BA50	RO	Hosted music status flags: bit 0 = hosted SFX playing, bit 1 = hosted music playing, bit 2 = SID source, bit 3 = WTS/MIDI source, bit 4 = hosted music or soundfont load in progress. Direct BASIC SOUND is not reflected here.
\$BA51	RO	Voice 1 current MIDI note (0 = silent). SID 1, voice 1.
\$BA52	RO	Voice 2 current MIDI note (0 = silent). SID 1, voice 2.
\$BA53	RO	Voice 3 current MIDI note (0 = silent). SID 1, voice 3.
\$BA54	RO	Voice 4 current MIDI note (0 = silent). SID 2, voice 1.
\$BA55	RO	Voice 5 current MIDI note (0 = silent). SID 2, voice 2.
\$BA56	RO	Voice 6 current MIDI note (0 = silent). SID 2, voice 3.
<i>BA57</i> – –BA5E	RO	Voices 7–14 current MIDI notes (0 = silent). WTS voices 1–8.
<i>BA5F</i> – –BA60	RO	Elapsed hosted playback frames at 60 Hz, little-endian.
<i>BA61</i> – –BA62	RO	Total hosted playback frames at 60 Hz, little-endian when known.

These registers are read by the `PLAYING` and `MNOTE()` functions.

DMA Controller Register Map

The DMA Controller occupies BA63–BA75 and provides bulk memory transfers across six unified memory spaces. Writing \$01 to `DmaCmd` triggers the configured operation; poll `DmaStatus` for completion.

DMA Registers

Address	Name	Access	Description
\$BA60	<code>DmaCmd</code>	W	Command byte; write \$01 to start transfer.
\$BA61	<code>DmaStatus</code>	RO	Status: 00=idle,01=busy, 02=ok,03=error.

Address	Name	Access	Description
\$BA62	DmaErrCode	RO	Error detail code (see Appendix limits).
\$BA63	DmaSrcSpace	R/W	Source memory space (0–5).
\$BA64	DmaDstSpace	R/W	Destination memory space.
\$BA65	DmaSrcL	R/W	Source address low byte.
\$BA66	DmaSrcM	R/W	Source address mid byte.
\$BA67	DmaSrcH	R/W	Source address high byte.
\$BA68	DmaDstL	R/W	Destination address low byte.
\$BA69	DmaDstM	R/W	Destination address mid byte.
\$BA6A	DmaDstH	R/W	Destination address high byte.
\$BA6B	DmaLenL	R/W	Transfer length low byte.
\$BA6C	DmaLenM	R/W	Transfer length mid byte.
\$BA6D	DmaLenH	R/W	Transfer length high byte.
\$BA6E	DmaMode	R/W	Mode flags: bit 0 = fill mode.
\$BA6F	DmaFillValue	R/W	Fill byte (used when bit 0 of DmaMode is set).
\$BA70	DmaCountL	RO	Bytes transferred, low byte.
\$BA71	DmaCountM	RO	Bytes transferred, mid byte.
\$BA72	DmaCountH	RO	Bytes transferred, high byte.

DMA Memory Spaces

ID	Space
0	CPU RAM (64 KB)
1	VGC Character RAM (4,000 bytes)
2	VGC Color RAM (4,000 bytes)
3	VGC Graphics Bitmap (64,000 bytes)
4	VGC Sprite Shapes (32,768 bytes; 256 slots)
5	Expansion RAM (uses current XBANK)

ID	Space
6	VGC Tile Data (32,768 bytes)
7	VGC Text Attribute RAM (4,000 bytes)

Blitter Controller Register Map

The Blitter Controller occupies BA80–BA9F and provides 2D rectangular copy and fill operations with configurable row stride. It shares the same six memory spaces as the DMA controller. Writing \$01 to `BltCmd` triggers the configured operation.

Blitter Registers

Address	Name	Access	Description
\$BA80	<code>BltCmd</code>	W	Command byte; write \$01 to start blit.
\$BA81	<code>BltStatus</code>	RO	Status: 00=idle,01=busy, 02=ok,03=error.
\$BA82	<code>BltErrCode</code>	RO	Error detail code (see Appendix limits).
\$BA83	<code>BltSrcSpace</code>	R/W	Source memory space (0–5).
\$BA84	<code>BltDstSpace</code>	R/W	Destination memory space (0–5).
\$BA85	<code>BltSrcL</code>	R/W	Source base address low byte.
\$BA86	<code>BltSrcM</code>	R/W	Source base address mid byte.
\$BA87	<code>BltSrcH</code>	R/W	Source base address high byte.
\$BA88	<code>BltDstL</code>	R/W	Destination base address low byte.
\$BA89	<code>BltDstM</code>	R/W	Destination base address mid byte.
\$BA8A	<code>BltDstH</code>	R/W	Destination base address high byte.
\$BA8B	<code>BltWidthL</code>	R/W	Rectangle width low byte.
\$BA8C	<code>BltWidthH</code>	R/W	Rectangle width high byte.
\$BA8D	<code>BltHeightL</code>	R/W	Rectangle height low byte.
\$BA8E	<code>BltHeightH</code>	R/W	Rectangle height high byte.
\$BA8F	<code>BltSrcStrideL</code>	R/W	Source row stride low byte.

Address	Name	Access	Description
\$BA90	BltSrcStrideH	R/W	Source row stride high byte.
\$BA91	BltDstStrideL	R/W	Destination row stride low byte.
\$BA92	BltDstStrideH	R/W	Destination row stride high byte.
\$BA93	BltMode	R/W	Mode flags: bit 0 = fill, bit 1 = color-key.
\$BA94	BltFillValue	R/W	Fill byte (when bit 0 of BltMode is set).
\$BA95	BltColorKey	R/W	Transparent color (when bit 1 of BltMode is set).
\$BA96	BltCountL	RO	Bytes written, low byte.
\$BA97	BltCountM	RO	Bytes written, mid byte.
\$BA98	BltCountH	RO	Bytes written, high byte.

Note

When color-key mode is active (bit 1 of **BltMode**), source pixels matching **BltColorKey** are skipped and the destination byte is left unchanged. This enables transparent sprite and graphics overlays.

NIC Register Map

The Network Interface Controller occupies A100–A13F and provides message-oriented TCP networking over four connection slots. Writing a command byte to **NicCmd** triggers the operation on the slot selected by **NicSlot**.

NIC Registers

Address	Name	Access	Description
\$A100	NicCmd	W	Command byte; writing triggers execution.
\$A101	NicStatus	RO	Global status flags.
\$A102	NicSlot	R/W	Active slot ID (0–3).
\$A103	NicIrqCtrl	R/W	IRQ enable mask (one bit per slot).
\$A104	NicIrqStatus	RO	IRQ pending flags (reading clears).

Address	Name	Access	Description
\$A105	NicCmdSeq	R/W	Command sequence byte used by shared NIC helpers.
\$A106	NicCmdShadow	R/W	Last command byte written before <code>NicCmd</code> .
\$A107	NicHostCtrl	WO	Host-only control; bit 0 starts RX payload DMA.
\$A108	NicRemotePortL	R/W	Remote port low byte (for connect).
\$A109	NicRemotePortH	R/W	Remote port high byte.
\$A10A	NicLocalPortL	R/W	Local port low byte (for listen).
\$A10B	NicLocalPortH	R/W	Local port high byte.
\$A110	NicDmaAddrL	R/W	DMA RAM address low byte.
\$A111	NicDmaAddrH	R/W	DMA RAM address high byte.
\$A112	NicDmaLen	R/W	DMA length (1–255; \$00 = 256).
\$A113	NicMsgLen	RO	Last received message length.
\$A114	NicDmaStatus	RO	Hardware payload DMA status.
\$A115	NicDmaErr	RO	Hardware payload DMA error code.
\$A118	NicSlotStatus0	RO	Slot 0 status flags.
\$A119	NicSlotStatus1	RO	Slot 1 status flags.
\$A11A	NicSlotStatus2	RO	Slot 2 status flags.
\$A11B	NicSlotStatus3	RO	Slot 3 status flags.
A120–A13F	NicNameBuf	R/W	Hostname buffer (32 bytes ASCII).

NIC Command Codes

Code	Name	Behavior
\$01	NicCmdConnect	Connect slot to remote host (hostname in <code>NicNameBuf</code> , port in <code>NicRemotePortL/H</code>).
\$02	NicCmdDisconnect	Close active slot connection.
\$03	NicCmdSend	Send message from CPU RAM (<code>NicDmaAddrL/H</code> , <code>NicDmaLen</code>).

Code	Name	Behavior
\$04	NicCmdRecv	Receive next queued message into CPU RAM.
\$05	NicCmdListen	Start TCP server on local port (NicLocalPortL/H).
\$06	NicCmdAccept	Accept pending connection on listening slot.

NIC Slot Status Bits

Bit	Name	Meaning
0	Connected	Slot has an active TCP connection.
1	DataReady	Receive queue has at least one message (or pending accept).
2	SendReady	Slot is ready to accept a send operation.
3	Error	Connection failed, I/O error, or queue overflow.
4	RemoteClosed	Remote peer closed the connection.

NIC Global Status Bits (\$A101)

Bit	Name	Meaning
0	Ready	NIC is initialized and ready.
1	AnyData	At least one slot has queued data.
7	AnyError	At least one slot has an error flag set.

NIC Payload DMA Bits

NicCmdSend copies the requested CPU RAM payload into the FPGA NIC TX buffer before NovaHost receives the NIC event. For receive, NovaHost writes the FPGA NIC RX buffer, writes NicMsgLen, then writes bit 0 of NicHostCtrl; hardware copies the staged payload into CPU RAM at NicDmaAddrL/H.

Register	Bit/Value	Meaning
NicHostCtrl	bit 0	Start RX DMA from the NIC RX buffer into CPU RAM.
NicDmaStatus	bit 0	TX buffer contains the latest SEND payload.
NicDmaStatus	bit 1	RX DMA completed.
NicDmaStatus	bit 6	DMA error.
NicDmaStatus	bit 7	DMA busy.

Register	Bit/Value	Meaning
NicDmaErr	\$00	No DMA error.
NicDmaErr	\$01	Requested CPU RAM range is invalid.
NicDmaErr	\$02	DMA engine was already busy.

FIO Register Map

The File I/O Controller occupies B9A0–B9EF. Writing to \$B9A0 (**FioCmd**) triggers the operation. The caller polls \$B9A1 (**FioStatus**) for completion.

FIO Registers

Address	Name	Access	Description
\$B9A0	FioCmd	R/W	Command byte; writing triggers the operation.
\$B9A1	FioStatus	RO	Result status: 0=idle, 2=ok, 3=error.
\$B9A2	FioErrCode	RO	Error detail code (see below).
\$B9A3	FioNameLen	R/W	Filename length in bytes (1–63).
\$B9A4	FioSrcL	R/W	Source/destination address, low byte; NVGLOAD XRAM staging address low byte.
\$B9A5	FioSrcH	R/W	Source/destination address, high byte; NVGLOAD XRAM staging address middle byte.
\$B9A6	FioEndL	R/W	End address, low byte (used by SAVE to determine program extent); NVGLOAD XRAM staging address high byte.
\$B9A7	FioEndH	R/W	End address, high byte.
\$B9A8	FioSizeL	RO	Loaded data size low byte; directory-entry size byte 0 after DIR read.

Address	Name	Access	Description
\$B9A9	FioSizeH	RO	Loaded data size high byte; directory-entry size byte 1.
\$B9AA	FioGSpace / FioSize2	R/W	Graphics memory space for GSAVE/GLOAD/NVGLOAD; XRAM address high byte for XLOAD/XSAVE; directory-entry size byte 2 after DIR read.
\$B9AB	FioGAddrL	R/W	Graphics/XRAM offset, low byte; NVGLOAD bitmap destination low byte.
\$B9AC	FioGAddrH	R/W	Graphics/XRAM offset, high byte; NVGLOAD bitmap destination high byte.
\$B9AD	FioGLenL	R/W	Graphics/XRAM transfer length, low byte; ignored by NVGLOAD.
\$B9AE	FioGLenH	R/W	Graphics/XRAM transfer length, high byte; ignored by NVGLOAD.
\$B9AF	FioDirType	RO	Directory entry type: 0=PRG, 1=SID.
B9B0–B9EF	FioName	R/W	Filename buffer (64 bytes ASCII, not null-terminated).

FIO Command Codes

Code	Name	Behavior
\$01	FioCmdSave	Save bytes from FioSrcL/H to FioEndL/H (exclusive) to disk; prepends a 2-byte load-address.
\$02	FioCmdLoad	Load file into RAM at FioSrcL/H; skips the 2-byte load-address prefix; sets FioSizeL/H.

Code	Name	Behavior
\$03	FioCmdDirOpen	Open the program directory; populates FioName , FioSizeL/H/2 , and FioDirType with the first entry.
\$04	FioCmdDirRead	Advance to the next directory entry; populates FioName , FioSizeL/H/2 , and FioDirType .
\$05	FioCmdDelete	Delete the named program from disk.
\$06	FioCmdGSave	Save VGC memory space to a .gfx file. FioGSpace=space , FioGAddrL/H=offset , FioGLenL/H=length .
\$07	FioCmdGLoad	Load .gfx file into VGC memory space. FioGSpace=space , FioGAddrL/H=offset , FioGLenL/H=max length .
\$08	FioCmdSidPlay	Load and play a .sid file. FioSrcL=song number (1-based).
\$09	FioCmdSidStop	Stop SID file playback.
\$0A	FioCmdInstrument	Legacy/host audio ABI for instrument presets. NovaBASIC INSTRUMENT writes SID runtime state directly.
\$0B	FioCmdSound	Legacy/host audio ABI for simple SFX. NovaBASIC SOUND writes SID registers directly.
\$0C	FioCmdVolume	Legacy/host audio ABI for volume. NovaBASIC VOLUME writes SID registers directly.
\$0D	FioCmdMSeq	Load MML sequence. FioSrcL=voice (1–6), FioEndL/H=string pointer , FioNameLen=string length .
\$0E	FioCmdMPlay	Start music playback.
\$0F	FioCmdMStop	Stop music playback.
\$10	FioCmdMTempo	Set tempo. FioSrcL/H=BPM (16-bit).
\$11	FioCmdMLoop	Set loop. FioSrcL=0 (off) or 1 (on).

Code	Name	Behavior
\$18	FioCmdXLoad	Load a file directly into XRAM. FioGSpace/GAddrL/GAddrH=24-bit destination, FioGLenL/H=max length or 0 for full file.
\$19	FioCmdXSave	Save XRAM directly to a file. FioGSpace/GAddrL/GAddrH=24-bit source, FioGLenL/H=length.
\$2B	FioCmdNvgLoad	Clear graphics bitmap memory, decode a .nvg/NVG1 file into the caller-provided 64,000-byte XRAM staging buffer, then blit that buffer into graphics memory. FioGSpace must be 3; FioGAddrL/H is the bitmap destination offset; FioSrcL/H plus FioEndL form the 24-bit XRAM staging address. FioSizeL/H returns pixels written.

FIO Status Codes

Value	Name	Meaning
\$00	FioStatusIdle	No operation in progress.
\$02	FioStatusOk	Operation completed successfully.
\$03	FioStatusError	Operation failed; see FioErrCode.

FIO Error Codes

Value	Name	Meaning
\$00	FioErrNone	No error.
\$01	FioErrNotFound	File not found on disk.
\$02	FioErrIo	Host I/O error (invalid name, end address <= start, OS exception).
\$03	FioErrEndOfDir	No more directory entries (returned for DirOpen on empty dir or after last entry).

XMC Register Map

The Expansion Memory Controller occupies BA00–BA3F. The ROM/runtime XMC command processor consumes \$BA00 (XmcCmd) and updates the status registers; the FPGA exposes the registers

and XRAM windows but does not contain a general-purpose hardware allocator. Memory windows (BC00–BFFF) provide direct CPU-bus access to mapped XRAM pages.

XRAM Workspace Conventions

The 512 KB XRAM device is still fully addressable. To avoid a hardware heap, shared runtimes reserve fixed bands by convention:

XRAM range	Use
\$000000–\$03FFFF	BASIC/XMC named-block heap and raw user data.
\$040000–\$04FFFF	NovaZ dynamic story memory workspace.
\$050000–\$053FFF	NovaZ 16 KB static-story page cache.
\$054000–\$05FFFF	NovaZ save/restore scratch workspace.
\$060000–\$06FFFF	Reserved app/runtime workspace.
\$070000–\$07F9FF	NVG decode staging buffer.
\$07FA00–\$07FEFF	XMC allocator metadata (control page, page bitmap, and block directory).
\$07FF00–\$07FFFF	Reserved.

XMC Registers

Address	Name	Access	Description
\$BA00	XmcCmd	R/W	Command byte consumed by the runtime XMC command processor.
\$BA01	XmcStatus	RO	Result status: 0=idle, 2=ok, 3=error.
\$BA02	XmcErrCode	RO	Error detail code (see below).
\$BA03	XmcCfg	R/W	Reserved configuration byte.
\$BA04	XmcAddrL	R/W	XRAM address, low byte.
\$BA05	XmcAddrM	R/W	XRAM address, middle byte.
\$BA06	XmcAddrH	R/W	XRAM address, high byte.
\$BA07	XmcRamL	R/W	CPU RAM address, low byte.
\$BA08	XmcRamH	R/W	CPU RAM address, high byte.
\$BA09	XmcLenL	R/W	Transfer length, low byte.

Address	Name	Access	Description
\$BA0A	XmcLenH	R/W	Transfer length, high byte.
\$BA0B	XmcData	R/W	Byte data port (used by <code>GetByte/PutByte</code>).
\$BA0C	XmcBank	R/W	Default 64 KB bank selector.
\$BA0D	XmcBanks	RO	Total number of 64 KB banks available (read-only).
\$BA0E	XmcPagesUsedL	RO	Used 256-byte pages, low byte.
\$BA0F	XmcPagesUsedH	RO	Used 256-byte pages, high byte.
\$BA10	XmcPagesFreeL	RO	Free 256-byte pages, low byte.
\$BA11	XmcPagesFreeH	RO	Free 256-byte pages, high byte.
\$BA12	XmcNameLen	R/W	Name length for named block operations (1–28).
\$BA13	XmcHandle	R/W	Block handle returned by <code>Alloc/NStash/DirRead</code> .
\$BA14	XmcDirCountL	RO	Count of named blocks, low byte.
\$BA15	XmcDirCountH	RO	Count of named blocks, high byte.
\$BA16	XmcWinCtl	R/W	Window enable bitmask (bit 0=window 0, bit 1=window 1, etc.).
\$BA18	XmcWin0AL	R/W	Window 0 mapped XRAM base address, low byte.
\$BA19	XmcWin0AM	R/W	Window 0 mapped XRAM base address, middle byte.
\$BA1A	XmcWin0AH	R/W	Window 0 mapped XRAM base address, high byte.
\$BA1B	XmcWin1AL	R/W	Window 1 mapped XRAM base address, low byte.
\$BA1C	XmcWin1AM	R/W	Window 1 mapped XRAM base address, middle byte.

Address	Name	Access	Description
\$BA1D	XmcWin1AH	R/W	Window 1 mapped XRAM base address, high byte.
\$BA1E	XmcWin2AL	R/W	Window 2 mapped XRAM base address, low byte.
\$BA1F	XmcWin2AM	R/W	Window 2 mapped XRAM base address, middle byte.
\$BA20	XmcWin2AH	R/W	Window 2 mapped XRAM base address, high byte.
\$BA21	XmcWin3AL	R/W	Window 3 mapped XRAM base address, low byte.
\$BA22	XmcWin3AM	R/W	Window 3 mapped XRAM base address, middle byte.
\$BA23	XmcWin3AH	R/W	Window 3 mapped XRAM base address, high byte.
BA24–BA3F	XmcName	R/W	ASCII name buffer (28 bytes, not null-terminated).

XMC Command Codes

Code	Name	Behavior
\$01	XmcCmdGetByte	Read byte at XmcAddrL/M/H into XmcData .
\$02	XmcCmdPutByte	Write XmcData to XmcAddrL/M/H ; marks page used.
\$03	XmcCmdStash	Copy XmcLenL/H bytes from CPU RAM at XmcRamL/H to XRAM at XmcAddrL/M/H . len=0 is a no-op success.
\$04	XmcCmdFetch	Copy XmcLenL/H bytes from XRAM at XmcAddrL/M/H to CPU RAM at XmcRamL/H . len=0 is a no-op success.
\$05	XmcCmdFill	Fill XmcLenL/H bytes in XRAM starting at XmcAddrL/M/H with XmcData .

Code	Name	Behavior
\$07	XmcCmdStats	Refresh the PagesUsed/PagesFree/DirCount read-only registers.
\$08	XmcCmdResetUsage	Clear all usage tracking, block records, and named-block metadata (destructive).
\$09	XmcCmdRelease	Mark XRAM range (XmcAddrL/M/H , XmcLenL/H) as free; removes any overlapping block records.
\$0A	XmcCmdAlloc	Allocate XmcLenL/H bytes; sets XmcAddrL/M/H , XmcHandle , and XmcBank .
\$0B	XmcCmdNStash	Named stash: create or update named block from CPU RAM; name read from XmcName/XmcNameLen .
\$0C	XmcCmdNFetch	Named fetch: copy named block to CPU RAM at XmcRamL/H ; len=0 fetches full block.
\$0D	XmcCmdNDelete	Delete named block by name.
\$0E	XmcCmdNDirOpen	Open named-block directory; emits first entry to registers.
\$0F	XmcCmdNDirRead	Advance to the next named-block directory entry.

XMC Status Codes

Value	Name	Meaning
\$00	XmcStatusIdle	No operation in progress.
\$02	XmcStatusOk	Operation completed successfully.
\$03	XmcStatusError	Operation failed; see XmcErrCode .

XMC Error Codes

Value	Name	Meaning
\$00	XmcErrNone	No error.
\$01	XmcErrRange	XRAM address or length out of bounds.
\$02	XmcErrBadArgs	Invalid arguments (e.g., len<=0 for Alloc , unknown command).
\$03	XmcErrNotFound	Named block not found.

Value	Name	Meaning
\$04	XmcErrNoSpace	No contiguous free pages of the required size, or handle pool exhausted.
\$05	XmcErrName	Name length is 0 or exceeds 28, or name is blank after trimming.
\$06	XmcErrEndOfDir	No more named-block directory entries.

System Vectors

The address range 0200–027F is the system vector table. Each entry is a 16-bit little-endian pointer initialized from ROM at cold start. BASIC uses the lower portion; the upper portion is reserved for future use.

Address	Purpose
0200–0202	Ctrl-C flag, byte, and timeout.
0203–0204	VEC_CC – Ctrl-C check vector.
0205–0206	VEC_IN – input device vector.
0207–0208	VEC_OUT – output device vector.
0209–020A	VEC_LD – load program vector.
020B–020C	VEC_SV – save program vector.
020D–0216	IRQ handler code area. Set by the IRQ statement.
0217–0220	NMI handler code area. Set by the NMI statement.
0221–027F	Reserved for future system use.

Note

The vector layout at 0200–020C is inherited from EhBASIC 2.22p5. Hardware controller base addresses are fixed constants; see the Address Space Overview table at the start of this appendix.

Limits, Errors, and Edge Cases

This appendix documents all known numeric limits, argument validation rules, and edge-case behaviors derived from a direct audit of the ROM source and the Avalonia hardware controller implementations. Where ROM behavior and host behavior differ, both are noted.

Numeric Argument Conversion Rules

Most command arguments are passed through one of two shared ROM helpers before reaching the hardware layer. Understanding their constraints prevents unexpected function-call errors.

Helper	Behavior
LAB_GTBY	Converts the FAC (floating-point accumulator) to an unsigned byte. Accepts values 0–255. Any value outside this range, or any negative value, raises a function-call error before the command reaches the hardware.
LAB_GTWRD	Converts the FAC to an unsigned 16-bit integer. Accepts values 0–65535. Negative values or values above 65535 raise a function-call error.

Warning

Commands that accept addresses (POKE, DOKE, SYS, WAIT, STASH, FETCH...) route through LAB_GTWRD. Passing a value such as -1 will raise an error rather than wrapping to \$FFFF.

Variable Name Limits

Topic	Behavior
Significant characters	BASIC variable names may be written with more than two characters, but only the first two characters identify the variable. SC, SCORE, and score are the same numeric variable.
Case	Variable names are case-insensitive outside quoted strings.
Strings	A trailing \$ makes a string variable. SC and SC\$ are separate variables.
Arrays and DEF FN	Array names, DEF FN names, and function parameters use the same two-character variable-name rule.
Recommended style	Use one- or two-character mnemonics in listings. Reserve a prefix, such as Z*, for shared framework code that must not collide with a game program.

File Command Limits

Topic	Behavior
Filename length	The ROM filename parser accepts 1–63 characters. A length of 0 or greater than 63 causes the FIO controller to return an I/O error (the ReadFilename guard in <code>FileIoController.cs</code>).

Topic	Behavior
Allowed filename characters	The host implementation enforces the pattern <code>[A-Za-z0-9_.\-]+</code> . Any character outside this set causes <code>ReadFilename</code> to return <code>null</code> and the operation to fail with <code>FioErrIo</code> .
<code>.bas</code> extension	If the filename does not already end in <code>.bas</code> (case-insensitive), the host appends it automatically before forming the filesystem path.
Missing file	<code>LOAD "name"</code> and <code>DEL "name"</code> on a non-existent file set <code>FioStatus=03</code> and <code>FioErrCode=01</code> (<code>FioErrNotFound</code>). The ROM interprets this as “File not found”.
I/O fault	Any OS-level exception during <code>SAVE/LOAD/DEL</code> sets <code>FioStatus=03</code> and <code>FioErrCode=02</code> (<code>FioErrIo</code>). The ROM surfaces this as “I/O Error”.
<code>SAVE</code> end address	If <code>FioEndH:FioEndL <= FioSrcH:FioSrcL</code> , the save is rejected immediately with <code>FioErrIo</code> before the file is opened.
DIR on empty catalog	<code>DirOpen</code> sets <code>FioStatus=03/FioErrCode=03</code> (<code>FioErrEndOfDir</code>) when no <code>.bas</code> files exist. The ROM DIR handler treats this as a silent empty listing.
DIR after last entry	Each <code>DirRead</code> beyond the final file sets <code>FioErrEndOfDir</code> ; the ROM stops iterating.

Graphics and Sprite Edge Cases

Command / Feature	Limit and edge behavior
<code>GCOLOR c</code>	Only the low nibble of <code>c</code> is used (<code>c & 0x0F</code>). Values 0–15 are valid; values above 15 wrap silently into the 0–15 range.
Color 0 on the graphics layer	Color index 0 means transparent on the graphics bitmap. <code>UNPLOT x,y</code> explicitly sets a pixel to 0 to restore transparency.
<code>PLOT/UNPLOT/PAINT</code> bounds	The ROM dispatches coordinates without host-side pre-clipping. The host <code>BlockGraphics</code> implementation performs pixel-level bounds checks; out-of-range coordinates are silently ignored.
<code>LINE/RECT/CIRCLE/FILL</code> bounds	Drawing operations are clipped by the host renderer (<code>BlockGraphics.cs</code>). Portions of the shape outside the 320x200 pixel area are dropped; no error is raised.

Command / Feature	Limit and edge behavior
FILL rectangle	Coordinates are clamped to the screen boundary before drawing; the swap of x0/x1 or y0/y1 to ensure a positive rectangle is handled by the host.
SPRITE <i>n</i> , ... invalid index	Sprite index <i>n</i> must be 0–15. The VGC command handlers check <i>n</i> >= MaxSprites and return immediately without error; no ROM-level error is raised.
SPRITEDATA <i>n</i> ,row,...	Row must be 0–15. The CmdSprRow handler checks both sprite index and row; an invalid row causes the command to be silently ignored.
SPRITESHape	Tokenised and recognised by the ROM parser; writes the shape slot field in the sprite register block at \$A044 + <i>n</i> *8.
SPRITESET <i>n</i> ,field,value	Field must be 0–7. Field 0 accepts an unsigned 16-bit X value; other fields accept byte values. Sprite attribute writes are buffered by hardware and published at the frame boundary.
Sprite coordinates	Sprite positions are Nova canvas coordinates. X=0–319 and Y=0–199 are visible; a 16x16 sprite is fully visible at X=0–304, Y=0–184. Out-of-range positions are clipped and do not draw into the border.
SPRITE _X (<i>n</i>) / SPRITE _Y (<i>n</i>)	SPRITE_X returns the unsigned 16-bit Nova canvas X register; SPRITE_Y returns the unsigned 8-bit Nova canvas Y register. Y high is reserved and reads as 0.
SPRCOLL/SPRBG	The VGC updates 16-bit collision masks each frame. RegColSt/RegColBg hold sprites 0–7, and RegColStHi/RegColBgHi hold sprites 8–15. SPRCOLL and SPRBG read and clear the full masks.
Default sprite priority	On reset all sprites default to priority 2 (in front of everything). This matches the SpritePriInFront constant.
CmdSprFlip flags	Only bits 0–1 of the flags byte are used (flags & 0x03): bit 0 = horizontal flip, bit 1 = vertical flip.
CmdSprPri clamping	Priority values above 2 are clamped to 2 (Math.Min(value, 2)). Values 0, 1, and 2 are the only meaningful levels.

Sound and Music Limits

Command / Feature	Limit and edge behavior
SOUND <i>note</i> , <i>dur</i> [, <i>inst</i>]	<i>note</i> is a MIDI note byte. <i>dur</i> is duration in video frames (0–255 byte range). <i>inst</i> is instrument slot (0–15, default 0). Duration 0 starts no new sound. Active SOUND durations advance when BASIC runs VSYNC.
Master volume	Only the low nibble of the volume byte is used (<i>level</i> & 0x0F). The SID master volume powers on at 0, so programs should set VOLUME before playing sound.
INSTRUMENT parameters	All six parameters are bytes (0–255). Waveform should be one of 10,20, 40,80. ADSR values 0–15 are meaningful; higher values use the low nibble only for sustain.
INSTRUMENT slots	16 slots (0–15). Slot 0 is pre-initialized at boot. All other slots start as copies of slot 0.
MUSIC <i>voice</i> ,...	Voice must be 1–6. The MML string is read from CPU memory via pointer; maximum practical length limited by available RAM.
MUSIC TEMPO	BPM is a 16-bit value (0–65535). Default is 120.
MUSIC PRIORITY	1–6 voice numbers. Applies to hosted MML playback on hosts that support priority routing; simple SID SOUND uses its own round-robin voice scheduler.
MML pulse width	Range 0–4095. Default 2048. PWM sweep step: +/-32 per frame.
MML filter cutoff	Range 0–2047. Resonance 0–15. Filter sweep step: +/-8 per frame.
MML vibrato	Depth is any positive integer; 0 = off. Oscillates at 2.9 Hz.
MML loops	Non-nesting. Maximum practical depth limited by string expansion.
Copper events	Maximum 256 events per list; 128 lists available. Events at duplicate position/register replace existing values.
WAVE	Deprecated. Raises a syntax error. Use INSTRUMENT instead.
File-backed SID/MIDI playback	SIDPLAY, SIDSTOP, MIDPLAY, MIDSTOP, and SFLOAD use the shared FIO status/error contract. On unsupported hosts they report the FIO unsupported/error status instead of silently succeeding.

XRAM Limits and Failure Modes

Command / Feature	Limit and edge behavior
<code>XBANK n</code>	The ROM verifies that <code>n</code> is less than the value stored at <code>XmcBanks</code> (\$BA0D). An out-of-range bank number triggers a function-call error in the ROM before any XMC command is issued.
Window number (<code>XMAP/XUNMAP</code>)	Window must be 0–3. The ROM validates this; an invalid window triggers a function-call error.
Window address space	The four CPU-visible windows occupy BC00–BFFF (4 x 256 bytes). Window 0 maps to BC00, window 1 to BD00, window 2 to BE00, window 3 to BF00.
Unmapped window reads/writes	If a window is not enabled in <code>XmcWinCtl</code> , the XMC does not own that address and the read/write falls through to flat RAM. No error is returned.
Named block name length	The ROM enforces a 1–28 byte name (<code>XmcNameLen</code> is capped at 28 by the name buffer size: BA24–BA3F = 28 bytes). The host trims whitespace; a blank name after trimming is rejected with <code>XmcErrName</code> .
Named block name case	Name lookup is case-insensitive in the host (<code>StringComparer.OrdinalIgnoreCase</code>). Storing “SPRITE” and retrieving “sprite” will succeed.
<code>XALLOC len</code> with <code>len<=0</code>	The ROM passes zero through <code>LAB_GTWDRD</code> , which itself rejects negative values. The XMC command handler rejects <code>len<=0</code> with <code>XmcErrBadArgs</code> .
<code>XALLOC</code> with no free space	If the low-DRAM software heap cannot satisfy the requested run of pages, the command fails with <code>XmcErrNoSpace</code> .
<code>STASH/FETCH (raw)</code> with <code>len=0</code>	The XMC host treats a zero-length raw transfer as a no-op and returns <code>XmcStatusOk</code> . No data is moved and no pages are marked.
<code>FETCH "name",ram</code> (named fetch)	The ROM sends <code>XmcLenL/H = 0</code> for the named-fetch command. The host interprets <code>requested=0</code> as “fetch the entire stored block”: <code>len = (requested <= 0) ? block.Length : min(requested, block.Length)</code> .
<code>STASH "name",ram,len</code> over existing block	If the named block already exists and the new length fits in the allocated pages, only <code>block.Length</code> is updated (no reallocation). If it does not fit, the old block is freed and a new allocation is attempted.

Command / Feature	Limit and edge behavior
XRESET	Clears the <code>_usedPages</code> array, resets <code>_usedPageCount</code> to 0, and removes all block and name records. The raw XRAM byte array is <i>not</i> zeroed; data remains but is inaccessible through the allocation system.
XFREE <code>off,len</code>	Frees all usage-tracking pages in the given range and removes any tracked blocks (named or unnamed) whose page range overlaps with the freed region.
XPOKE/XPEEK bank offset	The ROM constructs the XRAM address as <code>bank * 65536 + offset</code> . The host validates the resulting 24-bit address against the total XRAM size; out-of-range addresses return <code>XmcErrRange</code> .
RAM range validation for STASH/FETCH	The host prevents writes to ROM space: any <code>FETCH</code> operation whose destination range would extend into \$C000 or above is rejected with <code>XmcErrRange</code> . Reads (<code>STASH</code>) may source from ROM addresses, allowing code capture.

DMA and Blitter Limits

Command / Feature	Limit and edge behavior
Memory space IDs	Valid range: 0–5 (CPU RAM, Char, Color, Gfx, Sprite, XRAM). An invalid space ID sets the error code to <code>BadSpace</code> (\$02).
24-bit addressing	All DMA and blitter addresses are 24-bit (low/mid/high byte). For XRAM (space 5), the current <code>XBANK</code> value is used as the high address byte automatically by the BASIC commands.
DMACOPY/DMAFILL	Length is 24-bit; zero-length transfers are rejected with <code>BadArgs</code> (04). Out-of-range addresses set <code>Range</code> (03). Writes to ROM space set <code>WriteProt</code> (\$05).
BLITCOPY/BLITFILL	Width and height are 16-bit (1–65535 each). Zero width or height is rejected with <code>BadArgs</code> (\$04). Stride is the byte offset between row starts; it may be larger than width.
Color-key mode (blitter)	When bit 1 of <code>BltMode</code> is set, source bytes matching <code>BltColorKey</code> are skipped. In fill mode, color-key is ignored.
Row buffer	When source and destination overlap in the same space, the blitter uses a temporary row buffer to prevent read-after-write corruption.

Command / Feature	Limit and edge behavior
DMASTATUS/BLITSTATUS	Returns the raw status register value: 0=idle, 1=busy, 2=ok, 3=error. BASIC commands check status automatically and raise a function-call error if the operation fails.

Network Limits

Feature	Limit and edge behavior
Connection slots	4 slots (0–3). Each slot is an independent outbound client connection.
Message size	Maximum 256 bytes per message. Messages are length-prefixed on the wire (1 byte: 00 = 256, 01–FF = 1 – 255). <i> Receivequeue Upto16messagesperslot.Ifthequeueover bytebuffer,null – terminated). Portrange 1 – –65535(16 – bit). Connecttimeout 10seconds.Iftheremotehostdoesnotresp</i> Returns a string of up to 255 characters. If no message is queued, returns an empty string. NSTATUS(slot) Returns the slot status byte. Bit 4 (RemoteClosed) indicates the remote peer has disconnected; the slot should be closed with NCLOSE. NREADY(slot)‘

Status and Error Code Quick Reference

File I/O Controller (FIO) Codes

Value	Symbol	Meaning
\$00	FioStatusIdle	No operation has been issued since last reset.
\$02	FioStatusOk	Last operation succeeded.
\$03	FioStatusError	Last operation failed; check error code.
\$00	FioErrNone	No error.
\$01	FioErrNotFound	File not found on disk.
\$02	FioErrIo	Host I/O error (bad name, OS exception, end address <= start).
\$03	FioErrEndOfDir	Directory enumeration exhausted.

Expansion Memory Controller (XMC) Codes

Value	Symbol	Meaning
\$00	XmcStatusIdle	No operation in progress.
\$02	XmcStatusOk	Last operation succeeded.
\$03	XmcStatusError	Last operation failed; check error code.
\$00	XmcErrNone	No error.
\$01	XmcErrRange	XRAM address or transfer endpoint out of XRAM bounds, or FETCH would write into ROM.
\$02	XmcErrBadArgs	Invalid argument (<code>len</code> ≤ 0 for Alloc, unknown command byte).
\$03	XmcErrNotFound	Named block not found in directory.
\$04	XmcErrNoSpace	No contiguous free pages available, or handle pool (1–255) exhausted.
\$05	XmcErrName	Name length 0 or >28, or name is blank after trimming.
\$06	XmcErrEndOfDir	No more named-block directory entries.

DMA Controller Codes

Value	Symbol	Meaning
\$00	DmaStatusIdle	No operation in progress.
\$01	DmaStatusBusy	Transfer is executing.
\$02	DmaStatusOk	Last transfer succeeded.
\$03	DmaStatusError	Last transfer failed; check error code.
\$00	DmaErrNone	No error.
\$01	DmaErrBadCmd	Invalid command byte.
\$02	DmaErrBadSpace	Invalid source or destination space ID.
\$03	DmaErrRange	Address out of bounds for the given space.
\$04	DmaErrBadArgs	Invalid arguments (e.g., length ≤ 0).
\$05	DmaErrWriteProt	Destination is write-protected (ROM).

Blitter Controller Codes

Value	Symbol	Meaning
\$00	BltStatusIdle	No operation in progress.
\$01	BltStatusBusy	Blit is executing.
\$02	BltStatusOk	Last blit succeeded.

Value	Symbol	Meaning
\$03	BltStatusError	Last blit failed; check error code.
\$00	BltErrNone	No error.
\$01	BltErrBadCmd	Invalid command byte.
\$02	BltErrBadSpace	Invalid source or destination space ID.
\$03	BltErrRange	Rectangular region extends out of bounds.
\$04	BltErrBadArgs	Invalid arguments (width ≤ 0 , height ≤ 0).
\$05	BltErrWriteProt	Destination is write-protected (ROM).

Tip

The BASIC runtime maps FIO, XMC, DMA, and Blitter error returns to one of three user-visible messages: “**File not found**”, “**I/O Error**”, or a function-call error. For low-level programs that POKE the controller registers directly, use the tables above to interpret the raw status and error bytes.